Geometric bipartite matching exact offline server algorithms with subquadratic nearest-neighbor queries

Sharath Raghvendra, Pouyan Shirzadian, Rachita Sowle

arxiv:2504.06079; CC BY 4.0

Editorial scope. Only designated main theorem counted. Complete author native main text and appendices A-E preserved under arxiv2504.06079v1 own CC BY4.0. Original figures and bibliography retained. Comments omitted and inputs inlined; title and bibliography commands replaced mechanically by wrapper. Other results uncounted.

Allowed standard prerequisites (editorial).

- Minimum-cost matching, Hungarian primal-dual searches and dynamic weighted nearest-neighbor data structures. Full author definitions and bibliography retained; all present source-specific algorithms and explicit appendices included.

1 Introduction

This paper considers two classical optimization problems: the *offline k-server* problem and the *minimum-cost bipartite matching* problem in geometric settings.

Offline k-server problem and its variant. Consider a sequence of requests $\varsigma = \langle r_1, \ldots, r_m \rangle$ in a metric space equipped with the cost function $\mathrm{d}(\cdot, \cdot)$. The cost of a single server *servicing* the requests in ς is the sum of the distances between every consecutive pair of points in the sequence, i.e., $¢(\varsigma) = \sum_{i=1}^{m-1} \mathrm{d}(r_i, r_{i+1})$. Given a sequence $\sigma = \langle r_1, \ldots r_n \rangle$ of n requests and an integer $1 \leq k \leq n$, the *k-sequence partitioning problem* (or simply the k-SP problem) requires partitioning the requests in σ into k subsequences $\varsigma_1, \ldots, \varsigma_k$ so that $\sum_{i=1}^{k} ¢(\varsigma_i)$ is minimized. The optimal solution for the k-sequence partitioning problem is the cheapest way for k servers to service all of the requests in σ. We also consider a variant: the *k-sequence partitioning with initial locations problem* (or the k-SPI problem). Here, in addition to the requests σ, we are also given the initial locations $\eta = \langle s_1, \ldots, s_k \rangle$ for the k servers. The objective of this problem is to partition $\sigma' = \eta\sigma$ into k subsequences $\varsigma_1, \ldots, \varsigma_k$ so that the initial location s_j of server j appears in the first element of the subsequence ς_j and the cost $\sum_{i=1}^{k} ¢(\varsigma_k)$ is minimized. The optimal solution to the k-SPI problem is the offline optimal solution to the well-known k-server problem.

We assume that the requests in σ and the initial locations in η are scaled and translated so that they are contained inside the unit hypercube $[0, 1]^d$. Such scaling and translation do not impact the optimal solutions for the k-SP and k-SPI problems. We define the diameter $\mathrm{Diam}(\sigma)$ to be the largest distance between any

two request locations in σ and the *closest pair distance*, denoted by $\mathrm{CP}(\sigma)$, to be the smallest non-zero distance between any two requests in σ. The *spread*, denoted by Δ, is the ratio of the diameter to the closest-pair distance, i.e., $\Delta = \mathrm{Diam}(\sigma)/\mathrm{CP}(\sigma)$.

Minimum-cost bipartite matching. Consider a weighted bipartite graph $G(A \cup B, E \subseteq A \times B)$, where each edge between u and v is assigned a real-valued cost. Let $n := \min\{|A|, |B|\}$. A *matching* of size $t \leq n$, or simply a *t-matching*, is a set of t edges in G that are vertex-disjoint. The *cost* of any matching M is the sum of the costs of its edges. Given a parameter $t \leq n$, the *minimum-cost bipartite t-matching* problem seeks to find a t-matching with a minimum cost. When $|A| = |B| = n$, a matching of size n is called a *perfect matching*. When A and B are d-dimensional point sets and the cost of any edge between $a \in A$ and $b \in B$ is the ℓ_p distance $\|a - b\|_p$, the problem of finding the minimum-cost bipartite t-matching is also called the (partial) *geometric bipartite matching problem*.

Relating the two problems. Chrobak et al. [5] established a reduction from the minimum-cost bipartite t-matching problem to the k-SPI problem.

Lemma 1.1. *[5, Theorem 11] Any algorithm that computes an optimal solution to the n-SPI in an arbitrary metric space in $T(n)$ time can also find, in $T(n) + O(n^2)$ time, a minimum-cost perfect matching in any complete bipartite graph with real-valued costs.*

We strengthen the connection between the two problems by showing a reduction in the reverse direction, i.e., we reduce the k-SP (resp. k-SPI) problem to the minimum-cost bipartite t-matching problem. Given an input sequence σ of requests to the k-SP problem, we construct a bipartite graph $\mathcal{G}_\sigma$ with a vertex set $A \cup B$ and a set of edges $\mathcal{E}$ as follows. *Vertex Set:* For each request r_i, we create a vertex b_i (resp. a_i) in B (resp. A) and designate it as the *entry* (resp. *exit*) gate for request r_i. *Edge Set:* The exit gate a_i of request r_i is connected to the entry gate b_j of every subsequent request r_j with $j > i$ with an edge. The cost of this edge is $\mathrm{d}(a_i, b_j) = \|r_i - r_j\|_p$. It is easy to see that a minimum-cost $(n-k)$-matching M in $\mathcal{G}_\sigma$ can be used to find an optimal solution to the k-SP problem. See Figure 1.

For the k-SPI problem, for each server j, we add a vertex a^j at the initial location s_j to A and connect a^j to the entry gate b_i of every request r_i in σ. The cost of this edge is $\mathrm{d}(a^j, b_i) = \|s_j - b_i\|_p$. A minimum-cost n-matching in this graph can be converted to an optimal solution for the k-SPI problem. The formal proof of correctness for both these reductions is presented in Appendix A.1.

A different reduction from k-SP and k-SPI problems to the minimum-cost flow problem has been presented in previous works [5, 25, 24], leading to the development of $O(n^2 k)$ time algorithm. However, unlike our reduction, their approach generates instances that include edges whose costs are $-\infty$ and fails to maintain the metric properties of costs.

Lemma 1.2. *An optimal solution for an instance σ (resp. σ') of k-SP (resp. k-SPI) problem can be found by computing a minimum-cost matching of size $n - k$ (resp. n) in $\mathcal{G}_\sigma$ (resp. $\mathcal{G}_{\sigma'}$).*

Finally, we extend the reduction Chrobak et al. [5] to geometric settings and provide a reduction from the geometric minimum-cost matching problem under the ℓ_1-norm to the geometric version of the n-SPI problem. This reduction, however, creates an instance of the n-SPI problem with a spread of 3^n (See Appendix A.2 for details). Lemma 1.3 follows directly from this reduction.

Lemma 1.3. *Any algorithm that can solve the d-dimensional n-SPI problem under the ℓ_1 costs in $T(n)$ time can also be used to solve an instance of minimum-cost bipartite matching under the ℓ_1 costs on a complete graph in $T(n)$ time.*

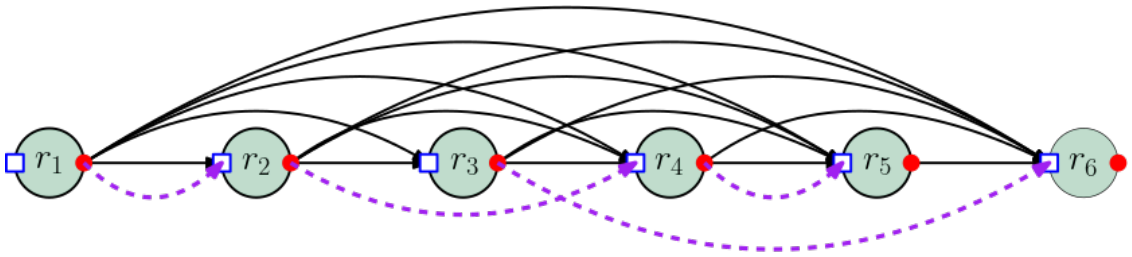


Figure 1: The graph $\mathcal{G}_\sigma$ constructed for $\sigma = \langle r_1, r_2, \ldots, r_6 \rangle$. The vertex set A (red disks) and B (blue squares) represent the exit and entry gates of each request, and the purple dashed lines show an $(n-2)$-partial matching on $\mathcal{G}_\sigma$ representing a 2-partitioning $\langle r_1, r_2, r_4, r_5 \rangle$ and $\langle r_3, r_6 \rangle$.

Related work. For a graph with m edges and n vertices, the classical Hungarian algorithm computes a minimum-cost t-matching for all values of $0 \leq t \leq n$ [14, 19]. The algorithm begins with an empty matching M, and in each iteration i, updates a minimum-cost $(i-1)$-matching to a minimum-cost i-matching in $O(m + n \log n)$ time by finding a *minimum net-cost augmenting path*, i.e., an augmenting path that increases the matching cost by the smallest value. The overall execution time of the Hungarian algorithm is $O(nm + n^2 \log n)$, or $O(n^3)$ when $m = \Theta(n^2)$. Despite substantial efforts, this remains the most efficient algorithm for the problem. Notable exceptions include specialized cases, such as graphs with small vertex separators [16] as well as graphs where the edge weights are integers [3, 7, 8].

In geometric settings, Vaidya showed that each iteration of the Hungarian algorithm can be implemented in $\tilde{O}(n\Phi(n))$ time, where $\Phi(n)$ represents the query/update time of a dynamic weighted nearest neighbor (DWNN) data structure with respect to the edge costs. Thus, the minimum-cost t-matching can be computed in $O(n^2\Phi(n))$ time, which is sub-cubic in n provided $\Phi(n)$ is sub-linear. For instance, for the ℓ_1 norm and d dimensions, $\Phi(n) = O(\log^d n)$ [29] and for the ℓ_2 norm and 2 dimensions, $\Phi(n) = \log^{O(1)} n$ [11]. In these cases, the Hungarian algorithm can be implemented in near-quadratic time.

The design of algorithms that compute a minimum-cost matching with a sub-quadratic number of queries to a DWNN data structure remains an important open problem in computational geometry. There are three notable exceptions to this. First, for points with integer coordinates and the ℓ_1 and ℓ_∞ norms, the edge costs are integers. Using this fact, Sharathkumar and Agarwal [28] adapted an existing cost-scaling-based graph algorithm [7] and presented an algorithm that executes in $\tilde{O}(n^{3/2}\Phi(n))$ time. Second, Sharathkumar [27] extended this result to two-dimensional point sets with integer coordinates and the ℓ_2 costs. Their result, however, relies on the points being planar as well as the edge costs being the square root of integers and does not extend to d-dimensional points with real-valued coordinates. Third, Gattani et al. [9] presented a divide-and-conquer algorithm (GRS algorithm) to compute a minimum-cost perfect matching. The worst-case execution time of their algorithm is $\tilde{O}(n^2\Phi(n)\log\Delta)$; here, Δ is the spread of the point sets A and B. For *stochastic* points sampled from an unknown distribution μ in two dimensions, the algorithm finds the minimum-cost matching in $\tilde{O}(n^{7/4}\Phi(n)\log\Delta))$ time in expectation.

The GRS algorithm uses a (randomly-shifted) quadtree to divide the problem into smaller sub-problems. Within each square $\square$ of the quadtree, their algorithm recursively computes a *minimum-cost extended matching*, where each point of B inside $\square$ can either match to another point of A inside $\square$ or to the boundary of $\square$ (Figure 2(left)). The algorithm combines minimum-cost extended matchings of the child squares by *erasing* their common boundary, freeing all points of B that matched to this boundary (Figure 2(middle)), and iteratively matching the freed points.

Interestingly, Gattani et al. [9] showed that when A and B are samples from the same distribution μ,

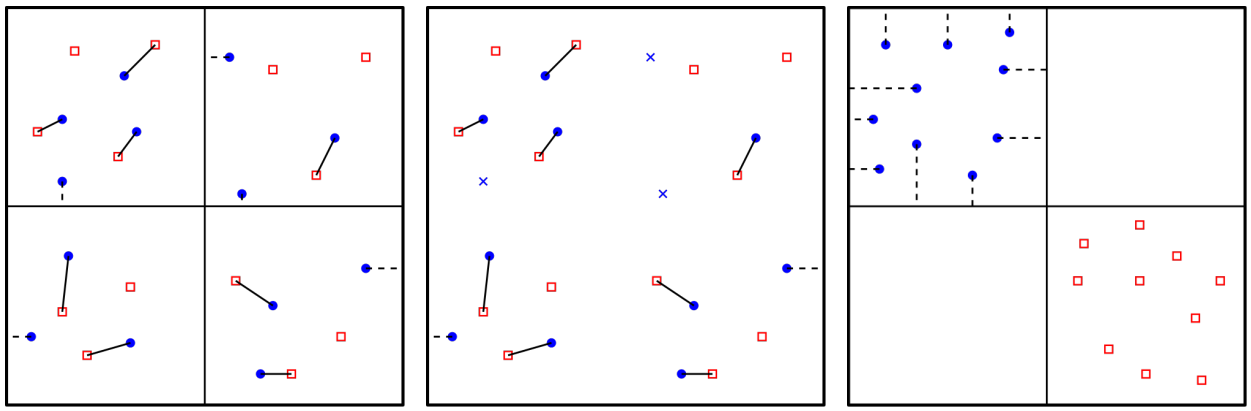


Figure 2: Illustration of the GRS algorithm.

most points of B will match to a close-by point in A, leaving only $\tilde{O}(n^{3/4})$ many points in B matching to the boundary. Thus, erasing the common boundary only creates $\tilde{O}(n^{3/4})$ free points, each of which can be matched in $O(n\Phi(n))$ time, leading to an overall execution time of $\tilde{O}(n^{7/4}\Phi(n))$. However, this efficiency does not extend to arbitrary point sets. For instance, suppose all points of A are in a child $\square_1$ and all points of B are in another child $\square_2$. In this case, all edges of the minimum-cost matching cross the boundaries of $\square_1$ and $\square_2$. The minimum-cost extended matching at $\square_2$, therefore, will match every point of B to the boundary of $\square_2$ (See Figure 2(right)). Erasing the common boundary between $\square_1$ and $\square_2$ creates n free points, causing the conquer step to take $\Omega(n^2\Phi(n))$ time. Furthermore, unlike the Hungarian algorithm, the GRS algorithm does not guarantee the optimality of intermediate matchings. Therefore, it cannot be used to produce minimum-cost t-matchings.

Our results. The optimal solutions to the k-SP (resp. k-SPI) problems can be computed in $\tilde{O}(nk\Phi(n))$ time by non-trivially adapting the Hungarian algorithm to find a minimum-cost $(n-k)$-matching (resp. n-matching) in $\mathcal{G}_\sigma$ (resp. $\mathcal{G}_{\sigma'}$). However, it is worth noting that this algorithm still makes quadratic queries to the DWNN data structure when $k = \Theta(n)$. The main contribution of this paper is the design of a novel algorithm that, for any k, computes the optimal solution to the k-SP and k-SPI problems while making only a sub-quadratic number of queries to the DWNN data structure.

Theorem 1.4. *Given any sequence σ (resp. $\sigma' = \eta\sigma$) of n requests (resp. n requests and k initial server locations) in 2 dimensions with a spread of Δ, and a value $1 \le k \le n$, there exists a deterministic algorithm that computes the optimal solution for the instance of k-SP (resp. k-SPI) problem under the ℓ_p norm in $\tilde{O}(\min\{nk, n^{1.8}\log\Delta\}\cdot\Phi(n))$ time.*

Our algorithm also extends to higher dimensions, and for any dimension $d \ge 2$, it computes optimal solutions to the k-SP and k-SPI problems in $\tilde{O}(\min\{nk, n^{2-\frac{1}{2d+1}}\log\Delta\}\cdot\Phi(n))$ time (Theorem C.6 in Appendix C.5).

Developing algorithms for geometric bipartite matching that perform sub-quadratic queries to a DWNN data structure is a challenging task. Nevertheless, it follows from Theorem 1.4 that the bipartite matching instances generated by k-SP and k-SPI problems (with bounded spread) can be solved by making sub-quadratic queries to a DWNN data structure. Solving instances with unbounded spread with sub-quadratic queries to DWNN, at least for the k-SPI problem, remains challenging. This is because, as established in Lemma 1.3, any such algorithm can find the d-dimensional minimum-cost bipartite matching algorithm under ℓ_1 costs in sub-quadratic time.

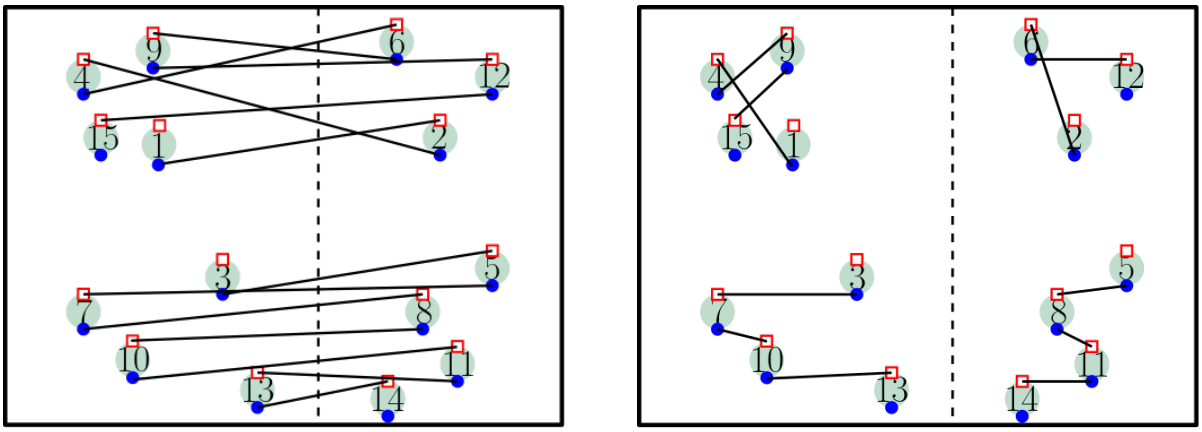


Figure 3: (left) A sub-problem of the k-SP problem, where the optimal solution has a high cost, and (right) there exists a low-cost high-cardinality matching inside the sub-problems.

Our algorithm uses a hierarchical partitioning tree, whose nodes (referred to as cells) are axis-parallel rectangles with an aspect ratio of at most 3. Each cell is divided into two smaller rectangles, forming its children. At any point during the execution of the algorithm, it maintains a set of *current* cells $\mathcal{C}$ that partition the input points. It computes a minimum-cost extended $(n-k)$-matching where points of B are allowed to match to the boundaries of these cells by repeatedly identifying a minimum net-cost augmenting path and augmenting the matching along this path. Once a minimum-cost extended $(n-k)$-matching is computed, the algorithm removes a pair of sibling rectangles from the set of current cells and instead makes their parent cell current. By doing so, the common boundary between the siblings is erased, creating additional free points, which are then matched again by finding the minimum net-cost augmenting paths. When all the boundaries are erased, the algorithm terminates with the desired minimum-cost $(n-k)$-matching.

There are three major hurdles in proving the efficiency of our algorithm. The first challenge is in finding a minimum net-cost augmenting path, which typically requires a search on the entire graph. However, for extended matchings, we show that the minimum net-cost augmenting path is fully contained inside one of the current cells. This significantly improves efficiency, as the search can be limited to individual current cells, and the overall minimum can then be determined by selecting the path with the smallest net-cost among all current cells. The second challenge is in bounding the time to merge two cells. Similar to the worst-case example for the GRS algorithm, the k-SP (or k-SPI) may have current cells with $\Theta(n)$ points, where the optimal solution matched each point of B to a point of A that is very far (see Figure 3 (left)). Despite this, we show that any minimum-cost extended matching has only $O(n^{0.8})$ points that are matched to their common boundary, helping us in bounding the time required to merge cells. To establish this, we critically use the fact that every sub-problem has a hidden low-cost high-cardinality matching M (with only sub-linearly many free points, see Figure 3 (right)).

The final challenge in the analysis is the following. The k free points associated with the extended $(n-k)$-matching M maintained by our algorithm may differ from the k free points in the minimum-cost $(n-k)$ matching M^*. For instance, a point $b \in B$ may be free in M^* but matched to the boundary in M, thereby leaving some other point b' unmatched. Our algorithm corrects them via alternating and augmenting paths, each of which can take $\Theta(n)$ time to find. Since there can be k such corrections, a naïve analysis leads to an upper bound of $O(nk)$. Surprisingly, by exploiting geometry, we show that the number of such corrections inside any current cell cannot exceed $O(n^{0.8})$. Using this observation, we bound the overall execution time of our algorithm by $\tilde{O}(n^{1.8}\Phi(n)\log\Delta)$.

Our proof techniques also improve the analysis of the GRS algorithm for the case where the cost between points $a \in A$ and $b \in B$ is given by $\|a-b\|_2^q$ for any $q \geq 2$. Previous work established that the GRS algorithm computes the minimum-cost perfect matching between two-dimensional stochastic points in $\tilde{O}(n^{2-\frac{1}{2(q+1)}}\Phi(n))$ time [9]. Notably, as q approaches ∞, this analysis implies a quadratic number of queries to the DWNN data structure. Raghvendra et al. [23] recently observed that, for any $q \geq 1$, there is a low-cost high-cardinality matching with only sub-linearly many free points between stochastic point sets. By incorporating this observation with the novel analysis techniques in our paper, we show that the GRS algorithm makes only a sub-quadratic number of queries to the DWNN data structure, regardless of the value of q.

Theorem 1.5. *Suppose U is a set of $2n$ points inside the unit square and A is a subset chosen uniformly at random from all subsets of size n. Let $B = U \setminus A$. Then, there exists an algorithm that computes the minimum-cost perfect matching on the complete bipartite graph on A and B under ℓ_2^q costs in $\tilde{O}(n^{7/4}\Phi(n)\log\Delta)$ expected time.*

Note that the setting considered by Gattani et al. [9] in the analysis of the GRS algorithm, where the two sets A and B are i.i.d samples from the same distribution, is a special case of randomly partitioning a set of $2n$ points into two sets A and B of n points, considered in Theorem 1.5.

Organization. In Section 2, we establish our primal-dual framework for the minimum-cost bipartite matching problem and use it in Section 3 to present our sub-quadratic algorithms for the k-SP and k-SPI problems. We present our fast implementation of the Hungarian algorithm in Section 4 and the improved analysis of the GRS algorithm in Section 5. We conclude the paper by discussing some open questions in Section 6.

2 Geometric Primal-Dual Framework

Let σ be an input to the k-SP problem, where the distance between two locations a and b is given by $\mathrm{d}(a,b) = \|a-b\|_p$. In this section, we introduce a primal-dual framework based on hierarchical partitioning to compute a minimum-cost $(n-k)$-matching in $\mathcal{G}_\sigma = (A \cup B, E \subset A \times B)$. We begin by describing the hierarchical partitioning scheme.

2.1 Hierarchical Partitioning

Using $\lambda := 9n^{-1/5}$, we construct a hierarchical partitioning $\mathcal{H}$ recursively. Each node of $\mathcal{H}$ is an axis-parallel rectangle, referred to as a *cell*. The root node, $\square^* := [-3n, 3n]^2$, contains all points in $A \cup B$. For each node $\square$, let $A_\square$ and $B_\square$ be the points of A and B inside $\square$ and let $n_\square = |A_\square \cup B_\square|$. If $n_\square \leq 2$ (i.e., $\square$ is empty or contains the entry and exit gates of a single request), then $\square$ is marked as a leaf node. Otherwise, we partition $\square$ into two smaller rectangles as follows. Let $\ell_\square$ be the larger of the length and width of rectangle $\square$. Without loss of generality, assume that $\ell_\square$ is the width of $\square$ and let $x_{\min}$ be the x-coordinate of the bottom-left corner of $\square$. For any value $\hat{x} \in [x_{\min}+\frac{\ell_\square}{3}, x_{\min}+\frac{2\ell_\square}{3}]$, define $\Lambda(\hat{x}) := \{u \in A_\square \cup B_\square : |u_x - \hat{x}| \leq \ell_\square \lambda\}$; here, u_x denotes the x coordinate of the point u. Let $x^* := \arg\min_{\hat{x} \in [x_{\min}+\frac{\ell_\square}{3}, x_{\min}+\frac{2\ell_\square}{3}]} |\Lambda(\hat{x})|$. We partition $\square$ into two smaller rectangles by using a vertical line defined by $x = x^*$ and add them as the children of $\square$ to $\mathcal{H}$. We refer to the segment partitioning $\square$ into its two children as its *divider* and denote it by $\Gamma_\square$. See Figure 4. For any cell $\square$, the four sides of its rectangle are defined by the dividers of its ancestor

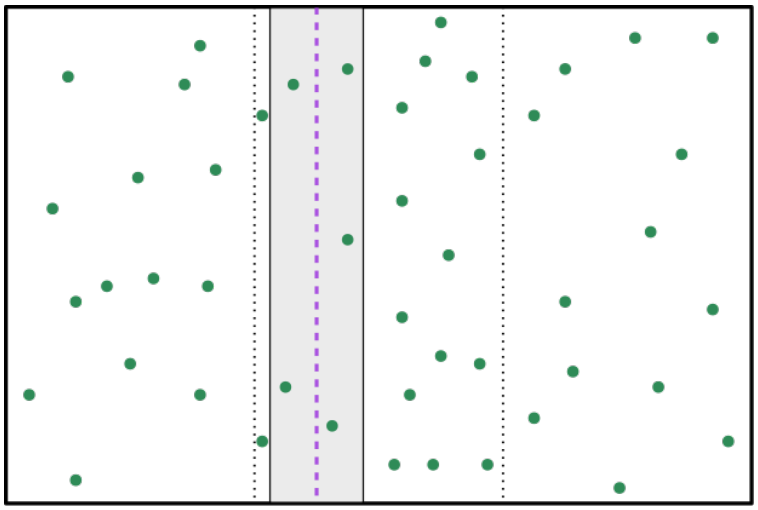


Figure 4: We Partition a rectangle into two children by picking a divider (the purple dashed vertical line) with the minimum number of points close to it (gray shaded area) within the middle-third of its longer side (the part between the two vertical dotted lines).

or the boundaries of the root square. This completes the construction of $\mathcal{H}$. Note that the height of the tree is $O(\log n\Delta)$. A simple sweep-line algorithm can compute x^* in $O(n_\square \log n_\square)$ time. Using this procedure, we construct $\mathcal{H}$ in $\tilde{O}(n \log(n\Delta))$ time.

Lemma 2.1. *For each cell $\square$ of $\mathcal{H}$, the ratio of the largest to the smallest side of $\square$ is at most 3. Furthermore, the number of points of $A_\square \cup B_\square$ with a distance smaller than $\ell_\square \lambda$ to the divider $\Gamma_\square$ is $O(n_\square \lambda)$.*

Recollect that a matching M in $\mathcal{G}_\sigma$ is a subset of vertex-disjoint edges. We refer to a point $b \in B$ as *unmatched* in M if it does not have an edge of M incident on it and *matched* otherwise. The following structural property of the k-SP problem will be critical in bounding the efficiency of our algorithm.

Lemma 2.2. *For any cell $\square$ of $\mathcal{H}$, there exists a matching M' between $A_\square \cup B_\square$ that matches all except $O(n_\square^{4/5})$ points of $B_\square$ and has a cost $O(\ell_\square n_\square^{3/5})$.*

Proof. We place a grid G with cell-side-length $\ell_\square n_\square^{-2/5}$ inside $\square$. For each square ξ of G, let σ_ξ denote the subset of requests that lie inside ξ, and define M_ξ to be the bipartite matching that corresponds to a single server serving the requests of σ_ξ. Define $M' := \bigcup_{\xi \in G} M_\xi$. See Figure 5(left). $\square$

2.2 Extended Bipartite Matching

Suppose we are given a subset $\mathcal{C}$ of cells from $\mathcal{H}$ that partitions $\square^*$, i.e., the cells of $\mathcal{C}$ are interior disjoint and these cells cover the root square $\square^*$. Let $\mathrm{d}(u, \square)$ denote the shortest distance from u to the boundaries of the cell $\square$. For any point u inside $\square^*$, let $\square_u$ be the cell of $\mathcal{C}$ that contains u. We define $\mathrm{d}(b, \mathcal{C}) = \mathrm{d}(b, \square_b)$. We extend the definition of matching to allow points in B to match to the boundaries of cells in $\mathcal{C}$. A *$\mathcal{C}$-extended matching* consists of a matching M as well as a subset $B^{\mathcal{C}}$ of points of B that are unmatched in M but instead matched to the boundaries of the cells of $\mathcal{C}$ that contain them. See Figure 5(right). The cost of a

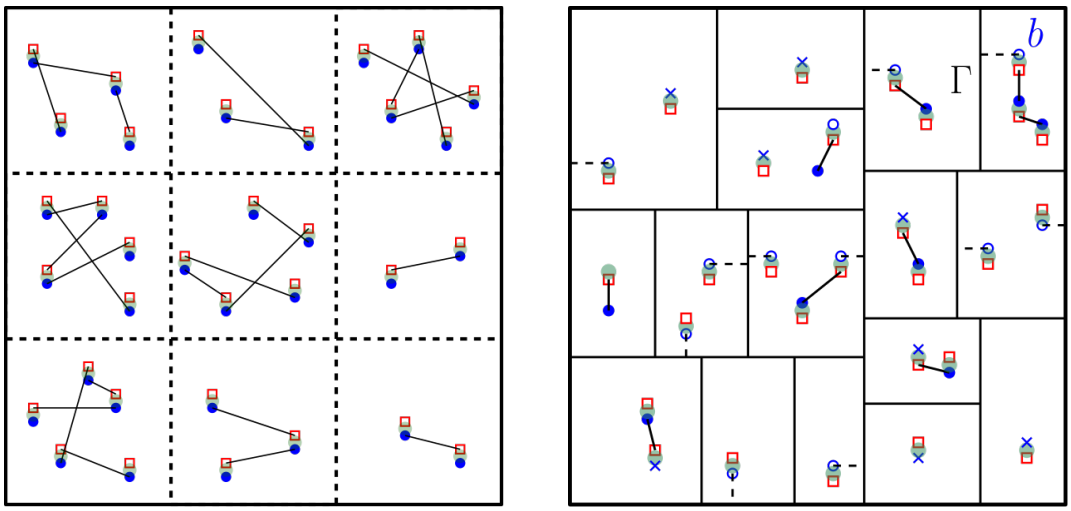


Figure 5: (left) The low-cost high-cardinality matching constructed inside a cell $\square$ of $\mathcal{H}$, and (right) a $\mathcal{C}$-extended 20-matching with 9 matched points (blue discs), 11 boundary-matched points (blue circles), and 8 free points (blue crosses). The matching cost is the total length of the solid and dashed lines. The boundary-matched point b is matched to the boundary Γ.

$\mathcal{C}$-extended matching $M^{\mathcal{C}}$ is

$$w_{\mathcal{C}}(M^{\mathcal{C}}) := \sum_{(a,b)\in M} \mathrm{d}(a,b) + \sum_{b\in B^{\mathcal{C}}} \mathrm{d}(b,\mathcal{C}). \tag{1}$$

We refer to all points of $B^{\mathcal{C}}$ as *boundary-matched*. All points of B that are neither matched in M nor boundary-matched are considered *free*. All points of A that are not matched in M are also considered free. The size of $M^{\mathcal{C}}$ is equal to $|M| + |B^{\mathcal{C}}|$. When $\mathcal{C}$ is clear from the context, we refer to $M^{\mathcal{C}}$ as an extended matching. An extended matching $M^{\mathcal{C}}$ of size t with the minimum cost is called a *minimum-cost $\mathcal{C}$-extended t-matching*.

Consider the partitioning $\mathcal{C}^* = \{\square^*\}$, where $\square^*$ is the root cell of $\mathcal{H}$. Using the fact that the input points are far from the boundaries of $\square^*$, we show in Lemma 2.3 below that any minimum-cost $\mathcal{C}^*$-extended matching $M^{\mathcal{C}^*}$ of size t is also a minimum-cost t-matching, i.e., no point of B is boundary-matched in $M^{\mathcal{C}^*}$.

Lemma 2.3. *Suppose $\mathcal{C}^* = \{\square^*\}$, where $\square^*$ is the root cell of $\mathcal{H}$. Let $M^{\mathcal{C}^*} = (M, B^{\mathcal{C}^*})$ be a minimum-cost extended t-matching on $\mathcal{G}_\sigma$, for $t < n$. Then, the matching M is a minimum-cost t-matching.*

Let $M^{\mathcal{C}} = (M, B^{\mathcal{C}})$ denote a $\mathcal{C}$-extended matching. Any path P on the graph $\mathcal{G}_\sigma$ whose edges alternate between matching and non-matching edges in M is called an *alternating path*. An alternating path P is called an *augmenting path* if P starts from a free point $b \in B$ and ends with either (i) a free point $a \in A$, or (ii) a point $b' \in B$. We *augment* the extended matching $M^{\mathcal{C}}$ along an augmenting path P by updating its matching $M \leftarrow M \oplus P$ and for case (ii), we match b' to the boundary and update $B^{\mathcal{C}}$ to include b'. The *net-cost* of P in case (i) is $\phi(P) := \sum_{(a,b)\in P\setminus M} \mathrm{d}(a,b) - \sum_{(a,b)\in P\cap M} \mathrm{d}(a,b)$, and in case (ii) is

$$\phi(P) := \mathrm{d}(b',\mathcal{C}) + \sum_{(a,b)\in P\setminus M} \mathrm{d}(a,b) - \sum_{(a,b)\in P\cap M} \mathrm{d}(a,b).$$

From Lemma 2.3, given a sequence σ of n requests in the unit square, one can compute an optimal solution to the k-SP problem on σ by computing a minimum-cost $\mathcal{C}^*$-extended $(n-k)$-matching on $\mathcal{G}_\sigma$. Given a partitioning $\mathcal{C}$, to compute a minimum-cost $\mathcal{C}$-extended $(n-k)$-matching, similar to the Hungarian algorithm, one can start from an empty extended matching $M^{\mathcal{C}}$ and iteratively augment $M^{\mathcal{C}}$ along a minimum net-cost augmenting path. In the next section, we present a primal-dual framework that our algorithm uses to efficiently compute minimum net-cost augmenting paths.

2.3 A Constrained Dual Formulation for Extended Matchings

Suppose $\mathcal{C}$ is a set of cells of $\mathcal{H}$ partitioning the root cell $\square^*$. Consider a $\mathcal{C}$-extended matching $M^{\mathcal{C}} = (M, B^{\mathcal{C}})$ on $\mathcal{G}_\sigma$ along with a set of non-negative dual weights $y : A \cup B \to \mathbb{R}_{\geq 0}$. Let A_F be the set of free points of A with respect to $M^{\mathcal{C}}$. We say that $M^{\mathcal{C}}, y(\cdot)$ is *feasible* if,

$$y(b) - y(a) \leq \mathrm{d}(a,b), \qquad \forall (a,b) \in E, \tag{2}$$

$$y(b) - y(a) = \mathrm{d}(a,b), \qquad \forall (a,b) \in M, \tag{3}$$

$$y(b) \leq \mathrm{d}(b,\mathcal{C}), \qquad \forall b \in B, \tag{4}$$

$$y(b) = \mathrm{d}(b,\mathcal{C}), \qquad \forall b \in B^{\mathcal{C}}, \tag{5}$$

$$y(a) = 0, \qquad \forall a \in A_F. \tag{6}$$

For any edge $(a,b) \in E$, the *slack* of (a,b) is defined as $s(a,b) := \mathrm{d}(a,b) - y(b) + y(a)$. The edge (a,b) is *admissible* if $s(a,b) = 0$. For any point $b \in B$, the slack of b is defined as $s(b) := \mathrm{d}(b,\mathcal{C}) - y(b)$. For any feasible extended matching $M^{\mathcal{C}}, y(\cdot)$, the slack of every edge as well as every point $b \in B$ is non-negative. Recall that an augmenting path P starts at a free point $b \in B$ and ends at (i) a free point $a \in A$ or (ii) a point $b' \in B$. The path P is *admissible* if all edges of P are admissible and in case (ii), the slack of the end-point b' is $s(b') = 0$. The following properties of extended feasible matchings are critical in the design of an efficient and correct algorithm.

Lemma 2.4. *Given a feasible $\mathcal{C}$-extended t-matching $M^{\mathcal{C}} = (M, B^{\mathcal{C}})$ and a set of non-negative dual weights $y(\cdot)$ on $A \cup B$, let P be a minimum net-cost augmenting path with respect to $M^{\mathcal{C}}$. Let, for any $\square \in \mathcal{C}$, $y_\square = \max_{b' \in B_\square} y(b')$. Then,*

(a) all points of P lie inside a single cell of $\mathcal{C}$, and

(b) if, for every cell $\square \in \mathcal{C}$, $y_\square \leq \phi(P)$ and for all free points $b \in B_\square$, $y(b) = y_\square$, then $M^{\mathcal{C}}$ is a minimum-cost extended t-matching.

The property described in Lemma 2.4(a) is important for the design of an efficient algorithm. Unlike the Hungarian algorithm, which searches the entire graph for the minimum net-cost augmenting path, our algorithm can find the minimum net-cost augmenting path by searching for the cheapest augmenting path inside each cell $\square \in \mathcal{C}$ and then taking the smallest among them. Thus, we can replace a global search with a search inside each cell of $\mathcal{C}$. The property in Lemma 2.4(b) is important for the design of a correct algorithm since it provides conditions under which an extended matching is a minimum-cost extended matching. Our algorithm is designed to maintain these conditions as invariants during its execution. Lemma 2.5 provides a method to update the dual weights, which will be essential during the process of merging cells.

Lemma 2.5. *Suppose $M^{\mathcal{C}}, y(\cdot)$ is a feasible $\mathcal{C}$-extended matching and P is a minimum net-cost augmenting path. For any cell $\square \in \mathcal{C}$, define $y_\square = \max_{b' \in B_\square} y(b')$, and suppose $y_\square \leq \phi(P)$ and $y(b_f) = y_\square$ for all free*

points $b_f \in B_\square$. Then, one can update the dual weights in $\tilde{O}(n_\square \Phi(n_\square))$ time such that $M^{\mathcal{C}}, y(\cdot)$ remains feasible, $y(b) \leq \phi(P)$ for all $b \in B_\square$, and $y(b_f) = \phi(P)$ for all free points $b_f \in B_\square$.

The next lemma provides critical properties that allow for correctly and efficiently merging cells in $\mathcal{C}$.

Lemma 2.6. *For a cell $\square$ of $\mathcal{H}$, suppose $\square'$ and $\square''$ denote its two children, and let $\mathcal{C}$ denote a partitioning containing $\square'$ and $\square''$. Let $\mathcal{C}' = \mathcal{C} \cup \{\square\} \setminus \{\square', \square''\}$. Given a feasible $\mathcal{C}$-extended matching $(M, B^{\mathcal{C}}), y(\cdot)$, let $B^{\mathcal{C}}_\square \subseteq B^{\mathcal{C}}$ denote the subset of boundary-matched points that are matched to the divider $\Gamma_\square$ of $\square$. Then,*

(a) the $\mathcal{C}'$-extended matching $(M, B^{\mathcal{C}} \setminus B^{\mathcal{C}}_\square), y(\cdot)$ is also feasible, and,

(b) $|B^{\mathcal{C}}_\square| = O(n^{4/5})$.

From Lemma 2.6(a), erasing a divider does not cause the feasibility conditions to be violated. The proof of Lemma 2.6(a) relies on the fact that when we erase the divider of a cell in $\mathcal{C}$, the RHS of (4) will only increase and so it is not violated. Despite preserving the feasibility conditions, erasing the boundary may result in the violation of Lemma 2.4(b), i.e., the matching may no longer be a minimum-cost extended matching. In our algorithm in Section 3, we describe a process to adjust the matching $M^{\mathcal{C}}$ and dual weights $y(\cdot)$ and obtain a minimum-cost extended matching.

Residual Graph. Similar to the Hungarian algorithm, we define a residual graph that assists in finding the minimum net-cost augmenting path. Consider a feasible $\mathcal{C}$-extended matching $M^{\mathcal{C}} = (M, B^{\mathcal{C}})$ along with a set of dual weights $y(\cdot)$ on points of $A \cup B$. For each cell $\square \in \mathcal{C}$, we define a residual graph $\mathcal{G}_\square$. The vertex set of $\mathcal{G}_\square$ is a source vertex s and the points in $A_\square \cup B_\square$. For any edge $(a, b) \in E$ inside $\square$, if (a, b) is an edge in M (resp. not an edge in M), there is an edge directed from a to b (resp. from b to a) with a weight $s(a, b)$ in $\mathcal{G}_\square$. Furthermore, there is an edge directed from s to every free point $b \in B$ with a weight $y(b)$.

3 A Sub-Quadratic Algorithm for the k-SP and k-SPI Problems

In this section, we describe an algorithm that, given a sequence σ of n requests in 2-dimensions, computes the optimal solution to the k-SP and k-SPI problems in $\tilde{O}(n^{1.8}\Phi(n)\log(n\Delta))$ time. We begin by describing our algorithm for the k-SP problem and discuss how it can be extended to the k-SPI problem in Section 3.4.

3.1 Algorithm for the k-SP Problem

Initialize $\mathcal{C}$ to the leaf cells of $\mathcal{H}$. Let $M^{\mathcal{C}} = (M, B^{\mathcal{C}})$ be the extended matching maintained by the algorithm and initialized to $M = \emptyset$ and $B^{\mathcal{C}} = \emptyset$. For each point $v \in A \cup B$, let $y(v)$ denote its dual weight initialized to $y(v) = 0$. Let B_F, initialized to B, be the free points of B with respect to $M^{\mathcal{C}}$. For each cell $\square$ that contains at least one free point $b \in B_F$, let $P_\square$ denote the minimum net-cost augmenting path inside $\square$. Initially, since $\square$ is a leaf of $\mathcal{H}$, it contains only one point $b \in B_F$, and therefore, $P_\square$ is this point with a net-cost equal to $\mathrm{d}(b, \mathcal{C})$. Our algorithm maintains a priority queue PQ storing every leaf cell $\square \in \mathcal{C}$ with at least one free vertex with a key of $\phi(P_\square)$. At any time during the execution of our algorithm, let $\square_{\min}$ be the cell with the smallest key in PQ and let φ be the key of $\square_{\min}$. Execute the following steps until PQ becomes empty:

- While $|B_F| > k$,
  - *Extended Hungarian search step:* Extract the cell $\square$ with the minimum key of φ from PQ. Augment the matching $M^{\mathcal{C}}$ along $P_{\square}$ and update the key of $\square$ in PQ (See Section 3.1.1 for details).
- *Merge step:* If $\mathcal{C} = \{\square^*\}$, remove $\square^*$ from PQ and return the matching M of $M^{\mathcal{C}}$. Otherwise, pick a cell $\square' \in \mathcal{C}$ with the smallest perimeter and let $\square$ and $\square''$ be the parent and sibling of $\square'$ in $\mathcal{H}$, respectively. Erase the divider of $\square$, i.e., set $\mathcal{C} = \mathcal{C} \setminus \{\square', \square''\} \cup \{\square\}$. We execute a Merge procedure that updates the matching $M^{\mathcal{C}}$ inside the cell $\square$ so that $\phi(P_{\square})$ is at least φ (See Section 3.1.2 for details). At this point, the size of the updated extended matching $M^{\mathcal{C}}$ may not be $(n-k)$, i.e., there may be more than k free points.

Invariants. For each cell $\square \in \mathcal{C}$, let $y_{\square} = \max_{b \in B_{\square}} y(b)$. During the execution of our algorithm,

(I1) the extended matching $M^{\mathcal{C}}, y(\cdot)$ is feasible,

(I2) For each cell $\square \in \mathcal{C}$, $y_{\square} \leq \varphi$ and for all free point $b \in B_{\square}$, $y(b) = y_{\square}$, and,

(I3) The φ-value is non-decreasing. Furthermore, after each step of the algorithm, φ denotes the smallest net-cost of all augmenting paths with respect to $M^{\mathcal{C}}$.

3.1.1 Details of the Extended Hungarian Search Step

Given a feasible extended matching $M^{\mathcal{C}}, y(\cdot)$ and a cell $\square \in \mathcal{C}$, the extended Hungarian search procedure computes the minimum net-cost augmenting path $P_{\square}$ and augments $M^{\mathcal{C}}$ along $P_{\square}$. It then computes the new minimum net-cost augmenting path and updates the key of $\square$ in PQ to be its net-cost. This procedure is similar to the classical Hungarian search procedure executed on $\mathcal{G}_{\square}$ and is mildly modified to include the augmenting paths that end at the boundary of $\square$. Details of the procedure are as follows.

1. *Update duals:* With s as the source vertex, execute Dijkstra's shortest path algorithm on the residual graph $\mathcal{G}_{\square}$. Let P_v be the shortest path from s to each $v \in A_{\square} \cup B_{\square}$ and let κ_v be the cost of P_v. Let

$$\kappa = \min\{\min_{a \in A^F_{\square}} \kappa_a, \min_{b \in B_{\square}} \kappa_b + s(b)\}. \tag{7}$$

   For any $v \in A_{\square} \cup B_{\square}$, if $\kappa_v < \kappa$, set $y(v) \leftarrow y(v) + \kappa - \kappa_v$.

2. *Augment:* Let $u \in A^F_{\square} \cup B_{\square}$ be the point realizing the minimum distance in Equation (7). Let P be the augmenting path obtained by removing s. Augment $M^{\mathcal{C}}$ along P.

3. *Update key:* If $B_{\square}$ has no free points, then remove $\square$ from PQ. Otherwise,
   (a) Recompute the residual graph $\mathcal{G}_{\square}$ with respect to the updated matching,
   (b) With s as the source, execute the Dijkstra's shortest path algorithm on $\mathcal{G}_{\square}$. For each $v \in A_{\square} \cup B_{\square}$, let κ_v be the distance from s to v.
   (c) Update the key of $\square$ in PQ to $\kappa_{\square} = \min\{\min_{a \in A^F_{\square}} \kappa_a, \min_{b \in B_{\square}} \kappa_b + s(b)\}$.

Lemma 3.1 establishes the properties of the extended Hungarian search procedure.

Lemma 3.1. *After the execution of the extended Hungarian search procedure for a cell $\square$, the extended matching $M^{\mathcal{C}}, y(\cdot)$ remains feasible, $y(v) \leq \varphi$ for all points $v \in A_{\square} \cup B_{\square}$, and $y(b_f) = \varphi$ for all free points $b_f \in B_{\square}$. Furthermore, the path P computed by the procedure is a minimum net-cost augmenting path inside $\square$. After augmenting along P, the updated key for $\square$ is the smallest net-cost of all augmenting paths inside $\square$ and is at least φ.*

3.1.2 Details of the Merge Step

Given a feasible extended matching $M^{\mathcal{C}} = (M, B^{\mathcal{C}}), y(\cdot)$, any cell $\square$ of $\mathcal{H}$ where both of its children $\square'$ and $\square''$ are in $\mathcal{C}$, and a value φ, the merge procedure uses the algorithm in Lemma 2.5 to update the dual weights $y(\cdot)$ inside $\square'$ (resp. $\square''$) so that $M^{\mathcal{C}}, y(\cdot)$ remains feasible, the dual weights of all points in $\square'$ (resp. $\square''$) are at most φ, and the dual weight of all free points $b'_f \in B_{\square'}$ (resp. $b''_f \in B_{\square''}$) is $y(b'_f) = \varphi$ (resp. $y(b''_f) = \varphi$). The procedure then updates $\mathcal{C} \leftarrow \mathcal{C} \cup \{\square\} \setminus \{\square', \square''\}$ and makes the points matched to the divider $\Gamma_{\square}$ free. While there exists a free point $b \in B_{\square}$ with $y(b) < \varphi$,

1. With s as the source, execute Dijkstra's shortest path algorithm on the residual graph $\mathcal{G}_{\square}$. For each $v \in A_{\square} \cup B_{\square}$, let P_v be the shortest path from s to v in $\mathcal{G}_{\square}$ and let κ_v be its cost. Define
$$\kappa = \min\{\min_{a \in A^F_{\square}} \kappa_a, \min_{b \in B_{\square}} \kappa_b + s(b), \min_{b \in B_{\square}} \kappa_b + \varphi - y(b)\}. \tag{8}$$
2. Let $u \in A^F_{\square} \cup B_{\square}$ be the point realizing the minimum value in Equation (8). Let P be the path obtained by removing s from the path P_u.
   (a) If $u \in B_{\square}$ and $\kappa = \kappa_u + \varphi - y(u)$ (i.e., κ is determined by the third element in the RHS of Equation (8)), then P is a path from a free point $b \in B_{\square}$ to the matched point u. For each $v \in A_{\square} \cup B_{\square}$, if $\kappa_v < \kappa$, update its dual weight to $y(v) \leftarrow y(v) + \kappa - \kappa_v$. The path P is an admissible alternating path with respect to the updated dual weights. Set $M \leftarrow M \oplus P$. Note that u is now a free point with $y(u) = \varphi$.
   (b) Otherwise, P is an augmenting path. For each $v \in A_{\square} \cup B_{\square}$, if $\kappa_v < \kappa$, update its dual weight to $y(v) \leftarrow y(v) + \kappa - \kappa_v$. The path P is an admissible augmenting path with respect to the updated dual weights. Augment M along P.

At the end of this execution, all free points of B in $\square$ have a dual weight of φ. Finally, we update the key for $\square$ by executing steps 3(a)–(c) from the extended Hungarian search step.

The following lemma states the useful properties of the merge step.

Lemma 3.2. *After the execution of the merge procedure on a cell $\square$, the updated extended matching $M^{\mathcal{C}}, y(\cdot)$ is feasible, the dual weights $y(v)$ for every $v \in A_{\square} \cup B_{\square}$ is at most φ, and $y(b_f) = \varphi$ for all free points $b_f \in B_{\square}$. Furthermore, the updated key for $\square$ is the smallest net-cost of all augmenting paths within $\square$ and is at least φ.*

3.2 Analysis

We begin by showing in Section 3.2.1 that the three invariants (I1)–(I3) hold during the execution of our algorithm and use them to show the correctness of our algorithm. We then show in Section 3.2.2 that the running time of our algorithm is $\tilde{O}(n^{9/5}\Phi(n)\log n\Delta)$.

3.2.1 Correctness

Our algorithm initializes $M^{\mathcal{C}}$ with a feasible $\mathcal{C}$-extended matching and sets all dual weights and φ to 0. Therefore, (I1) and (I2) hold at the start of the algorithm. The extended Hungarian search (Lemma 3.1) as well as the merge step (Lemma 3.2) do not violate the feasibility of the extended matching and therefore (I1) holds during the execution of the algorithm. The extended Hungarian search (Lemma 3.1) and the merge (Lemma 3.2) procedures keep the dual weight of every point inside $\square$ at or below φ, while ensuring that the dual weight of free points inside $\square$ is φ; hence, (I2) holds. Finally, both the extended Hungarian search (Lemma 3.1) and the merge (Lemma 3.2) procedures update the key of $\square$ to the smallest net-cost of all augmenting paths inside $\square$ and do not decrease the key of any cell. From this observation, (I3) follows in a straightforward way.

From (I1) and (I2), our algorithm maintains a feasible $\mathcal{C}$-extended matching where, for each cell $\square \in \mathcal{C}$, $y_{\square} \leq \varphi$. From (I3), φ is equal to the minimum net-cost of all augmenting paths with respect to $M^{\mathcal{C}}$; combining this with Lemma 2.4(b), we conclude that $M^{\mathcal{C}}$ is a minimum-cost $\mathcal{C}$-extended matching. Upon termination, the algorithm returns a minimum-cost $\mathcal{C}^*$-extended $(n-k)$-matching for $\mathcal{C}^* = \{\square^*\}$, which from Lemma 2.3 has no boundary-matched points. Therefore, this matching is also a minimum-cost $(n-k)$-matching, as desired.

3.2.2 Efficiency

Both the merge step and the extended Hungarian search step require an execution of Dijkstra's shortest path algorithm on $\mathcal{G}_{\square}$. In Section 3.3, we show that this execution takes $\tilde{O}(n_{\square}\Phi(n_{\square}))$ time. Using this, we analyze the execution time.

We begin by establishing a bound on the execution time of the merge step, which combines $\square'$ and $\square''$ into a single cell $\square$. At the start of this step, the dual weight of all free points in $\square'$ and $\square''$ are raised to φ (from Lemma 2.5). As a result, once the divider is removed, the only free points with dual weights below φ are those that are matched to the divider $\Gamma_{\square}$. From Lemma 2.6(b), the number of points matched to $\Gamma_{\square}$ is $O(n^{4/5})$. Therefore, the while-loop in the merge procedure executes only $O(n^{4/5})$ times. Since each iteration takes $\tilde{O}(n_{\square}\Phi(n_{\square}))$ time, the total execution time of a single execution of the merge step is $\tilde{O}(n^{4/5}n_{\square}\Phi(n_{\square}))$. Given that each point is in only $O(\log n\Delta)$ many cells of $\mathcal{H}$, the total time taken by the merge step across all cells of $\mathcal{H}$ is $\tilde{O}(n^{9/5}\Phi(n)\log n\Delta)$.

Similarly, if the execution time for the extended Hungarian search within a single cell $\square$ is bounded by $O(n^{4/5}n_{\square}\Phi(n_{\square}))$, then the cumulative execution time of the extended Hungarian search across all cells – and consequently, the total runtime of the entire algorithm – can be bounded by $\tilde{O}(n^{9/5}\Phi(n)\log n\Delta)$. In the remainder of our analysis, we establish a bound of $O(n^{4/5}n_{\square}\Phi(n_{\square}))$ for the time taken by the extended Hungarian search within a single cell $\square$. Recall that the algorithm selects a cell $\square$ containing the minimum net-cost augmenting path from the priority queue PQ and performs an extended Hungarian search procedure within $\square$, requiring $\tilde{O}(n_{\square}\Phi(n_{\square}))$ time. To analyze the total execution time of the extended Hungarian search procedure, we show that any cell $\square$ can be selected by the algorithm at most $O(n^{4/5})$ times.

We categorize the selection of $\square$ as a *low-net-cost* case if $\varphi \leq \ell_{\square}n^{-1/5}$ and a *high-net-cost* case otherwise. First, we show that in the high-net-cost case ($\varphi > \ell_{\square}n^{-1/5}$), the number of free points remaining in $\square$ is $O(n^{4/5})$, and as a result, the total number of high-net-cost selections of $\square$ is $O(n^{4/5})$.

Lemma 3.3. *Given a feasible extended matching $M^{\mathcal{C}}, y(\cdot)$ and any cell $\square \in \mathcal{C}$, if the net-cost of the minimum net-cost augmenting path inside $\square$ is greater than $\ell_{\square}n^{-1/5}$, then the number of free points of $M^{\mathcal{C}}$ is $O(n^{4/5})$.*

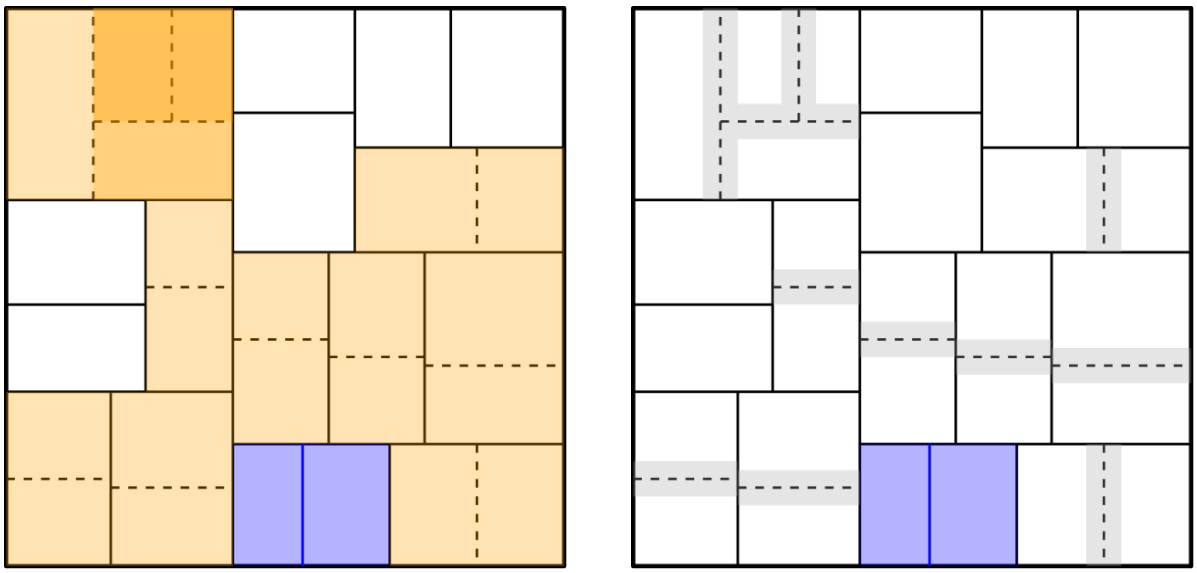


Figure 6: (left) The set $\mathcal{C}_\square$ (the cells shaded in orange with dashed lines as their divider) for a cell $\square$ (shaded in blue), and (right) any boundary-matched point in the gray area might cause a low-net-cost execution of the extended Hungarian search procedure on $\square$.

Proof. Let M be the matching corresponding to the extended matching $M^{\mathcal{C}}$ and let M' be the matching defined in Lemma 2.2. Every free point of $M^{\mathcal{C}}$ participates in an augmenting or an alternating path in $M \oplus M'$. The number of alternating paths in the symmetric difference cannot exceed the number of free points of M', which is $O(n^{4/5})$. Furthermore, the combined net-cost of these augmenting paths is at most $w(M') = O(\ell_\square n^{3/5})$, and each one has a net-cost at least $\ell_\square n^{-1/5}$. Thus, there are $O(n^{4/5})$ augmenting paths in the symmetric difference. See Appendix C.4 for a complete proof. $\square$

Next, we bound the number of low-net-cost selections of $\square$. Define $\mathcal{C}_\square$ as the set of all cells $\square' \in \mathcal{H}$ that are processed by the merge procedure while $\square \in \mathcal{C}$ and $\varphi \le \ell_\square n^{-1/5}$ (Figure 6). Our algorithm always picks the smallest perimeter cells to merge, and as a result, we obtain the following lemma.

Lemma 3.4. *For any non-leaf cell $\square$ in $\mathcal{H}$ and any cell $\square' \in \mathcal{C}_\square$, $\frac{1}{3}\ell_{\square'} \le \ell_\square \le \frac{9}{4}\ell_{\square'}$.*

For any cell $\square' \in \mathcal{C}_\square$, the merge step erases the divider $\Gamma_{\square'}$ and makes the points that are matched to $\Gamma_{\square'}$ free. Each of these new free points might cause $\square$ to be selected by the algorithm. Therefore, to bound the number of low-net-cost selections of $\square$, we show that the total number of points that are matched to the dividers of the cells in $\mathcal{C}_\square$ is at most $O(n^{4/5})$. For any cell $\square' \in \mathcal{C}_\square$, let $\mathcal{B}_{\square'}$ denote the set of points of $B^{\mathcal{C}}$ that are matched to the divider $\Gamma_{\square'}$ when our algorithm starts the merge step on $\square'$. Thus, we have to bound $\sum_{\square' \in \mathcal{C}_\square} |\mathcal{B}_{\square'}|$ by $O(n^{4/5})$. By invariant (I2), for each point $b \in \mathcal{B}_{\square'}$, $y(b) \le \varphi \le \ell_\square n^{-1/5}$. Furthermore, by Condition (5), $y(b) = \mathrm{d}(b, \square') = \mathrm{d}(b, \Gamma_{\square'})$. Therefore, the point b is within a distance $\ell_\square n^{-1/5}$ from the divider $\Gamma_{\square'}$. From Lemma 3.4, for each cell $\square' \in \mathcal{C}_\square$ and each point $b \in \mathcal{B}_{\square'}$, the point b is within a distance $\ell_\square n^{-1/5} < \ell_{\square'} \lambda$ from the divider $\Gamma_{\square'}$, and from the construction of $\mathcal{H}$ (Lemma 2.1), $|\mathcal{B}_{\square'}| = O(n_{\square'} n^{-1/5})$. Furthermore, from Lemma 3.4 and the construction of $\mathcal{H}$, each point $b \in B$ can lie inside a constant number of cells in $\mathcal{C}_\square$; hence, $\sum_{\square' \in \mathcal{C}_\square} |\mathcal{B}_{\square'}| = O(\sum_{\square' \in \mathcal{C}_\square} n_{\square'} n^{-1/5}) = O(n^{4/5})$. Therefore, the total number of low-net-cost selections of $\square$ by the extended Hungarian search procedure is $O(n^{4/5})$, as claimed.

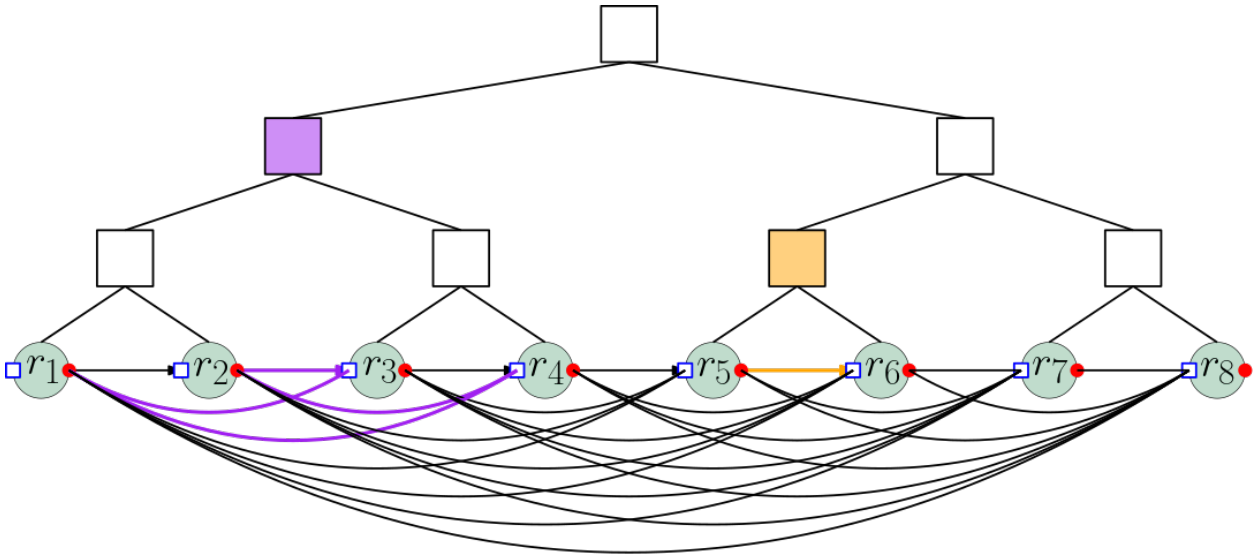


Figure 7: The balanced binary search tree T used in the construction of our dynamic weighted bichromatic closest pair data structure. The purple (resp. orange) edges in the graph are stored at the purple (resp. orange) node of T.

3.3 Fast Search Procedures for the Geometric k-SP and the k-SPI Problems

Given an instance of the geometric k-SP, in this section, we show that the search for the minimum net-cost augmenting path using a Hungarian search style procedure (as in the merge and extended Hungarian search procedures of our algorithm) can be done in $O(n\Phi(n)\log^3 n)$ time, where $\Phi(n)$ is the query/update time of an existing dynamic weighted nearest neighbor data structure (DWNN).

Given a matching M and dual weights $y(\cdot)$ satisfying Conditions (2) and (3), the Hungarian search (which executes a Dijkstra's shortest path procedure) can be efficiently implemented on a complete bipartite graph using only $\tilde{O}(n)$ queries to a dynamic weighted bichromatic closest pair data structure (BCP). This search procedure grows a tree $\mathcal{T}$ by finding the cheapest cut edge under a weighted distance. See [1, 28, 29] where the Hungarian search procedure that uses $O(n)$ queries to a BCP has been described. Furthermore, one can implement a BCP using a DWNN in $\tilde{O}(\Phi(n))$ time; here $\Phi(n)$ is the update/query time of the DWNN [6].

In our case, the graph $\mathcal{G}_\sigma$ is not a complete graph. However, we can easily decompose the graph into a set of $O(n)$ complete bipartite graphs as follows. Build a balanced binary search tree T where the indices of requests $\{1, \ldots, n\}$ form the leaves. Let, for any node v, $L(v)$ denote the indices at the leaves of the left subtree of v and $R(v)$ denote the indices at the leaves of the right subtree of v. We partition the edges as follows: At each node v in T, we store all edges that go from the exit gates of requests with indices in $L(v)$ to the entry gates of the requests with indices in $R(V)$. See Figure 7 for an example. Thus, at each node, we simply store the complete bipartite graph between points in $L(v)$ and $R(v)$. It is easy to see that any request r_j participates in $O(\log n)$ cliques.

While executing the Hungarian search, we maintain a BCP at each node v of this tree. Let $\mathcal{T}$ denote the Dijkstra's shortest path tree maintained by the Hungarian search. Our data structure stores the nodes of $L(v)$ that are currently in $\mathcal{T}$ and the nodes of $R(v)$ that are currently not in $\mathcal{T}$. The weighted bichromatic closest pair for each of these cliques is stored in a global priority queue. The query to this data structure will

simply return the pair at the top of this priority queue. Adding a node v to the Dijkstra's shortest path tree will trigger insertion or deletion of v in the $O(\log n)$ cliques that it participates in. For each of these cliques, we update their representative pair in the global priority queue. Thus, all insertions, deletions, and queries can be done in $\tilde{O}(\Phi(n))$ time. We obtain a fast Hungarian search by using this data structure in place of the standard dynamic weighted bichromatic closest pair data structure as used in [28].

3.4 Extension of our Sub-Quadratic to the k-SPI Problem

Our algorithm for the k-SP problem computes a minimum-cost $(n-k)$-matching on a graph $\mathcal{G}_\sigma$ between a point set A of size n and a point set B of size n. For the k-SPI problem on an instance $\sigma' = \eta\sigma$, since there exists a matching on the graph $\mathcal{G}_{\sigma'}$ that matches all points of B, one can use the same algorithm by setting $k = 0$ to compute a minimum-cost n-matching on $\mathcal{G}_{\sigma'}$. Our algorithm therefore easily extends to the k-SPI problem. Note that the efficiency of our algorithm only requires the existence of a matching that matches the majority of the points of $B_\square$ in each cell $\square$ and has a low cost (Lemma 2.2). We observe that the same property holds for the k-SPI problem, as the initial locations of the servers only introduce new points to the set $A_\square$ and the number of unmatched points of $B_\square$ in the matching constructed in Lemma 2.2 remains unchanged. Therefore, our algorithm runs in $\tilde{O}(n^{9/5}\Phi(n)\log\Delta)$ time for the k-SPI problem as well.

4 Fast Implementations of the Hungarian Algorithm for the k-SP and the k-SPI Problems

In this section, given a sequence of n requests in 2 dimensions, we present an adaptation of the Hungarian algorithm [14] for computing a minimum $(n-k)$-matching under any ℓ_p norm in $\tilde{O}(nk\Phi(n))$ time. We begin by describing the notions used extensively by existing bipartite matching algorithms and then present our algorithm for the k-SP problem.

4.1 Background

For a matching M on $\mathcal{G}_\sigma$, let B_F (resp. A_F) denote the set of free points of B (resp. A) with respect to M. The matching M along with a set of non-negative dual weights $y : A \cup B \to \mathbb{R}_{\geq 0}$ is *feasible* if

$$y(b) - y(a) \leq \mathrm{d}(a,b), \qquad \forall (a,b) \in E, \tag{9}$$

$$y(b) - y(a) = \mathrm{d}(a,b), \qquad \forall (a,b) \in M, \tag{10}$$

$$y(a) = 0, \qquad a \in A_F. \tag{11}$$

Any feasible t-matching $M, y(\cdot)$, where $y(b) = \max_{b' \in B} y(b')$ for all free points $b \in B_F$, is a *dual-optimal* matching. The following lemma relates the dual-optimal t-matchings and minimum-cost t-matchings.

Lemma 4.1. *For any $t > 0$ and any dual-optimal t-matching $M, y(\cdot)$ on a graph $\mathcal{G}_\sigma$, the matching M is a minimum-cost t-matching on $\mathcal{G}_\sigma$.*

For any feasible matching $M, y(\cdot)$ and any edge $(a,b) \in E$, the *slack* of (a,b), denoted by $s(a,b)$, is defined as $s(a,b) := \mathrm{d}(a,b) - y(b) + y(a)$. The edge (a,b) is *admissible* if $s(a,b) = 0$. Any alternating path P is admissible if all edges in P are admissible.

Reversed residual graph. We construct a reversed residual graph $\bar{G}$ of $\mathcal{G}_\sigma$ and matching M as follows. The vertex set of $\bar{G}$ is the point set $A \cup B$ in addition to a source vertex s. For any pair $(a, b) \in E$, if (a, b) is a matching (resp. non-matching) edge, there is an edge directed from b to a (resp. from a to b) with a weight $s(a, b)$ in $\bar{G}$. Furthermore, there is an edge from s to every point $b \in B$ with a weight $y_{\max} - y(b)$, where $y_{\max} := \max_{b' \in B} y(b')$.

Reverse augmenting paths. Define a *reverse augmenting path* as an alternating path P with the first and last edges as matching edges, i.e., a directed path from a point $b \in B$ to a point $a \in A$ in the reversed residual graph. Given a reverse augmenting path P, we *reduce* the matching M along P by setting $M \leftarrow M \oplus P$. Reducing M along P decreases the number of matching edges of M by one.

4.2 Efficient Hungarian Algorithm for the k-SP Problem

At a high level, our algorithm starts with a matching M on $\mathcal{G}_\sigma$ corresponding to the optimal solution to the 1-SP problem, i.e., the matching M is the maximum matching on $\mathcal{G}_\sigma$ representing the 1-partitioning $\langle r_1, r_2, \ldots, r_n \rangle$. Our algorithm then iteratively increments the number of partitions (i.e., increases the number of free points of B by one) while ensuring that the maintained partitioning has a minimum cost. In particular, using a reversed version of the Hungarian search procedure, our algorithm finds and updates the matching along the minimum (negative) net-cost reverse augmenting path (i.e., an alternating path with its first and last edges as matching edges). Our algorithm stops and returns the matching when $|M| = n - k$. We provide the details next.

Algorithm. Set $M = \{(a_i, b_{i+1}) \mid i \in [1, n-1]\}$ as a matching that corresponds to serving all requests in $\langle r_1, r_2, \ldots, r_n \rangle$ by a single server. Assign dual weights to the points in $A \cup B$ as follows. Set $y(a_n) \leftarrow 0$. Starting from $i = n-1$, set $y(a_i) \leftarrow \max\{0, \max_{j>i+1} y(b_j) - \mathrm{d}(a_i, b_j)\}$ and $y(b_{i+1}) \leftarrow \mathrm{d}(a_i, b_{i+1}) + y(a_i)$. Finally, set $y(b_1) \leftarrow \max_{i>1} y(b_i)$. The matching $M, y(\cdot)$ is a dual-optimal $(n-1)$-matching on $\mathcal{G}_\sigma$.

While $|M| > n - k$ (i.e., there are less than k free points in B), execute the REVERSEHUNGARIANSEARCH procedure described below, which updates the dual weights and returns an admissible reverse augmenting path P. Reduce M along P.

REVERSEHUNGARIANSEARCH *Procedure.* Execute the Dijkstra's shortest path procedure from the source vertex s on the reversed residual graph $\bar{G}$, which computes the distance κ_u of each vertex $u \in A \cup B$ from s. Define $\kappa := \min_{a \in A} \kappa_a + y(a)$. For any point $u \in A \cup B$ with $\kappa_u < \kappa$, set $y(u) \leftarrow y(u) - \kappa + \kappa_u$. Let $a \in A$ denote the point with the minimum distance (i.e., $\kappa_a = \kappa$), and let P denote the shortest path from s to a. Return the path P' obtained by removing s from P.

Lemma 4.2. *For any $1 \leq t \leq k$, the matching $M, y(\cdot)$ maintained by our algorithm after $(t-1)$ iterations is a dual-optimal $(n-t)$-matching on $\mathcal{G}_\sigma$.*

The initialization step of our algorithm takes $O(n^2)$ time to compute the set of dual weights and the matching M. Additionally, our algorithm runs k iterations, where in each iteration, it executes the REVERSEHUNGARIANSEARCH procedure in $O(n^2)$ time. Therefore, the total running time of our algorithm would be $O(n^2 k)$ time. In geometric settings, using the BCP data structure described in Section 3.3, the initialization step, as well as the REVERSEHUNGARIANSEARCH procedure, can be executed in $\tilde{O}(nk\Phi(n))$ time.

4.3 Extension of the Fast Hungarian Algorithm to the k-SPI Problem

For the k-SPI problem, given $\mathcal{G}_{\sigma'}$ with $|A| = n + k$ and $|B| = n$, the goal is to compute a minimum-cost n-matching of A and B that matches all points in B. We extend our algorithm for the k-SP problem to the k-SPI problem in a straightforward way as follows: Our algorithm initializes a matching M corresponding to the 1-SP optimal solution for the requests σ. It then iteratively introduces a new server's initial location and finds a minimum net-cost alternating path from the free point of A corresponding to the new server to a matched point of A. The details are provided next.

Set $M \leftarrow \{(a^1, b_1)\} \cup \{(a_i, b_{i+1}) \mid i \in [1, n-1]\}$ as a matching that corresponds to the 1-partitioning $\langle s_1, r_1, r_2, \ldots, r_n \rangle$ with the s_1 as the initial location. Assign dual weights to the points in $A \cup B$ as follows. Set $y(a_n) \leftarrow 0$. Starting from $i = n-1$, set $y(a_i) \leftarrow \max\{0, \max_{j>i+1} y(b_j) - \mathrm{d}(a_i, b_j)\}$ and $y(b_{i+1}) \leftarrow \mathrm{d}(a_i, b_{i+1}) + y(a_i)$. Finally, set $y(a^1) \leftarrow \max\{0, \max_i y(b_i) - \mathrm{d}(a^1, b_i)\}$ and $y(b_1) \leftarrow \mathrm{d}(a^1, b_1) + y(a^1)$. Define $\mathcal{G} = \mathcal{G}_{\{s_1\}\sigma}$ as the graph representation of the 1-SPI problem with s_1 as the only server. The matching $M, y(\cdot)$ is a dual-optimal $(n-1)$-matching on $\mathcal{G}$.

In each iteration $1 \leq t \leq k-1$, our algorithm adds a point a^{t+1} to the vertex set A of the graph $\mathcal{G}$ and connects it to all points $b_i \in B$ with a cost $\mathrm{d}(a^{t+1}, b_i) = \mathrm{d}(s_{t+1}, r_i)$. Assign $y(a^{t+1}) \leftarrow \max\{0, \max_i y(b_i) - \mathrm{d}(a^{t+1}, b_i)\}$. Construct a reversed residual graph, similar to the one constructed for the k-SP problem, with the difference that the source vertex s is only connected to the point a^{t+1} with a zero cost. Execute the REVERSEHUNGARIANSEARCH procedure on the reversed residual graph to update the dual weights and compute an admissible alternating path P from a^{t+1} to a point $a \in A$ with a zero dual weight. Set $M \leftarrow M \oplus P$.

5 Bipartite Matching on Randomly Colored Points

Given a set U of $2n$ points inside the unit square, suppose A is a subset of n points of U chosen uniformly at random and let $B = U \setminus A$. Define G to be a complete bipartite graph between A and B, i.e., $G = (A \cup B, E = A \times B)$ is a bipartite graph on the vertex set $A \cup B$ with an edge set $E = A \times B$. In this section, we show that our algorithm from Section 3 can be used to compute the minimum-cost perfect matching between A and B under ℓ_2^q distances, for any $q \geq 1$, in $\tilde{O}(n^{7/4}\Phi(n)\log\Delta)$ time in expectation. For simplicity in presentation, we first present our analysis for $q = 2$. In Section E.2 in the appendix, we extend our analysis to any dimension $d \geq 2$ and any $q \geq 1$.

To compute a minimum-cost perfect matching between A and B, we construct a hierarchical partitioning $\mathcal{H}$ as described in Section 2.1 with a parameter $\lambda = 9n^{-1/4}$. We then execute our algorithm from Section 3, where the parameter k is set to 0. In this way, our algorithm computes a minimum-cost $\mathcal{C}^*$-extended n-matching $M^{\mathcal{C}^*}$ for the partitioning $\mathcal{C}^* = \{\square^*\}$ for the root cell $\square^*$ of $\mathcal{H}$. In the following lemma, we show that the matching M of the extended matching $M^{\mathcal{C}^*}$ computed by our algorithm is a minimum-cost perfect matching, as desired.

Lemma 5.1. *Given the partitioning $\mathcal{C}^* = \{\square^*\}$ and any minimum-cost $\mathcal{C}^*$-extended n-matching $M^{\mathcal{C}} = (M, B^{\mathcal{C}^*})$, the matching M is a minimum-cost n-matching.*

Proof. For the sake of contradiction, suppose the matching M is not an n-matching and there exists a boundary-matched point $b \in B^{\mathcal{C}^*}$. In this case, there is a free point $a \in A$, where $\mathrm{d}(a, b) \leq \mathrm{D} < \mathrm{d}(b, \mathcal{C}^*)$, where D is the diameter of the points $A \cup B$ i.e., matching the point b to the free point a instead of the

boundaries of $\square^*$ reduces the cost of the extended matching, which is a contradiction to the assumption that $M^{\mathcal{C}^*}$ is a minimum-cost extended matching. Hence, $B^{\mathcal{C}^*} = \emptyset$, and M is a (minimum-cost) n-matching. $\square$

We next analyze the running time of our algorithm, leading to the following lemma.

Lemma 5.2. *Suppose U is a set of $2n$ points inside the unit square and A is a subset chosen uniformly at random from all subsets of size n. Let $B = U \setminus A$. Then, our algorithm computes the minimum-cost matching on the complete bipartite graph on A and B under squared Euclidean distances in $\tilde{O}(n^{7/4} \log \Delta)$ expected time.*

Similar to our efficiency analysis in Section 3.2, we first show that for any cell $\square$ of $\mathcal{H}$, there exists a low-cost high-cardinality matching inside $\square$. We then use this matching to prove a bound on the number of iterations of the Hungarian search procedure.

Lemma 5.3. *For any cell $\square$ of $\mathcal{H}$, there exists a matching M' that, in expectation, matches all except $O(n_\square^{3/4})$ points of $B_\square$ and has a cost $O(\ell_\square^2 n_\square^{1/4})$.*

For any cell $\square$ of $\mathcal{H}$, we next show that the number of iterations of the merge step and the total number of executions of the extended Hungarian search procedure on $\square$ is $\tilde{O}(n^{3/4})$.

Number of Iterations. Let $B_\square^{\mathcal{C}}$ denote the set of all boundary-matched points of $B_\square$ that are matched to the divider $\Gamma_\square$ in the matching maintained by our algorithm before executing the merge step on $\square$. We first show in Lemma 5.4 that the total number of iterations of the merge step and the extended Hungarian search procedure on $\square$ is $|B_\square^{\mathcal{C}}|$.

Lemma 5.4. *For any cell $\square$ of $\mathcal{H}$, the total number of iterations of the merge step and the total number of executions of the search procedure on $\square$ is $O(|B_\square^{\mathcal{C}}|)$.*

We next show that $|B_\square^{\mathcal{C}}| = O(n^{3/4})$. Since each iteration can be executed in $\tilde{O}(n_\square \Phi(n))$ time, the total processing time of our algorithm for each cell $\square$ of $\mathcal{H}$ is $\tilde{O}(n^{3/4} n_\square \Phi(n))$, and the total execution time across all cells of $\mathcal{H}$ would be $\tilde{O}(n^{7/4} \Phi(n) \log \Delta)$, leading to Lemma 5.2.

Lemma 5.5. *For any cell $\square$ of $\mathcal{H}$, $|B_\square^{\mathcal{C}}| = O(n^{3/4})$.*

Proof. Let M' denote the matching constructed in Lemma 5.3, and let $\mathcal{P}_{\text{aug}}$ (resp. $\mathcal{P}_{\text{alt}}$) denote the set of augmenting paths (resp. alternating paths) with an endpoint in $B_\square^{\mathcal{C}}$ in the symmetric difference $M_\square \oplus M'$. Then, $|\mathcal{P}_{\text{alt}}| = O(n^{3/4})$ since the number of alternating paths P in $\mathcal{P}_{\text{alt}}$ is bounded by the number of free points in M'. We partition the free endpoints of $\mathcal{P}_{\text{aug}}$ into the set $B_{\text{aug}}^{\text{close}}$ that are at a distance closer than $\lambda'_\square = \ell_\square n^{-1/4}$ to $\Gamma_\square$ and the set $B_{\text{aug}}^{\text{far}}$ that are at a distance further than $\lambda'_\square$ from $\Gamma_\square$. By Lemma 2.1, only $O(n_\square^{3/4})$ points are close to $\Gamma_\square$ and $|B_{\text{aug}}^{\text{close}}| = O(n^{3/4})$. For points in $B_{\text{aug}}^{\text{far}}$, we show that the dual weight of the points in $B_{\text{aug}}^{\text{far}}$ is bounded by the cost of M', and each point in $B_{\text{aug}}^{\text{far}}$ has a dual weight of at least $\lambda'_\square = \ell_\square n^{-1/4}$. Therefore, $|B_{\text{aug}}^{\text{far}}| \leq \frac{w(M')}{\lambda'_\square} = O(n^{4/5})$. The complete proof is provided in Appendix E.1. $\square$

Each iteration of the merge step and each execution of the search procedure takes $\tilde{O}(n_\square \Phi(n))$ time and therefore, the total execution time of our algorithm on $\square$ would be $\tilde{O}(n^{3/4} n_\square \Phi(n))$. Since each point participates in $O(\log(n\Delta))$ cells in $\mathcal{H}$, the total execution time of our algorithm across all cells would be $\tilde{O}(n^{7/4} \Phi(n) \log \Delta)$, proving Lemma 5.2.

6 Conclusion and Open Questions

In this paper, we presented an exact algorithm for the k-SP and k-SPI problems for the case where requests are d-dimensional points with a spread bounded by Δ. Our algorithm makes sub-quadratic calls to a weighted nearest-neighbor data structure and has a logarithmic dependence on the spread Δ. We conclude by raising two important open questions:

First, can we eliminate the $O(\log \Delta)$ dependency in the execution time of our algorithm? Achieving this would enable sub-quadratic algorithms for geometric bipartite matching, providing significant progress on a longstanding open question in computational geometry.

Second, we reduced the computation of the optimal offline solution for the k-server problem to finding the minimum-cost bipartite matching on a dense bipartite graph $\mathcal{G}_\sigma$. This reduction also proposes the k-server problem (online version) as a mild variant of the online metric matching problem. There has been extensive work on designing algorithms for the online k-server problem [2, 4, 12, 13, 15, 17, 22, 26]. By leveraging this reduction, the implementation of the work function algorithm, such as the scalable approach by Raghvendra and Sowle [22] can be simplified and improved. Furthermore, the best-known algorithm for online metric matching, the RM algorithm [18, 20], can be adapted to solve the online k-server problem. Notably, it has been shown that the RM algorithm has a better competitive ratio compared to the work function algorithm for online bipartite matching on a line metric [21]. Can we derive better upper bounds for the competitive ratio of the RM algorithm when applied to the k-server problem? Such results would mark progress toward proving the deterministic k-server conjecture.

Acknowledgement

The research presented in this paper was funded by NSF CCF-2223871. We thank the anonymous reviewers for their useful feedback.

A Relating Bipartite Matching Problem with the k-SP and k-SPI Problems

A.1 Reducing k-SP and k-SPI Problems to the Geometric Bipartite Matching Problem

In this section, we show that the k-sequence partitioning problem (resp. the k-sequence partitioning with initial locations problem) reduces to the problem of computing a minimum-cost matching of size $n - k$ (resp. n) in a bipartite graph with $2n$ (resp. $2n + k$) vertices, leading to Lemma 1.2.

Graph Representation. Given an input sequence σ for the k-SP problem, recall that in our reduction, we create two vertex sets A and B containing n points each. For each request r_i, we create a vertex b_i (resp. a_i) in B (resp. A) as the entry (resp. exit) gate for request r_i. The set of edges is created as follows. For any request r_i and every $j > i$, we connect the exit gate a_i of request r_i to the entry gate b_j of request r_j and assign it a cost $\mathrm{d}(a_i, b_j) = \|r_i - r_j\|_p$. This completes the construction of the graph $\mathcal{G}_\sigma$. Note that $|A| = |B| = n$.

Relating a Partial Matching to a k-Partitioning. Consider any matching M of size $(n-k)$ in $\mathcal{G}_\sigma$. There are exactly k requests whose entry gates are free in M and k requests whose exit gates are free in M. All these requests have at most one edge incident on them. Every other request has edges incident on both the entry and exit gates. Therefore, with respect to the matching edges, k requests have a degree at most one and all other requests have a degree exactly two. It is easy to see that these edges do not form a cycle, as there is no edge between the exit gate a_i of a request r_i to the entry gate b_j of a request r_j with $j < i$; therefore, the set of matching edges partitions the requests into a set of k paths, representing the k-partitioning for the k-SP problem. The following lemma states and proves our reduction more formally.

Lemma A.1. *Computing a minimum-cost k-partitioning for a sequence of requests σ reduces to computing a minimum-cost $(n-k)$-partial matching in the graph representation $\mathcal{G}_\sigma$.*

Proof. Consider any k-partitioning $\Sigma = \{\varsigma_1, \ldots, \varsigma_k\}$ of σ. We construct a matching M_Σ representing Σ as follows. For any subsequence $\varsigma_i \in \Sigma$ and any pair of consecutive requests r_{i_1}, r_{i_2} in ς_i, we add the edge (a_{i_1}, b_{i_2}) to the matching M_Σ. Since each request $r_i \in \sigma$ is included in exactly one subsequence of Σ, at most one edge of M_Σ is incident on each point of $A \cup B$. Furthermore, for each sub-sequence $\varsigma \in \Sigma$, among all entry and exit gates of the requests in ς, only the entry gate of the first request and the exit gate of the final request in ς are free in M_Σ. Hence, M_Σ is a matching of size $n - k$ and by the definition of the cost of k-partitionings and matchings, has the same cost as Σ.

Next, given a $(n-k)$-partial matching M on $\mathcal{G}_\sigma$, we construct a k-partitioning Σ_M of the requests σ with the same cost as M. Our construction relies on the following observation: since M is a $(n-k)$-partial matching, all except k points in B are matched in M. For each request $r_i \in \sigma$ whose entry gate b_i is free in M, we construct a subsequence ς_i of requests by following the matching edges as described next. Initialize $\varsigma_i = \langle r_i \rangle$. At any step, suppose $\varsigma_i = \langle r_{i_1}, \ldots, r_{i_j} \rangle$. If the exit gate a_{i_j} of the last request r_{i_j} is free in M, we stop the construction of ς_i; otherwise, a_{i_j} is connected to an entry gate b_l for a request r_l with $l > i_j$. Add r_l to the subsequence ς_i as $r_{i_{j+1}}$. This completes the construction of k subsequences, one for each of the k free entry gates. It is easy to confirm that the resulting subsequences are a k-partitioning of σ and its cost is equal to the cost of the matching M. □

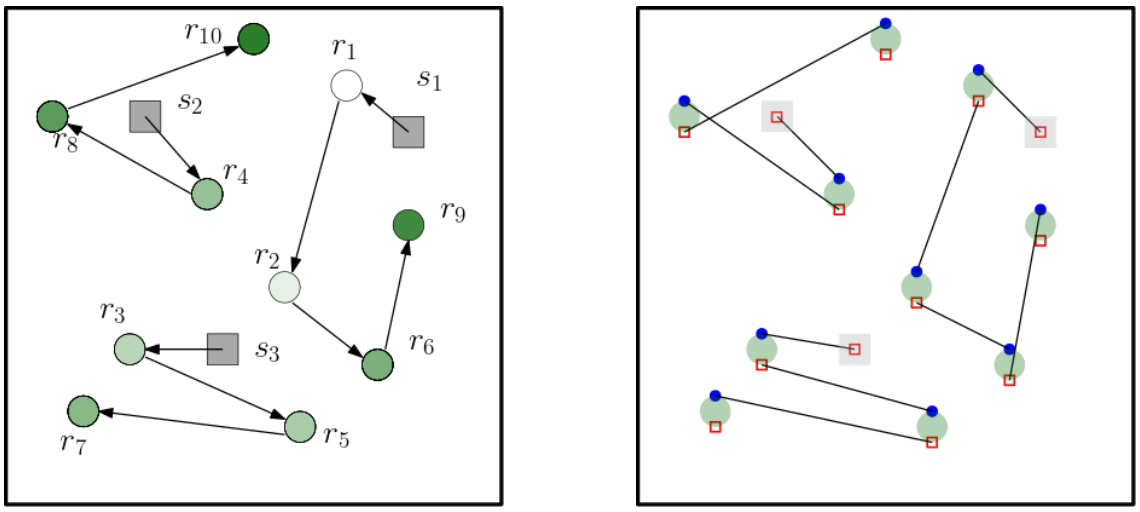


Figure 8: (Left) A 3-partitioning $\langle s_1, r_1, r_2, r_6, r_9\rangle$, $\langle s_2, r_4, r_8, r_{10}\rangle$, $\langle s_3, r_3, r_5, r_7\rangle$ for the 3-SPI problem and (right) its corresponding maximum matching on the graph representation of the problem.

Extension to the k-SPI Problem. For the k-sequence partitioning with initial locations problem, given a sequence σ of n requests and a set η of the initial locations of the k servers, we create two vertex sets A and B, where $|A| = n + k$ and $|B| = n$. For each request $r_i \in \sigma$, we create a vertex b_i (resp. a_i) in B (resp. A) as the entry (resp. exit) gate for request r_i. We also add a vertex a^j for each server $s_j \in \eta$ to A. The set of edges is created as follows. For any server s_j, we add an edge from a^j to the entry gate b_i of all requests r_i with a cost $\mathrm{d}(a^j, b_i) = \|s_j - r_i\|_p$. For any request r_i and every $j > i$, we connect the exit gate a_i of request r_i to the entry gate b_j of request r_j and assign it a cost $\mathrm{d}(a_i, b_j) = \|r_i - r_j\|_p$. See Figure 8. The following lemma, whose proof is a straightforward extension of the proof of Lemma A.1, states that the k-SPI problem reduces to the minimum-cost maximum matching problem on $\mathcal{G}_\sigma$.

Lemma A.2. *Computing a minimum-cost k-partitioning with initial locations for a sequence of requests σ and a set of servers η reduces to computing a minimum-cost maximum matching in the graph representation $\mathcal{G}_{\eta\sigma}$.*

Proof. For any k-partitioning $\Sigma = \{\varsigma_1, \ldots, \varsigma_k\}$ of σ, we construct a matching M_Σ with the same cost as follows. For any subsequence $\varsigma_i = \langle s_i, r_{i_1}, \ldots, r_{i_t}\rangle$, we add the edge (a^i, b_{i_1}) along with $(a_{i_j}, b_{i_{j+1}})$ for each $j \in [1, t-1]$ to M_Σ. Each server and each request is included in exactly one subsequence of Σ and therefore, each entry gate of each request has a degree exactly one in M_Σ and the exit gate of each server and each request is incident on at most one edge of M_Σ. Hence, M_Σ is a maximum matching, and by the construction of $\mathcal{G}_\sigma$, $w(M_\Sigma)$ is the same as the cost of Σ.

Next, for a maximum matching M on $\mathcal{G}_\sigma$, we construct a k-partitioning Σ_M of the servers and requests $\eta\sigma$ with the same cost as M. For each server s_i, we construct a subsequence ς_i as follows. If a^i is free, then we add the subsequence $\varsigma_i = \langle s_i\rangle$ to Σ_M. Otherwise, a^i is matched to a point b_j. Initialize $\varsigma_i = \langle s_i, r_j\rangle$. At any step, suppose $\varsigma_i = \langle s_i, r_{i_1}, \ldots, r_{i_j}\rangle$. If the exit gate a_{i_j} of the last request r_{i_j} is free in M, we stop the construction of ς_i; otherwise, a_{i_j} is connected to an entry gate b_l for a request r_l with $l > i_j$. Add r_l to the subsequence ς_i as $r_{i_{j+1}}$. This completes the construction of k subsequences, one for each of the k servers. By construction, the cost of Σ_M is $w(M)$. □

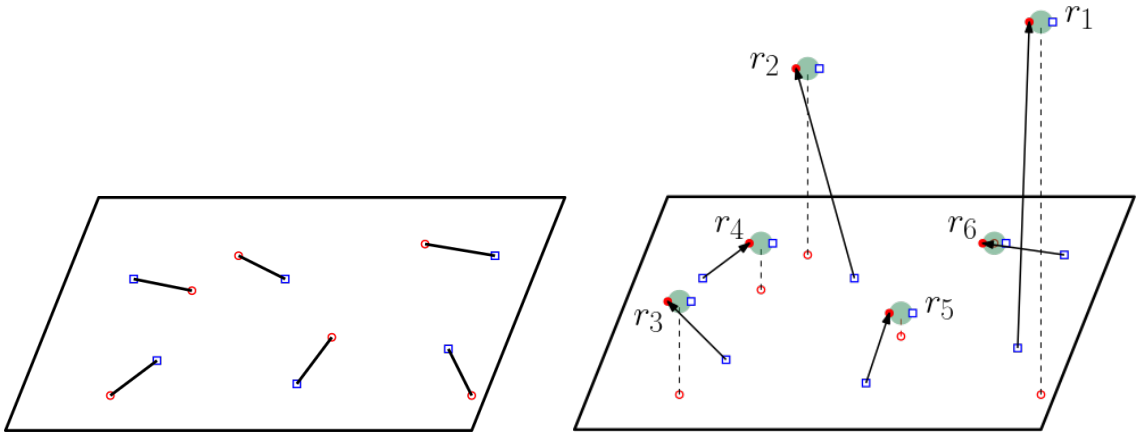


Figure 9: (left) the point set A (red points) and B (blue points) and a minimum-cost bipartite matching between them, (right) the instance of n-SPI problem we construct from $A \cup B$.

A.2 Reducing Geometric Bipartite Matching to the k-SPI Problem

Given an instance of the geometric bipartite matching on d-dimensional points $A = \{a_1, \ldots, a_n\}$ and $B = \{b_1, \ldots, b_n\}$ under the ℓ_1 costs, we create a $(d+1)$-dimensional instance of the k-SPI problem. The points of B will be used to create the initial locations of the n servers, and the points of A will be used to create the request sequence. Our construction will be such that the optimal solution to this n-SPI problem will create sub-sequences of length 2 mapping the initial location of each of the n servers to exactly one request. These subsequences can be shown to also correspond to the minimum-cost matching of A and B. We provide the details below.

Let D denote the diameter of $A \cup B$ under ℓ_1 costs. We create two sets A' and B' of $(d+1)$-dimensional points as follows. For each point $a_i \in A$, we create a point $a'_i \in A'$ by setting the $(d+1)$st coordinate of a_i to $3^{n-i+1}\mathrm{D}$. For each point $b_i \in B$, we create a point $b'_i \in B'$ and set its $(d+1)$st coordinate to 0. Consider the n-SPI problem with $\eta = \langle s_1 = b'_1, \ldots, s_n = b'_n \rangle$ and $\sigma = \langle r_1 = a'_1, \ldots, r_n = a'_n \rangle$ as inputs. By our construction, for any request r_i, any server s_l, and any request r_j with $j < i$,

$$\begin{aligned} \|r_i - s_l\|_1 &= \|a'_i - b'_l\|_1 = 3^{n-i+1}\mathrm{D} + \|a_i - b_l\|_1 \leq 3^{n-i+1}\mathrm{D} + \mathrm{D} \\ &< (3^{n-j+1} - 3^{n-i+1})\mathrm{D} \leq (3^{n-j+1} - 3^{n-i+1})\mathrm{D} + \|a_i - a_j\|_1 \\ &= \|a'_i - a'_j\|_1 = \|r_i - r_j\|. \end{aligned} \tag{12}$$

Intuitively, for any request r_i, it would be cheaper to directly serve r_i by a new server (e.g., s_l) rather than serving r_i by a server used to serve an earlier request (e.g., r_j). We next formally show that the optimal n-partitioning under ℓ_1 norm matches each of the n servers to exactly one request and the matching of servers to requests is also an optimal matching of points in A to B.

Let $\Sigma = \{\varsigma_1, \ldots, \varsigma_n\}$ be an optimal n-partitioning for $\sigma' = \eta\sigma$, where ς_i is the sub-sequence of σ' that is assigned to be served by the server s_i. Suppose there exists a server $s_i \in \eta$ with more than one request in $\varsigma(s_i)$, i.e., $\varsigma(s_i) = \langle s_i, r_{i_1}, \ldots, r_{i_t} \rangle$ for some $t > 1$. In this case, since there are n servers in η, there exists at least one server $s_j \in \eta$ with $\varsigma_j = \langle s_j \rangle$. By Equation (12), $\|s_j - r_{i_t}\|_1 < \|r_{i_{t-1}} - r_{i_t}\|_1$. Therefore, assigning the request r_{i_t} to s_j reduces the cost of Σ, which contradicts the assumption that Σ is optimal; thus, any optimal n-partitioning of σ' matches each server to a single request.

Note that any matching M of A and B can be used to construct an n-partitioning Σ_M of A' and B'. For

any vertex $a_i \in A$, let $b_{m(i)}$ be its match in M.

$$w(M) = \sum_{i=1}^{n} \|a_i - b_{m(i)}\|_1 = \sum_{i=1}^{n} \|a'_i - b'_{m(i)}\|_1 - 3^{n-i}\mathrm{D} = w(\Sigma_M) - \sum_{i=1}^{n} 3^{n-i}\mathrm{D}. \tag{13}$$

In other words, for any matching M and n-partitioning Σ_M, the difference in the cost of M and Σ_M is a constant that is independent of the input. Since the optimal n-partitioning maps each server to a unique request and has the smallest possible cost, its corresponding matching is also a matching of A and B with the smallest possible cost. Therefore, any algorithm that finds an optimal partitioning for the sequence partitioning problem can be used to compute a geometric bipartite matching. Note that our reduction extends to ℓ_q^q distances for any parameter $q \geq 1$ in a straightforward way.

B Missing Proofs and Details of Section 2

In this section, we first present a sweep-line algorithm for constructing the hierarchical partitioning described in Section 2.1. We then present the missing proofs of lemmas in Sections 2.

Constructing the hierarchical partitioning. Recall that, as described in Section 2.1, our algorithm constructs a hierarchical partitioning by recursively partitioning each non-leaf cell of $\mathcal{H}$ into two smaller cells as its children. We describe the partitioning procedure with respect to an arbitrary cell $\square$ of $\mathcal{H}$. Let ℓ_x and ℓ_y denote the width and length of $\square$, and without loss of generality, assume $\ell_x \geq \ell_y$. Let $x_{\min}$ (resp. $x_{\max}$) denote the x coordinate of the left (resp. right) side of $\square$. Recall that $\lambda = 9n^{-1/5}$. Define a priority queue $\mathcal{Q}$ storing two events for each point $u \in A_\square \cup B_\square$, namely an entry event $e_u^{\downarrow}$ at $u_x - \ell_\square\lambda$ and an exit event $e_u^{\uparrow}$ at $u_x + \ell_\square\lambda$; here, u_x denotes the x coordinate of point u. See the events $b^{\downarrow}$ and $b^{\uparrow}$ in Figure 10.

Consider a vertical sweep line that moves from left to right, searching for a value $x^* \in [x_{\min} + \frac{\ell_x}{3}, x_{\min} + \frac{2\ell_x}{3}]$ that minimizes $|\Lambda(x^*)|$; recall that $\Lambda(x) := \{u \in A_\square \cup B_\square : |u_x - x| \leq \ell_\square\lambda\}$ for any $x \in [x_{\min} + \frac{\ell_x}{3}, x_{\min} + \frac{2\ell_x}{3}]$. Let x' denote the state of the sweep line. Intuitively, as the sweep line moves from left to right (i.e., x' increases), any point $u \in A_\square \cup B_\square$ enters the set $\Lambda(x')$ at its entry event and exits $\Lambda(x')$ at its exit event. Thus, we can keep track of the number of points close to the sweep line by incrementing (resp. decrementing) the count at each entry (resp. exit) event and return the value x^* realizing the minimum count during the sweep-line procedure.

More formally, we maintain a value γ representing the number of points of $A_\square \cup B_\square$ at a distance at most $\ell_\square\lambda$ to the sweep line. We also store, for a set of $O(n_\square)$ values $x'' \in [x_{\min}, x_{\max}]$, a function $\eta : [x_{\min}, x_{\max}] \to \mathbb{Z}_{\geq 0}$ representing $|\Lambda(x'')|$. Initially, for the minimum event e in $\mathcal{Q}$, set $\gamma \leftarrow 1$ and $\eta(e) \leftarrow \gamma$. Iteratively, our sweep-line algorithm extracts the minimum event e from $\mathcal{Q}$. If e is an entry event, set $\gamma \leftarrow \gamma + 1$, and otherwise, e is an exit event and set $\gamma \leftarrow \gamma - 1$. Set $\eta(e) \leftarrow \gamma$. After processing all events in $\mathcal{Q}$, define the entry x' of $\eta(\cdot)$ in the middle third of x side of $\square$ with the minimum value, i.e.,

$$x^* := \arg \min_{x \in [x_{\min} + \frac{\ell_\square}{3}, x_{\min} + \frac{2\ell_\square}{3}] \cap S(\eta)} \eta(x),$$

where $S(\eta)$ denotes the support of η. Partition $\square$ using the vertical line $x = x^*$ into two smaller cells $\square_1$ and $\square_2$, and add the non-empty ones as the children of $\square$ to $\mathcal{H}$.

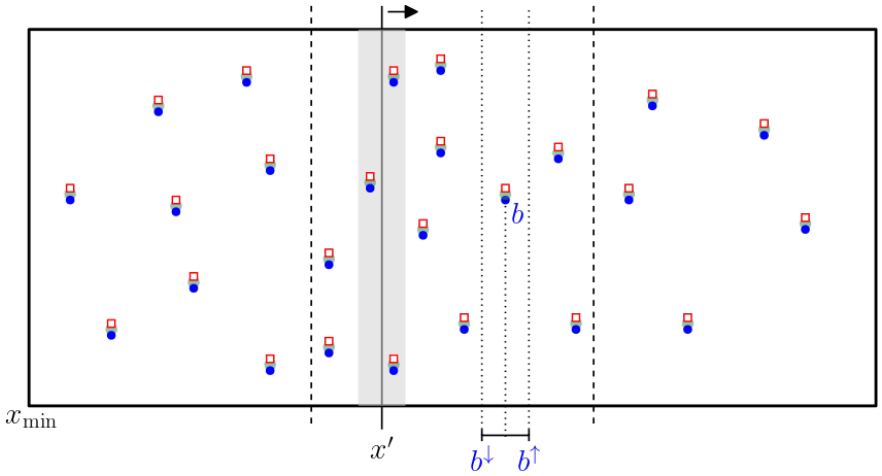


Figure 10: The solid vertical line shows the line our sweep-line procedure maintains. The two values $b^{\downarrow}$ and $b^{\uparrow}$ show the entry and exit events created for the blue point b.

B.1 Missing Proofs of Section 2

Lemma 2.1. *For each cell $\square$ of $\mathcal{H}$, the ratio of the largest to the smallest side of $\square$ is at most 3. Furthermore, the number of points of $A_\square \cup B_\square$ with a distance smaller than $\ell_\square \lambda$ to the divider $\Gamma_\square$ is $O(n_\square \lambda)$.*

Proof. We use an inductive proof to show that the aspect ratio of all cells of $\mathcal{H}$ is at most 3. Note that the root cell $\square^*$ is square and has a unit aspect ratio. For any non-root cell $\square \in \mathcal{H}$, let $\square'$ denote the parent of $\square$ in $\mathcal{H}$. By the inductive hypothesis, the aspect ratio of $\square'$ is at most 3. Let ℓ_x (resp. ℓ_y) denote the width (resp. length) of $\square$. Similarly, let ℓ'_x (resp. ℓ'_y) denote the width (resp. length) of $\square'$. Without loss of generality, assume $\ell'_x \geq \ell'_y$. In this case, we split $\square'$ into two rectangles using a vertical line in the middle third part of the x side of $\square'$, i.e., $\ell_y = \ell'_y$ and $\ell_x \in [\frac{\ell'_x}{3}, \frac{2\ell'_x}{3}]$. In this case, if $\ell_x \geq \ell_y$, then $\frac{\ell_x}{\ell_y} \leq \frac{\ell'_x}{\ell'_y} \leq 3$. Otherwise, $\ell_x < \ell_y$ and $\frac{\ell_y}{\ell_x} \leq \frac{\ell'_y}{\ell'_x/3} \leq 3$.

Next, we show that the number of points of $A_\square \cup B_\square$ at a distance at most $\ell_\square \lambda$ from the divider $\Gamma_\square$ is $O(n_\square \lambda)$. Consider the set $\zeta = \{x_{\min} + \frac{\ell_\square}{3} + 3i\ell_\square\lambda : 0 \leq i \leq \lfloor \frac{1}{9\lambda} \rfloor\}$. Recall that for any value $x' \in [x_{\min} + \frac{\ell_\square}{3}, x_{\min} + \frac{2\ell_\square}{3}]$, $\Lambda(x')$ denotes the set of all points of $A_\square \cup B_\square$ that are at a distance at most $\ell_\square\lambda$ from the vertical line $x = x'$. Note that for any pair of distinct values $x_1, x_2 \in \zeta$, $\Lambda(x_1) \cap \Lambda(x_2) = \emptyset$. See Figure 11. Furthermore,

$$\sum_{x' \in \zeta} |\Lambda(x')| \leq n_\square.$$

Define $x^* := \arg\min_{x' \in \zeta} |\Lambda(x')|$. In this case, the size of $\Lambda(x^*)$ is no more than the average size of the $\Lambda(x')$ for the values $x' \in \zeta$; more precisely,

$$|\Lambda(x^*)| \leq \frac{\sum_{x' \in \zeta} |\Lambda(x')|}{|\zeta|} \leq 9\lambda n_\square.$$

□

Lemma 2.2. *For any cell $\square$ of $\mathcal{H}$, there exists a matching M' between $A_\square \cup B_\square$ that matches all except $O(n_\square^{4/5})$ points of $B_\square$ and has a cost $O(\ell_\square n_\square^{3/5})$.*

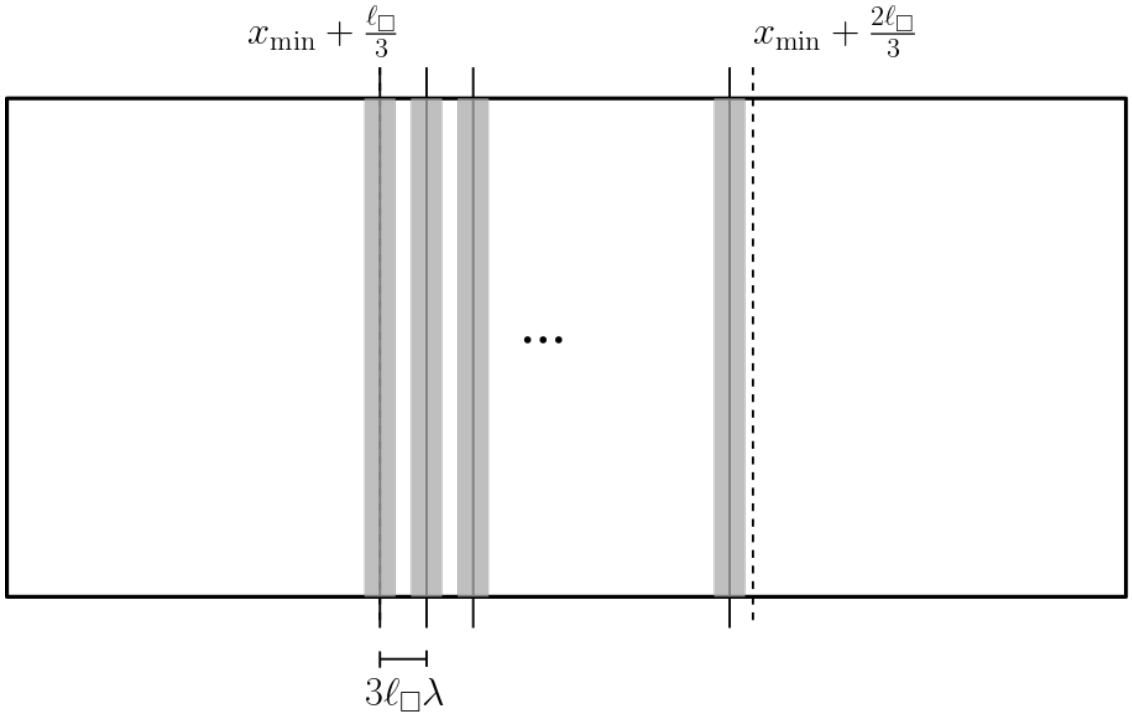


Figure 11: The solid vertical line shows the set ζ.

Proof. Define $\mathbb{G}$ to be a grid of cell-side-length $\ell_\square n_\square^{-2/5}$ that partitions $\square$ into $O(n_\square^{4/5})$ equal-sized rectangles. For each rectangle ξ of the grid $\mathbb{G}$, let $\sigma_\xi = \langle r'_1, \ldots, r'_m \rangle$ denote the sub-sequence of requests in σ that lie inside ξ, and let A_ξ (resp. B_ξ) denote the subset of $A_\square$ (resp. $B_\square$) that lie inside ξ. Define $\mathcal{G}_\xi$ as the sub-graph of $\mathcal{G}$ induced by $A_\xi \cup B_\xi$. Let M_ξ denote the set of edge $(a_{r'_i}, b_{r'_{i+1}})$ for all $i \in [1, m-1]$. M_ξ is a matching on $\mathcal{G}_\xi$ that matches all except one point of B_ξ. Furthermore, since each matching edge in M_ξ has a cost at most $2\ell_\square n_\square^{-2/5}$, the cost of M_ξ would be $O(\ell_\square |B_\xi| n_\square^{-2/5})$.

Define $M' := \bigcup_{\xi \in \mathbb{G}} M_\xi$ as the union of the matchings computed inside each cell of the grid $\mathbb{G}$. Since $|\mathbb{G}| = O(n_\square^{4/5})$ and each cell leaves at most one point of $B_\square$ unmatched, the matching M' matches all except $O(n_\square^{4/5})$ points of $B_\square$. In addition, the cost of M is at most

$$w(M') = \sum_{\xi \in \mathbb{G}} w(M_\xi) = O\left(\ell_\square n_\square^{-2/5} \sum_{\xi \in \mathbb{G}} |B_\xi|\right) = O(\ell_\square n_\square^{3/5}).$$

□

We next present a set of definitions, which are used in the proof of Lemma 2.3. Given a matching M on the graph $\mathcal{G}_\sigma$, any simple path P is an alternating path if the edges of P alternate between matching and non-matching edges with respect to M. The path P is called a (standard) augmenting path if it starts from an unmatched point $b \in B$ and ends at an unmatched point $a \in A$. One can augment M along P by setting $M \leftarrow M \oplus P$. The net-cost of P is defined as $\phi(P) := \sum_{(a,b) \in P \setminus M} d(a,b) - \sum_{(a,b) \in P \cap M} d(a,b)$, i.e., the net-cost of P is the change in the matching cost introduced by augmenting M along P.

Lemma 2.3. *Suppose $\mathcal{C}^* = \{\square^*\}$, where $\square^*$ is the root cell of $\mathcal{H}$. Let $M^{\mathcal{C}^*} = (M, B^{\mathcal{C}^*})$ be a minimum-cost extended t-matching on $\mathcal{G}_\sigma$, for $t < n$. Then, the matching M is a minimum-cost t-matching.*

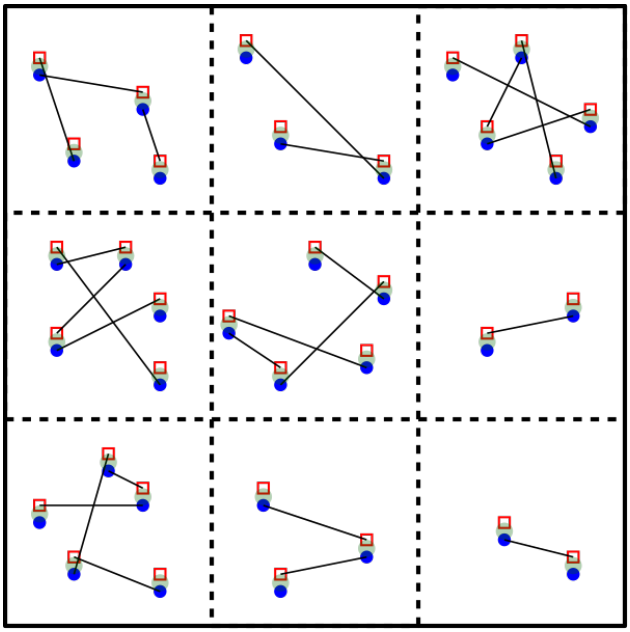


Figure 12: A matching M_ξ is computed inside each square ξ by connecting the requests in their arrival order.

Proof. For the sake of contradiction, suppose the matching M is not a t-matching, and there exist boundary-matched points in $B^{\mathcal{C}^*}$. To prove this lemma, we show that there exists an extended matching $\hat{M}^{\mathcal{C}^*} = (\hat{M}, \hat{B}^{\mathcal{C}^*})$ of size t such that $|\hat{M}| > |M|$ and $w_{\mathcal{C}^*}(\hat{M}^{\mathcal{C}^*}) < w_{\mathcal{C}^*}(M^{\mathcal{C}^*})$, contradicting the assumption that $M^{\mathcal{C}^*}$ is a minimum-cost extended t-matching.

Consider a minimum net-cost augmenting path P with respect to the matching M from an unmatched point $b \in B$. Since the path P has a length of at most $2n-1$, the net-cost of P is

$$\phi(P) = \sum_{(a,b)\in P\setminus M} d(a,b) - \sum_{(a,b)\in P\cap M} d(a,b) \leq \sum_{(a,b)\in P\setminus M} d(a,b) \leq 2n,$$

where the last inequality holds since all points are in the unit square and the ℓ_p distance of each pair is at most 2. By augmenting the matching M along the path P, we get a matching $\hat{M}$ with a cost $w(\hat{M}) = w(M) + \phi(P) \leq w(M) + 2n$. Also note that by the construction of $\square^*$, for any point $b' \in B$, $\mathrm{d}(b', \square^*) \geq 3n-1$. If the endpoint b of P is in $B^{\mathcal{C}^*}$, then for the extended matching $\hat{M}^{\mathcal{C}^*} = (\hat{M}, B^{\mathcal{C}^*} \setminus \{b\})$,

$$\begin{aligned} w_{\mathcal{C}^*}(\hat{M}^{\mathcal{C}^*}) &= w(\hat{M}) + \sum_{b'\in B^{\mathcal{C}^*}\setminus\{b\}} \mathrm{d}(b', \mathcal{C}^*) \leq w(M) + 2n + \sum_{b'\in B^{\mathcal{C}^*}\setminus\{b\}} \mathrm{d}(b', \mathcal{C}^*) \\ &< w(M) + \mathrm{d}(b, \mathcal{C}^*) + \sum_{b'\in B^{\mathcal{C}^*}\setminus\{b\}} \mathrm{d}(b', \mathcal{C}^*) = w_{\mathcal{C}^*}(M^{\mathcal{C}^*}). \end{aligned}$$

Otherwise, let $\hat{b} \in B^{\mathcal{C}^*}$ denote an arbitrary boundary-matching point of $M^{\mathcal{C}^*}$. Then, for the extended matching $\hat{M}^{\mathcal{C}^*} = (\hat{M}, B^{\mathcal{C}^*} \setminus \{\hat{b}\})$,

$$\begin{aligned} w_{\mathcal{C}^*}(\hat{M}^{\mathcal{C}^*}) &= w(\hat{M}) + \sum_{b'\in B^{\mathcal{C}^*}\setminus\{\hat{b}\}} \mathrm{d}(b', \mathcal{C}^*) \leq w(M) + 2n + \sum_{b'\in B^{\mathcal{C}^*}\setminus\{\hat{b}\}} \mathrm{d}(b', \mathcal{C}^*) \\ &< w(M) + \mathrm{d}(\hat{b}, \mathcal{C}^*) + \sum_{b'\in B^{\mathcal{C}^*}\setminus\{\hat{b}\}} \mathrm{d}(b', \mathcal{C}^*) = w_{\mathcal{C}^*}(M^{\mathcal{C}^*}). \end{aligned}$$

Note that in both cases, $\hat{M}^{\mathcal{C}^*}$ is also of size t, which is a contradiction to the assumption that $M^{\mathcal{C}^*}$ is a minimum-cost extended t-matching. Therefore, the extended matching $M^{\mathcal{C}^*} = (M, B^{\mathcal{C}^*})$ has no boundary-matched points and $|M| = t$. Consequently, $w_{\mathcal{C}^*}(M^{\mathcal{C}^*}) = w(M)$, and since $M^{\mathcal{C}^*}$ has the minimum cost among all extended t-matchings, the matching M would have a minimum cost among all t-matchings on $\mathcal{G}_\sigma$, i.e., M is a minimum-cost t-matching. □

B.2 Auxiliary Lemmas

The following lemmas are useful in proving the lemmas in Sections 2 and 3.

Lemma B.1. *Given a feasible $\mathcal{C}$-extended matching $M^{\mathcal{C}} = (M, B^{\mathcal{C}}), y(\cdot)$ on $\mathcal{G}_\sigma$, let P be any augmenting path from a free point $b \in B$ with respect to $M^{\mathcal{C}}$. If P ends with a free point of A, then $\phi(P) = y(b) + \sum_{(a'',b'')\in P} s(a'', b'')$. If P ends with a point $b' \in B$, then $\phi(P) = y(b) + s(b') + \sum_{(a'',b'')\in P} s(a'', b'')$.*

Proof. Note that by the definition of the slack of an edge, for any non-matching edge $(a, b) \in P$, $\mathrm{d}(a, b) = s(a, b) + y(b) - y(a)$. Furthermore, since the slack on the matching edges are 0, for each matching edge $(a, b) \in M$, $-\mathrm{d}(a, b) = s(a, b) - y(b) + y(a)$. If P is an augmenting path from b to a free point $a \in A$ (case (i)), then

$$\begin{aligned}
\phi(P) &= \sum_{(a'',b'')\in P\setminus M} \mathrm{d}(a'', b'') + \sum_{(a'',b'')\in P\cap M} (-\mathrm{d}(a'', b'')) \\
&= \sum_{(a'',b'')\in P\setminus M} (s(a'', b'') + y(b'') - y(a'')) + \sum_{(a'',b'')\in P\cap M} (s(a'', b'') - y(b'') + y(a'')) \\
&= \sum_{(a'',b'')\in P} s(a'', b'') + y(b) - y(a) = y(b) \sum_{(a'',b'')\in P} s(a'', b''),
\end{aligned}$$

where the last equality holds since $y(a) = 0$ by Condition (6). Otherwise, P is an alternating path from b to a point $b' \in B$ (case (ii)), and

$$\begin{aligned}
\phi(P) &= \mathrm{d}(b', \mathcal{C}) + \sum_{(a'',b'')\in P\setminus M} \mathrm{d}(a'', b'') + \sum_{(a'',b'')\in P\cap M} (-\mathrm{d}(a'', b'')) \\
&= \mathrm{d}(b', \mathcal{C}) + \sum_{(a'',b'')\in P\setminus M} (s(a'', b'') + y(b'') - y(a'')) + \sum_{(a'',b'')\in P\cap M} (s(a'', b'') - y(b'') + y(a'')) \\
&= \mathrm{d}(b', \mathcal{C}) + \sum_{(a'',b'')\in P} s(a'', b'') + y(b) - y(b') = y(b) + s(b') + \sum_{(a'',b'')\in P} s(a'', b''),
\end{aligned}$$

where the last equality holds since $s(b') = \mathrm{d}(b', \mathcal{C}) - y(b')$ by the definition of the slack of a point. □

The following is a straightforward corollary from Lemma B.1 and the definition of admissible augmenting paths.

Corollary B.2. *Given a feasible $\mathcal{C}$-extended matching $M^{\mathcal{C}} = (M, B^{\mathcal{C}}), y(\cdot)$ on $\mathcal{G}_\sigma$, let P be an admissible augmenting path from a free point $b \in B$ with respect to $M^{\mathcal{C}}$. Then, $\phi(P) = y(b)$.*

The next lemma shows that no edges of a feasible $\mathcal{C}$-extended matching would cross the boundaries of the cells in $\mathcal{C}$, which allows us to localize the computations inside the cells and is essential for the correctness of our algorithm.

Lemma B.3. *For any feasible extended matching $M^{\mathcal{C}} = (M, B^{\mathcal{C}}), y(\cdot)$, no edges of the matching M cross the boundaries of the cells in $\mathcal{C}$.*

Proof. For the sake of contradiction, suppose there is an edge $(a, b) \in M$, where a and b lie inside a cell $\square_a$ and $\square_b$ of $\mathcal{C}$ and $\square_a \neq \square_b$. By the feasibility condition (10),

$$\mathrm{d}(a, b) = y(b) - y(a) \leq y(b).$$

Since a is outside of $\square_b$, then $\mathrm{d}(b, \square_b) < \mathrm{d}(a, b)$. Therefore,

$$y(b) \geq \mathrm{d}(a, b) > \mathrm{d}(b, \square_b) = \mathrm{d}(b, \mathcal{C}),$$

which is a contradiction to the assumption that $M^{\mathcal{C}}, y(\cdot)$ is feasible (Condition (4) is violated). □

B.3 Proof of Lemma 2.4

Lemma 2.4. *Given a feasible $\mathcal{C}$-extended t-matching $M^{\mathcal{C}} = (M, B^{\mathcal{C}})$ and a set of non-negative dual weights $y(\cdot)$ on $A \cup B$, let P be a minimum net-cost augmenting path with respect to $M^{\mathcal{C}}$. Let, for any $\square \in \mathcal{C}$, $y_{\square} = \max_{b' \in B_{\square}} y(b')$. Then,*

(a) all points of P lie inside a single cell of $\mathcal{C}$, and

(b) if, for every cell $\square \in \mathcal{C}$, $y_{\square} \leq \phi(P)$ and for all free points $b \in B_{\square}$, $y(b) = y_{\square}$, then $M^{\mathcal{C}}$ is a minimum-cost extended t-matching.

Proof of Lemma 2.4(a). For the sake of contradiction, suppose $P = \langle b_1, a_1, b_2, \ldots, b_m, a_m \rangle$ is a minimum net-cost augmenting path that does not lie inside a single cell of $\mathcal{C}$. Let $\square$ denote the cell of $\mathcal{C}$ containing b_1, and let (b_i, a_i) be the first edge that goes outside of $\square$, i.e., all vertices $\{b_1, a_1, \ldots, b_i\}$ reside inside $\square$ (Note that, by Lemma B.3, no matching edges of M cross the boundaries of $\mathcal{C}$ and the edge (b_i, a_i) has to be a non-matching edge). For the alternating path $P' = \langle a_i, b_{i+1}, \ldots, a_m \rangle$, the net-cost of P' is

$$\begin{aligned}
\phi(P') &= \sum_{j=i+1}^{m} \mathrm{d}(a_j, b_j) - \sum_{j=i}^{m-1} \mathrm{d}(a_j, b_{j+1}) \\
&= \sum_{j=i+1}^{m} (s(a_j, b_j) + y(b_j) - y(a_j)) + \sum_{j=i}^{m-1} (s(a_j, b_{j+1}) - y(b_{j+1}) + y(a_j)) \\
&= \sum_{j=i+1}^{m} s(a_j, b_j) + \sum_{j=i}^{m-1} s(a_j, b_{j+1}) + y(a_i) - y(a_m) \geq 0,
\end{aligned}$$

where the last inequality holds since a_m is an unmatched point and, by Condition (6), has a zero dual weight, all edges have non-negative slacks, and all points have non-negative dual weights.

Additionally, the cost of matching the point b_i to the boundaries of $\square$ is less than the cost of matching it to the point a_i outside of $\square$, i.e., $\mathrm{d}(b_i, \mathcal{C}) \leq \mathrm{d}(a_i, b_i)$. Define $P'' = \langle b_1, a_1, \ldots, b_i \rangle$. Then,

$$\phi(P) = \phi(P'') + \mathrm{d}(a_i, b_i) + \phi(P') \geq \phi(P'') + \mathrm{d}(a_i, b_i) > \phi(P'') + \mathrm{d}(b_i, \mathcal{C}).$$

Therefore, the augmenting path P'', which is an augmenting path from the free point b_1 to the point b_i (case (ii)) has a lower net-cost than P, contradicting the assumption that P is a minimum net-cost augmenting path.

Proof of Lemma 2.4 (b). To prove this lemma, we first construct a set of dual weights $y'(\cdot)$ (as described in Lemma B.4 below) such that $M^{\mathcal{C}}, y'(\cdot)$ is feasible, $y'(b) = \phi(P)$ for all free points $b \in B$ and $y'(b) \leq \phi(P)$ for all points $b \in B$. We use the dual weights $y'(\cdot)$ as a certificate for the optimality of $M^{\mathcal{C}}$.

Let A_F (resp. B_F) denote the set of free points of A (resp. B) with respect to $M^{\mathcal{C}}$. Let $y_{\max} := \max_{b \in B} y'(b)$. By Condition (6), for each point $a \in A_F$, $y'(a) = 0$, and by Condition (5), for each boundary-matched point $b \in B^{\mathcal{C}}$, $y'(b) = \mathrm{d}(b, \mathcal{C})$. Using Condition (3), we can rewrite the cost of $M^{\mathcal{C}}$ as

$$\begin{aligned} w_{\mathcal{C}}(M^{\mathcal{C}}) &= \sum_{(a,b) \in M} \mathrm{d}(a,b) + \sum_{b \in B^{\mathcal{C}}} \mathrm{d}(b, \mathcal{C}) \\ &= \sum_{(a,b) \in M} y'(b) - y'(a) + \sum_{b \in B^{\mathcal{C}}} y'(b) \\ &= \left(\sum_{b \in B} y'(b) - \sum_{a \in A} y'(a) \right) - \sum_{b \in B_F} y'(b) + \sum_{a \in A_F} y'(a) \\ &= \left(\sum_{b \in B} y'(b) - \sum_{a \in A} y'(a) \right) - |B_F| \cdot y_{\max}. \end{aligned} \tag{14}$$

Let $\hat{M}^{\mathcal{C}} = (\hat{M}, \hat{B}^{\mathcal{C}})$ denote any minimum-cost extended t-matching on $\mathcal{G}_\sigma$. Let $\hat{A}_F$ (resp. $\hat{B}_F$) denote the set of points of A (resp. B) that are free in $\hat{M}^{\mathcal{C}}$. Since both $M^{\mathcal{C}}$ and $\hat{M}^{\mathcal{C}}$ are t-matchings, $|B_F| = |\hat{B}_F|$. Using Conditions (2) and (4),

$$\begin{aligned} w_{\mathcal{C}}(\hat{M}^{\mathcal{C}}) &= \sum_{(a,b) \in \hat{M}} \mathrm{d}(a,b) + \sum_{b \in \hat{B}^{\mathcal{C}}} \mathrm{d}(b, \mathcal{C}) \\ &\geq \sum_{(a,b) \in \hat{M}} y'(b) - y'(a) + \sum_{b \in \hat{B}^{\mathcal{C}}} y'(b) \\ &= \left(\sum_{b \in B} y'(b) - \sum_{a \in A} y'(a) \right) - \sum_{b \in \hat{B}_F} y'(b) + \sum_{a \in \hat{A}_F} y'(a) \\ &\geq \left(\sum_{b \in B} y'(b) - \sum_{a \in A} y'(a) \right) - |\hat{B}_F| \cdot y_{\max}, \end{aligned} \tag{15}$$

where the last inequality holds since $y'(b) \leq y_{\max}$ for each point $b \in B$ and $y'(a) \geq 0$ for each point $a \in A$. Combining Equations (14) and (15),

$$w_{\mathcal{C}}(M^{\mathcal{C}}) = \sum_{b \in B} y'(b) - \sum_{a \in A} y'(a) - |B_F| \cdot y_{\max} \leq w_{\mathcal{C}}(\hat{M}^{\mathcal{C}}).$$

Since $\hat{M}^{\mathcal{C}}$ is a minimum-cost extended t-matching, $w_{\mathcal{C}}(M^{\mathcal{C}}) = w_{\mathcal{C}}(\hat{M}^{\mathcal{C}})$, and $M^{\mathcal{C}}$ is also a minimum-cost extended t-matching.

B.4 Proof of Lemma 2.5

Lemma 2.5. *Suppose $M^{\mathcal{C}}, y(\cdot)$ is a feasible $\mathcal{C}$-extended matching and P is a minimum net-cost augmenting path. For any cell $\square \in \mathcal{C}$, define $y_\square = \max_{b' \in B_\square} y(b')$, and suppose $y_\square \leq \phi(P)$ and $y(b_f) = y_\square$ for all free points $b_f \in B_\square$. Then, one can update the dual weights in $\tilde{O}(n_\square \Phi(n_\square))$ time such that $M^{\mathcal{C}}, y(\cdot)$ remains feasible, $y(b) \leq \phi(P)$ for all $b \in B_\square$, and $y(b_f) = \phi(P)$ for all free points $b_f \in B_\square$.*

Proof. To prove this lemma, we first construct a residual graph $\mathcal{G}_\square$ for each cell $\square \in \mathcal{C}$ with respect to the extended matching $M^{\mathcal{C}} = (M, B^{\mathcal{C}})$. Our construction is identical to what is described in Section 2.3. The vertex set of $\mathcal{G}_\square$ is a source vertex s and the points in $A_\square \cup B_\square$. For any edge $(a, b) \in E$ inside $\square$, if (a, b) is a matching edge (resp. non-matching edge) in M, there is an edge directed from a to b (resp. from b to a) with a weight $s(a, b)$ in $\mathcal{G}_\square$. Furthermore, there is an edge directed from s to every free point $b \in B$ with a weight $y(b)$.

Define κ_v as the distance of each point $v \in A_\square \cup B_\square$ from the source vertex s. Define $\kappa = \phi(P)$. For any vertex $v \in A_\square \cup B_\square$ with $\kappa_v < \kappa$, set $y'(v) \leftarrow y(v) - \kappa_v + \kappa$; otherwise, set $y'(v) \leftarrow y(v)$. As discussed in Section 3.3, the distances κ_v can be computed in $\tilde{O}(n_\square \Phi(n_\square))$ time. Given the distances from the source, computing the set of dual weights $y'(\cdot)$ from $y(\cdot)$ takes $O(n_\square)$ time, and therefore, the total construction takes $\tilde{O}(n_\square \Phi(n_\square))$ time.

We next show that $M^{\mathcal{C}}, y'(\cdot)$ is feasible, $y'(b) \leq \phi(P)$ for all points $b \in A_\square \cup B_\square$, and $y'(b_f) = \phi(P)$ for all free points $b_f \in B_\square$. Note that by Lemma B.3, since the extended matching $M^{\mathcal{C}}, y(\cdot)$ is feasible, then no matching edges of $M^{\mathcal{C}}$ cross the boundaries of the cells in $\mathcal{C}$.

Feasibility of points.

1. For any point $b \in B \setminus B_\square$, $y'(b) = y(b)$ and Conditions (4) and (5) holds. Similarly, for any free point $a \in A \setminus A_\square$, $y'(a) = y(a) = 0$ and Condition (6) holds.

2. For any point $b \in B_\square$:

   - if b is a free point, then $\kappa_b = y(b)$, since the only path from s to b is an edge from s to b with a weight $y(b)$. In this case, $y'(b) = y(b) - \kappa_b + \kappa = \kappa$.
   - if b is a boundary-matched point, then there are no edges coming into b in $\mathcal{G}_\square$ and therefore, $y'(b) = y(b) = \mathrm{d}(b, \mathcal{C})$ and Condition (5) holds.
   - Otherwise, b is a matched point. In this case, we show that $\kappa \leq \kappa_b + s(b)$: Let P_b denote the shortest path from s to b, and let P' denote the path obtained by removing s from P_b, which has a free endpoint $b' \in B_\square$. If $\kappa > \kappa_b + s(b)$, then the net-cost of the augmenting path P' is, by Lemma B.1,

     $$\phi(P') = y(b') + s(b) + \sum_{(a'', b'') \in P} s(a'', b'') = \kappa_b + s(b) < \kappa = \phi(P),$$

     which contradicts the assumption that P is a minimum net-cost augmenting path. Therefore, $\kappa \leq \kappa_b + s(b)$ and Condition (4) holds for b since

     $$y'(b) = y(b) + \kappa - \kappa_b \leq y(b) + s(b) = \mathrm{d}(b, \mathcal{C}). \tag{16}$$

3. For any free point $a \in A^F_\square$, $\kappa_a \geq \kappa$, since otherwise, if $\kappa_a < \kappa$, then the path from the source to a defines an augmenting path whose net-cost is $\kappa_a < \kappa = \phi(P)$, which contradicts the assumption that P is a minimum net-cost augmenting path. Thus, $\kappa_a \geq \kappa$ and the procedure does not update the dual weight of a, i.e., $y'(a) = 0$ satisfying Condition (6).

Feasibility of edges. For any edge $(a, b) \in E$, let $s(a, b)$ denote the slack of (a, b) with respect to $y(\cdot)$. For any edge $(a, b) \in E$:

1. if $a \in A \setminus A_\square$ and $b \in B \setminus B_\square$, then $y'(a) = y(a)$ and $y'(b) = y(b)$ and Conditions (2) and (3) remains satisfied for (a, b).

2. if $a \in A_\square$ and $b \in B \setminus B_\square$, then the edge (a, b) is a non-matching edge, $y'(b) = y(b)$, and $y'(a) \geq y(a)$; hence, $y'(b) - y'(a) \leq y(b) - y(a) \leq \mathrm{d}(a, b)$ and Condition (2) is satisfied.

3. if $a \in A \setminus A_\square$ and $b \in B_\square$, then (a, b) is a non-matching edge and as discussed above, $y'(b) \leq \mathrm{d}(b, \mathcal{C}) \leq \mathrm{d}(a, b)$; hence, $y'(b) - y'(a) \leq y'(b) \leq \mathrm{d}(a, b)$ and Condition (2) is satisfied.

4. if $a \in A_\square$ and $b \in B_\square$:

   - If $(a, b) \in M$ is a matching edge, then $\kappa_b = \kappa_a$ since the only edge directed to b in the residual graph is the zero-slack edge (a, b). Thus, Condition (3) holds since

     $$y'(b) - y'(a) = (y(b) + \kappa - \kappa_b) - (y(a) + \kappa - \kappa_a) = y(b) - y(a) = \mathrm{d}(a, b).$$

   - Otherwise, (a, b) is a non-matching edge and $\kappa_a \leq \kappa_b + s(a, b)$, and Condition (2) holds since

     $$y'(b) - y'(a) = (y(b) + \kappa - \kappa_b) - (y(a) + \kappa - \kappa_a) \leq y(b) - y(a) + s(a, b) = \mathrm{d}(a, b).$$

     Note that if (a, b) is a non-matching edge on the shortest path tree, then $\kappa_a = \kappa_b + s(a, b)$ and $y'(b) - y'(a) = \mathrm{d}(a, b)$, i.e., (a, b) is admissible.

Finally, we show that the dual weight of each point $v \in A_\square \cup B_\square$ is at most $\phi(P)$. For each point $b \in B_\square$ such that $y'(b) > y(b)$, we have $\kappa_b < \kappa$. Suppose P_b denotes the shortest path from the source s to b, and let P' be the path obtained by removing s from P_b. Let b' denote the free endpoint of P'. By the assumption of the lemma, $y(b') = y_\square \geq y(b)$. Furthermore, since P' is a path on the shortest path tree, $\kappa_b = \kappa_{b'} + \sum_{(u,v) \in P'} s(u, v)$ and since all slacks are non-negative, $\kappa_{b'} \leq \kappa_b$; therefore,

$$y'(b) = y(b) - \kappa_b + \kappa \leq y(b') - \kappa_{b'} + \kappa = \kappa.$$

□

The following lemma is resulted from applying Lemma 2.5 on all cells $\square \in \mathcal{C}$.

Lemma B.4. *Given a partitioning $\mathcal{C}$ and a feasible extended matching $M^{\mathcal{C}}, y(\cdot)$, let P denote a minimum net-cost augmenting path. Suppose $y(b) \leq \phi(P)$ for all points $b \in B$ and $y(b) = \max_{b' \in B_\square} y(b')$ for all cells $\square \in \mathcal{C}$ and all free points $b \in B_\square$. Then, there exists a set of dual weights $y'(\cdot)$ such that $M^{\mathcal{C}}, y'(\cdot)$ is feasible, $y'(b) \leq \phi(P)$ for all $b \in B$, and $y'(b) = \phi(P)$ for all free points $b \in B$.*

B.5 Proof of Lemma 2.6

Lemma 2.6. *For a cell $\square$ of $\mathcal{H}$, suppose $\square'$ and $\square''$ denote its two children, and let $\mathcal{C}$ denote a partitioning containing $\square'$ and $\square''$. Let $\mathcal{C}' = \mathcal{C} \cup \{\square\} \setminus \{\square', \square''\}$. Given a feasible $\mathcal{C}$-extended matching $(M, B^{\mathcal{C}}), y(\cdot)$, let $B^{\mathcal{C}}_\square \subseteq B^{\mathcal{C}}$ denote the subset of boundary-matched points that are matched to the divider $\Gamma_\square$ of $\square$. Then,*

(a) the $\mathcal{C}'$-extended matching $(M, B^{\mathcal{C}} \setminus B^{\mathcal{C}}_\square), y(\cdot)$ is also feasible, and,

(b) $|B^{\mathcal{C}}_\square| = O(n^{4/5})$.

Proof of Lemma 2.6 (a). To prove this lemma, we show that the matching $\hat{M}^{\mathcal{C}} = (M, B^{\mathcal{C}} \setminus B^{\mathcal{C}}_{\square})$ along with the dual weights $y(\cdot)$ satisfy the feasibility conditions (2)–(6). First, note that for any edge $(a, b) \in E$ (resp. matching edge $(a, b) \in M$), the dual weights of a and b as well as their distance is unchanged; hence, Condition (2) (resp. (3)) holds trivially. Similarly, for any free point $a \in A_F$, the dual weight of a remains zero, and for any point $b \in B \setminus B_{\square}$ outside of $\square$, the dual weight of b, as well as its distance to the boundaries of the partitioning remains unchanged; therefore, Conditions (4)–(6) hold for all free points of A and all points of B that reside outside of $\square$. Next, we show that Conditions (4) and (5) are also true for the points of $B_{\square}$.

For any point $b \in B_{\square}$, let $\square_b \in \{\square', \square''\}$ be the child of $\square$ containing b. From the feasibility of $M^{\mathcal{C}}, y(\cdot)$, we have $y(b) \leq \mathrm{d}(b, \mathcal{C}) = \mathrm{d}(b, \square_b) \leq \mathrm{d}(b, \square) = \mathrm{d}(b, \mathcal{C}')$ and Condition (4) holds. For any boundary-matched point $b \in B_{\square} \cap (B^{\mathcal{C}} \setminus B^{\mathcal{C}}_{\square})$, since b is matched to the boundaries of $\square_b$ that are different from the divider $\Gamma_{\square}$, then $\mathrm{d}(b, \mathcal{C}) = \mathrm{d}(b, \mathcal{C}')$ and therefore, Condition (5) holds for all boundary-matched points of $\hat{M}^{\mathcal{C}}$ inside $\square$ as well.

Proof of Lemma 2.6 (b). Let M denote the matching of the extended matching $M^{\mathcal{C}}$, and let M' denote the matching constructed in Lemma 2.2 inside $\square$. From Lemma B.3, any point $b \in B_{\square}$ that is matched in M is matched to a point $a \in A_{\square}$ inside $\square$. Let $M_{\square}$ denote the subset of the matching edges of M that lie inside $\square$. Define $F(M_{\square})$ (resp. $F(M')$) as the set of unmatched points of $B_{\square}$ in the matching $M_{\square}$ (resp. M'). Note that $B^{\mathcal{C}}_{\square} \subseteq F(M_{\square})$. Any simple path P in the symmetric difference $M_{\square} \oplus M'$ from a free point $b \in B^{\mathcal{C}}_{\square}$ is called a (standard) alternating path if P ends at a point $b \in F(M')$ and a (standard) augmenting path if it ends at an unmatched point $a \in A_{\square}$ with respect to $M_{\square}$. Let $\mathcal{P}_{\mathrm{aug}}$ (resp. $\mathcal{P}_{\mathrm{alt}}$) denote the set of (standard) augmenting paths (resp. alternating paths) with an endpoint in $B^{\mathcal{C}}_{\square}$ in the symmetric difference $M_{\square} \oplus M'$. Note that $|B^{\mathcal{C}}_{\square}| = |\mathcal{P}_{\mathrm{aug}}| + |\mathcal{P}_{\mathrm{alt}}|$. For $\mathcal{P}_{\mathrm{alt}}$, since each alternating path in the symmetric difference has one free endpoint in $B^{\mathcal{C}}_{\square}$ and the other free endpoint in $F(M')$, $|\mathcal{P}_{\mathrm{alt}}| \leq |F(M')| = O(n^{4/5})$. Next, we show that $|\mathcal{P}_{\mathrm{aug}}| = O(n^{4/5})$. Similar to Section 2.2, define the net-cost of an augmenting path P as $\phi(P) = \sum_{(a,b) \in P \cap M'} \mathrm{d}(a, b) - \sum_{(a,b) \in P \cap M_{\square}} \mathrm{d}(a, b)$. Then,

$$\sum_{P \in \mathcal{P}_{\mathrm{aug}}} \phi(P) = \sum_{P \in \mathcal{P}_{\mathrm{aug}}} \left(\sum_{(a,b) \in P \cap M'} \mathrm{d}(a, b) - \sum_{(a,b) \in P \cap M_{\square}} \mathrm{d}(a, b) \right)$$
$$\leq \sum_{P \in \mathcal{P}_{\mathrm{aug}}} \left(\sum_{(a,b) \in P \cap M'} \mathrm{d}(a, b) \right) \leq w(M'). \tag{17}$$

For each path $P \in \mathcal{P}_{\mathrm{aug}}$, let $b_P \in B_{\square}$ and $a_P \in A_{\square}$ denote the two end-points of P. Define $B_{\mathrm{aug}} := \{b_P : P \in \mathcal{P}_{\mathrm{aug}}\}$. Using Lemma B.1 and Equation (17),

$$\sum_{b \in B_{\mathrm{aug}}} y(b) = \sum_{P \in \mathcal{P}_{\mathrm{aug}}} y(b_P) \leq \sum_{P \in \mathcal{P}_{\mathrm{aug}}} \phi(P) \leq w(M'). \tag{18}$$

Define $\alpha := \ell_{\square} n^{-1/5}$. Each free point $b \in B_{\mathrm{aug}}$ is called a *close* (resp. *far*) point if the distance of b to the divider of $\square$ is at most (resp. more than) α. Let $B^{\mathrm{close}}_{\mathrm{aug}}$ (resp. $B^{\mathrm{far}}_{\mathrm{aug}}$) denote the set of all close (resp. far) points of B_{aug}. By Lemma 2.1,

$$|B^{\mathrm{close}}_{\mathrm{aug}}| = O(n_{\square} n^{-1/5}) = O(n^{4/5}). \tag{19}$$

For each far point $b \in B^{\mathrm{far}}_{\mathrm{aug}}$, since b is mapped to the divider $\Gamma_{\square}$, $y(b) = \mathrm{d}(b, \Gamma_{\square}) \geq \alpha$; therefore,

$$\sum_{b \in B_{\mathrm{aug}}} y(b) \geq \sum_{b \in B^{\mathrm{far}}_{\mathrm{aug}}} y(b) \geq \alpha \times |B^{\mathrm{far}}_{\mathrm{aug}}|. \tag{20}$$

Combining Equations (18) and (20),

$$|B^{\text{far}}_{\text{aug}}| \leq \frac{\sum_{b \in B_{\text{aug}}} y(b)}{\alpha} \leq \frac{w(M')}{\alpha} = O(n^{4/5}). \tag{21}$$

Combining with Equation (19),

$$|B^{\mathcal{C}}_{\square}| \leq |\mathcal{P}_{\text{aug}}| + |\mathcal{P}_{\text{alt}}| \leq |B^{\text{close}}_{\text{aug}}| + |B^{\text{far}}_{\text{aug}}| + |\mathcal{P}_{\text{alt}}| = O(n^{4/5}).$$

C Missing Proofs of Section 3

In this section, we provide the missing proofs of lemmas and claims made in Section 3.

Lemma C.1. *For any partitioning $\mathcal{C}$ of the root square $\square^*$ of $\mathcal{H}$, any feasible extended matching $M^{\mathcal{C}} = (M, B^{\mathcal{C}}), y(\cdot)$ on $\mathcal{G}_\sigma$, and any admissible alternating or augmenting path P, the matching obtained after updating $M^{\mathcal{C}}$ along P remains feasible.*

Proof. For any edge $(a, b) \in P$, due to the admissibility of the edge, $y(b) - y(a) = \mathrm{d}(a, b)$. If $(a, b) \notin M$ prior to augmentation, it is a matching edge after the augmentation and Condition (3) holds for (a, b). Otherwise, $(a, b) \in M$ prior to augmentation, it is a non-matching edge after the augmentation, and Condition (2) holds for (a, b). Note that all dual weights remain unchanged and consequently, Conditions (4)–(6) remain satisfied. □

C.1 Correctness of the Extended Hungarian Search Procedure

Lemma 3.1. *After the execution of the extended Hungarian search procedure for a cell $\square$, the extended matching $M^{\mathcal{C}}, y(\cdot)$ remains feasible, $y(v) \leq \varphi$ for all points $v \in A_{\square} \cup B_{\square}$, and $y(b_f) = \varphi$ for all free points $b_f \in B_{\square}$. Furthermore, the path P computed by the procedure is a minimum net-cost augmenting path inside $\square$. After augmenting along P, the updated key for $\square$ is the smallest net-cost of all augmenting paths inside $\square$ and is at least φ.*

Let $y(\cdot)$ (resp. $y'(\cdot)$) denote the dual weights of the points after (resp. before) the update duals step of the extended Hungarian search procedure. To prove this lemma, we first show that after the update duals step, the extended matching $M^{\mathcal{C}}, y(\cdot)$ is feasible, $y(v) \leq \varphi$ for all $v \in A_{\square} \cup B_{\square}$, and $y(b_f) = \varphi$ for all free points $b_f \in B_{\square}$. We then show that the augmenting path P computed by the procedure is admissible and conclude that P is a minimum net-cost augmenting path inside $\square$. We then use Lemma C.1 to show that after augmentation, the extended matching $M^{\mathcal{C}}, y(\cdot)$ remains feasible. Finally, we show that the updated key of $\square$ is the smallest net-cost of augmenting paths inside $\square$ and is at least φ.

Feasibility of points.

1. For any point $b \in B$:
   - if $b \in B \setminus B_{\square}$ is outside of $\square$, then $y(b) = y'(b)$; therefore, Conditions (4) and (5) remains satisfied.

- Otherwise, $b \in B_\square$ and by definition, $\kappa \leq \kappa_b + s(b)$. Therefore, Condition (4) holds for b since

$$y(b) = y'(b) + \kappa - \kappa_b \leq y'(b) + s(b) = \mathrm{d}(b, \square). \tag{22}$$

2. For any free point $a \in A_F$:
   - if $a \in A \setminus A_\square$ is a free point outside of $\square$, then $y(a) = y'(a) = 0$ and Condition (6) remains true.
   - Otherwise, $a \in A_\square$ and by definition, $\kappa_a \geq \kappa$; hence, the procedure does not update the dual weight of a, i.e., the dual weight of a remains 0, satisfying Condition (6).

Feasibility of edges. For any edge $(a, b) \in E$, let $s(a, b)$ denote the slack of (a, b) before updating the dual weights.

1. if $a \in A \setminus A_\square$ and $b \in B \setminus B_\square$, then $y(a) = y'(a)$ and $y(b) = y'(b)$ and Conditions (2) and (3) remains satisfied for (a, b).
2. if $a \in A_\square$ and $b \in B \setminus B_\square$, then $y(b) = y'(b)$ and $y(a) \geq y'(a)$, since the extended Hungarian search procedure does not decrease the dual weight of any point inside $\square$; hence, $y(b) - y(a) \leq y'(b) - y'(a) \leq \mathrm{d}(a, b)$ and Condition (2) is satisfied. (Note that by Lemma B.3, the edge (a, b) is a non-matching edge).
3. if $a \in A \setminus A_\square$ and $b \in B_\square$, then $y(b) \leq \mathrm{d}(b, \mathcal{C}) \leq \mathrm{d}(a, b)$; hence, $y(b) - y(a) \leq y(b) \leq \mathrm{d}(a, b)$ and Condition (2) is satisfied. (Note that by Lemma B.3, the edge (a, b) is a non-matching edge).
4. if $a \in A_\square$ and $b \in B_\square$:
   - If $(a, b) \in M$ is a matching edge, then $\kappa_b = \kappa_a$ since the only edge directed to b in the residual graph is the zero-slack edge (a, b). Thus, Condition (3) holds since

$$y(b) - y(a) = (y'(b) + \kappa - \kappa_b) - (y'(a) + \kappa - \kappa_a) = y'(b) - y'(a) = \mathrm{d}(a, b).$$

   - Otherwise, (a, b) is a non-matching edge and $\kappa_a \leq \kappa_b + s(b, a)$; therefore, Condition (2) holds since

$$y(b) - y(a) = (y'(b) + \kappa - \kappa_b) - (y'(a) + \kappa - \kappa_a) \leq y'(b) - y'(a) + s(b, a) = \mathrm{d}(a, b). \tag{23}$$

Maximum dual weight. Let $y_\square := \max_{b \in B_\square} y'(b)$. By invariant (I2) prior to the extended Hungarian search procedure, for all free points $b \in B_\square$, $y'(b) = y_\square$. By the construction of the residual graph, $\kappa_b = y'(b) = y_\square$ for all free points $b \in B_\square$ and $\kappa_b \geq y_\square$ for all points $b \in B_\square$. Therefore, for any free point $b \in B_\square$,

$$y(b) = y'(b) + \kappa - \kappa_b = \kappa = \varphi,$$

where the last equality holds since κ is the net-cost of the minimum net-cost path inside $\square$, which is φ. Furthermore, for any point $b' \in B_\square$,

$$y(b) = y'(b) + \kappa - \kappa_b \leq y'(b) + \kappa - y_\square \leq \kappa = \varphi.$$

Note that for any free point $a \in A_\square$, $y(a) = 0$ and for any matched point $a \in A_\square$, if a is matched to a point $b \in B_\square$, then by Condition (3), $y(a) = y(b) - \mathrm{d}(a, b) \leq y(b) \leq \varphi$. Hence, we conclude $y(v) \leq \varphi$ for all points $v \in A_\square \cup B_\square$ and $y(b_f) = \varphi$ for all free points $b_f \in B_\square$ after the dual updates.

Net-cost of P. To prove that P is a minimum net-cost augmenting path, we show that P is an admissible augmenting path. Consequently, if $b \in B_\square$ is the free endpoint of P, by Corollary B.2, $\phi(P) = y(b) = \varphi$. Note that for all other augmenting paths P' from a free point $b' \in B_\square$, from Lemmas B.1, $\phi(P') \geq y(b') = \varphi$, and therefore, P would be a minimum net-cost augmenting path inside $\square$.

For each edge $(u, v) \in P$, $\kappa_v = \kappa_u + s(u, v)$, since (u, v) is an edge of the shortest path tree of the residual graph. Plugging into Equation (23), for each non-matching edge (b, a), $y(b) - y(a) = \mathrm{d}(a, b)$ and the edge (b, a) is admissible. Furthermore, if $\kappa = \kappa_b + s(b)$ for a point $b \in B$, then by Equation (22), $y(b) = \mathrm{d}(b, \square)$ and the point b would be a zero-slack point. Therefore, the path P computed by the algorithm would be an admissible path from a free point $b \in B_\square$ to either a free point $a \in A_\square$ or a zero-slack point $b' \in B_\square$, i.e., P is an admissible augmenting path.

From Lemma C.1, the extended matching $M^{\mathcal{C}}, y(\cdot)$ obtained after augmenting the matching $M^{\mathcal{C}}$ along P remains feasible.

Updated key. Note that the extended matching $M^{\mathcal{C}}, y(\cdot)$ after the augmentation step is feasible, $y(v) \leq \varphi$ for all vertices $v \in A_\square \cup B_\square$, and $y(b_f) = \varphi$ for all free points $b_f \in B_\square$. Let κ_v, for each $v \in A_\square \cup B_\square$, denote the distances computed in the update key step of the extended Hungarian search procedure. By Lemma B.1, for any augmenting path P from a free point $b \in B_\square$ to a free point $a \in A_\square$, the net-cost of P is

$$\phi(P) = y(b) + \sum_{(a'', b'') \in P} s(a'', b'') \geq \kappa_a,$$

where the last inequality holds by the construction of the residual graph and the definition of κ_a. In this case, the equality happens when P is a shortest path from s to a. Similarly, for any augmenting path P from a free point $b \in B_\square$ to a point $b' \in B_\square$, the net-cost of P is

$$\phi(P) = y(b) + s(b') + \sum_{(a'', b'') \in P} s(a'', b'') \geq \kappa_{b'} + s(b'),$$

and we get an equality if P is a shortest path from the source vertex s to b'. Therefore, the updated key of $\square$, i.e., $\kappa_\square = \min\{\min_{a \in A_\square^F} \kappa_a, \min_{b \in B_\square} \kappa_b + s(b)\}$ correctly computes the net-cost of the minimum net-cost augmenting path inside $\square$. Finally, note that for any augmenting path P from a free point $b \in B_\square$, by Lemma B.1, $\phi(P) \geq y(b) = \varphi$ and the key of $\square$ would be at least φ.

C.2 Correctness of the Merge Procedure

Lemma 3.2. *After the execution of the merge procedure on a cell $\square$, the updated extended matching $M^{\mathcal{C}}, y(\cdot)$ is feasible, the dual weights $y(v)$ for every $v \in A_\square \cup B_\square$ is at most φ, and $y(b_f) = \varphi$ for all free points $b_f \in B_\square$. Furthermore, the updated key for $\square$ is the smallest net-cost of all augmenting paths within $\square$ and is at least φ.*

Let $\hat{M}^{\mathcal{C}} = (M, \hat{B}^{\mathcal{C}}), \hat{y}(\cdot)$ denote the feasible extended matching maintained by our algorithm before the execution of the merge procedure. Let $B_\square^{\mathcal{C}}$ denote the subset of points in $\hat{B}^{\mathcal{C}}$ that are matched to the divider $\Gamma_\square$ of $\square$, and let $M^{\mathcal{C}} = (M, B^{\mathcal{C}} = \hat{B}^{\mathcal{C}} \setminus B_\square^{\mathcal{C}})$ be the extended matching after adding the boundary-matched points in $B_\square^{\mathcal{C}}$ to the free points. By Lemma 2.6, the matching $M^{\mathcal{C}}, \hat{y}(\cdot)$ is feasible. Next, we show that, given a feasible extended matching $M^{\mathcal{C}}, y'(\cdot)$, after one iteration of the while-loop in the merge step, the matching remains feasible and the dual weights of points in $\square$ are at most φ.

Let $y(\cdot)$ (resp. $y'(\cdot)$) denote the dual weights of the points after (resp. before) the execution of one iteration of the merge step. We first show that $M^{\mathcal{C}}, y(\cdot)$ is a feasible extended matching where $y(v) \leq \varphi$ for all $v \in A_\square \cup B_\square$. We then show that the path P is admissible and use Lemma C.1 to show that after augmentation, the extended matching $M^{\mathcal{C}}, y(\cdot)$ remains feasible.

Feasibility of points.

1. For any point $b \in B$:
   - if $b \in B \setminus B_\square$ is outside of $\square$, then $y(b) = y'(b)$; therefore, Conditions (4) and (5) remains satisfied.
   - Otherwise, $b \in B_\square$ and $\kappa \leq \kappa_b + s(b)$. Therefore, Condition (4) holds for b since
   $$y(b) = y'(b) + \kappa - \kappa_b \leq y'(b) + s(b) = \mathrm{d}(b, \square). \tag{24}$$
2. For any free point $a \in A_F$:
   - if $a \in A \setminus A_\square$ is a free point outside of $\square$, then $y(a) = y'(a) = 0$ and Condition (6) remains true.
   - Otherwise, $a \in A_\square$ and $\kappa_a \geq \kappa$; therefore, the procedure does not update the dual weight of a, i.e., the dual weight of a remains 0, satisfying Condition (6).

Feasibility of edges. For any edge $(a, b) \in E$, let $s(a, b)$ denote the slack of (a, b) with respect to $y'(\cdot)$.

1. if $a \in A \setminus A_\square$ and $b \in B \setminus B_\square$, then $y(a) = y'(a)$ and $y(b) = y'(b)$ and therefore, Conditions (2) and (3) remains satisfied for (a, b).
2. if $a \in A_\square$ and $b \in B \setminus B_\square$, then $y(b) = y'(b)$ and $y(a) \geq y'(a)$, since the merge procedure does not decrease the dual weight of any point inside $\square$; hence, $y(b) - y(a) \leq y'(b) - y'(a) \leq \mathrm{d}(a, b)$ and Condition (2) is satisfied (Note that by Lemma B.3, the edge (a, b) is a non-matching edge).
3. if $a \in A \setminus A_\square$ and $b \in B_\square$, then $y(b) \leq \mathrm{d}(b, \mathcal{C}) \leq \mathrm{d}(a, b)$; hence, $y(b) - y(a) \leq y(b) \leq \mathrm{d}(a, b)$ and Conditions (2) is satisfied (Note that by Lemma B.3, the edge (a, b) is a non-matching edge).
4. if $a \in A_\square$ and $b \in B_\square$:
   - If $(a, b) \in M$ is a matching edge, then $\kappa_b = \kappa_a$ since the only edge directed to b in the residual graph is the zero-slack edge (a, b). Thus, Condition (3) holds since
   $$y(b) - y(a) = (y'(b) + \kappa - \kappa_b) - (y'(a) + \kappa - \kappa_a) = y'(b) - y'(a) = \mathrm{d}(a, b).$$
   - Otherwise, (a, b) is a non-matching edge and since there is a directed edge from b to a, $\kappa_a \leq \kappa_b + s(b, a)$, and Condition (2) holds since
   $$y(b) - y(a) = (y'(b) + \kappa - \kappa_b) - (y'(a) + \kappa - \kappa_a) \leq y'(b) - y'(a) + s(b, a) = \mathrm{d}(a, b). \tag{25}$$

Maximum dual weight. Next, we show that $y(v) \leq \varphi$ for all $v \in A_\square \cup B_\square$. Note that for any point $b \in B_\square$, by the definition of κ, we have $\kappa \leq \kappa_b + \varphi - y'(b)$. Therefore, if $\kappa_b < \kappa$, then $y(b) = y'(b) - \kappa_b + \kappa \leq \varphi$. Otherwise, $\kappa_b \geq \kappa$ and $y(b) = y'(b) \leq \varphi$. Furthermore, for all free points $a \in A_\square$, by Condition (6), $y(a) = 0$ and for all matched points $a \in A_\square$, if a is matched to a point $b \in B_\square$, then $y(a) = y(b) - \mathrm{d}(a, b) \leq y(b) \leq \varphi$.

Net-cost of P. Next, we show that the path P computed in an iteration of the merge step is either (i) an admissible augmenting path, or (ii) an admissible alternating path from a free point $b \in B_\square$ to a point $b' \in B_\square$ with $y(b') = \varphi$. Then, from Lemma C.1, the extended matching obtained after updating $M^{\mathcal{C}}, y(\cdot)$ along P is feasible.

For each non-matching edge $(b, a) \in P$, since (b, a) is in the Dijkstra's shortest path tree, $\kappa_a = \kappa_b + s(b, a)$. Plugging into Equation (25), for each non-matching edge (b, a), $y(b) - y(a) = \mathrm{d}(a, b)$, and the edge (b, a) is admissible (i.e., all edges of P are admissible).

- If P is a path that ends at a free point $a \in A_\square$ (κ is determined by the first term in the RHS of Equation (8)), then P is an admissible augmenting path,
- otherwise, if P is a path that ends at a point $b' \in B_\square$ with $\kappa = \kappa_{b'} + s(b')$ (κ is determined by the second term in the RHS of Equation (8)), then from Equation (24), $y(b') = y'(b') + s(b') = \mathrm{d}(b', \mathcal{C})$, and P is an admissible augmenting path, and
- otherwise, P is a path that ends at a point $b' \in B_\square$ with $\kappa = \kappa_{b'} + \varphi - y'(b')$, and P is an admissible alternating path.

Therefore, P is an admissible alternating or augmenting path and the matching obtained by updating $M^{\mathcal{C}}$ along P remains feasible (Lemma C.1).

Termination. Let P be a path from a free point $b \in B_\square$ to a point $u \in A^F_\square \cup B_\square$.

- If $u \in A^F_\square$, then P is an admissible augmenting path and P is in case (i).
- Otherwise, if $u \in B_\square$ and $\kappa = \kappa_u + s(u)$, then P is an admissible augmenting path and P is in case (ii).
- Otherwise, $u \in B_\square$ and $\kappa = \kappa_u + \varphi - y'(u)$. In this case, P is an admissible alternating path from b to a free point $b' \in B_\square$ with $y'(b') < \varphi$ and $y(b') = \varphi$.

In either case, the number of free points $b \in B_\square$ with $y(b) < \varphi$ reduces by one, and the merge step terminates.

Dual weight of free points. Note that the while-loop terminates when there are no free points $b_f \in B_\square$ with $y(b_f) < \varphi$, whereas, as discussed above, all points will have a dual weight at most φ, i.e., the dual weight of each free point $b_f \in B_\square$ after the termination of the while-loop is φ.

Updated key. Finally, we show that the updated key of $\square$ denotes the net-cost of the minimum net-cost augmenting path inside $\square$. Note that the extended matching $M^{\mathcal{C}}, y(\cdot)$ after the termination of the while-loop is feasible, $y(v) \leq \varphi$ for all vertices $v \in A_\square \cup B_\square$, and $y(b_f) = \varphi$ for all free points $b_f \in B_\square$. Let κ_v, for each $v \in A_\square \cup B_\square$, denote the distances computed in the update key step of the extended Hungarian search procedure. By Lemma B.1, for any augmenting path P from a free point $b \in B_\square$ to a free point $a \in A_\square$, the net-cost of P is

$$\phi(P) = y(b) + \sum_{(a'', b'') \in P} s(a'', b'') \geq \kappa_a,$$

where the last inequality holds by the construction of the residual graph and we get an equality of P is the shortest path from s to a. Similarly, for any augmenting path P from a free point $b \in B_\square$ to a point $b' \in B_\square$, the net-cost of P is

$$\phi(P) = y(b) + s(b') + \sum_{(a'',b'')\in P} s(a'', b'') \geq \kappa_{b'} + s(b'),$$

and we get an equality if P is a shortest path from s to b'. Therefore, the updated key of $\square$, i.e., $\kappa_\square = \min\{\min_{a\in A^F_\square} \kappa_a, \min_{b\in B_\square} \kappa_b + s(b)\}$ correctly computes the net-cost of the minimum net-cost augmenting path inside $\square$. Finally, note that for any augmenting path P from a free point $b \in B_\square$, by Lemma B.1, $\phi(P) \geq y(b) = \varphi$ and the key of $\square$ would be at least φ.

C.3 Runtime Analysis of the Merge Step

For any non-leaf cell $\square$ with $\square'$ and $\square''$ as children, the merge step at $\square$ first increases the dual weights of the free points inside $\square'$ and $\square''$ in $\tilde{O}(n_\square \Phi(n_\square))$ time (Lemma 2.5). For the feasible extended matching $M^{\mathcal{C}} = (M, B^{\mathcal{C}}), y(\cdot)$ after this initial step, let $B^{\mathcal{C}}_\square$ denote the subset of the boundary-matched points in $B^{\mathcal{C}}$ that are matched to $\Gamma_\square$. As discussed in Section C.2, each iteration of the while-loop in the merge step reduces the number of free points with a dual weight less than φ by one; therefore, the total number of executions of the while-loop in the merge step is at most $|B^{\mathcal{C}}_\square|$.

Lemma C.2. *For any cell $\square$, the number of iterations of the merge step on $\square$ is $O(|B^{\mathcal{C}}_\square|)$.*

Each iteration requires the computation of the distance κ_v for each point $v \in A_\square \cup B_\square$, which takes $\tilde{O}(n^2_\square)$ time. As shown in Section 3.3, the efficiency of this computation can be improved to $\tilde{O}(n_\square \Phi(n_\square))$ using a dynamic weighted nearest neighbor data structure with a query/update time of $\Phi(n_\square)$. Furthermore, by Lemma 2.6, $|B^{\mathcal{C}}_\square| = O(n^{4/5})$. Computing the updated key of $\square$ also requires the computation of the distance of each point from the source in the residual graph, which also takes $\tilde{O}(n_\square \Phi(n_\square))$ time. Therefore, the total execution time of the merge step would be $\tilde{O}(n^{4/5} n_\square \Phi(n_\square))$.

C.4 Runtime Analysis of the Extended Hungarian Search Procedure

Recall that the extended Hungarian search procedure iteratively picks the cell with the minimum key from PQ to be processed. For each leaf cell $\square$ in $\mathcal{H}$, since $\square$ contains the points corresponding to a single request and contains only one point of B, the procedure picks $\square$ at most once, at which it matches the point $b \in B_\square$ to the boundaries of $\square$. Therefore, the total time of the extended Hungarian search procedure on all leaf cells of $\mathcal{H}$ is $O(n)$.

Next, we show that for any non-leaf cell $\square$ of $\mathcal{H}$, our algorithm executes the search procedure on $\square$ at most $O(n^{4/5})$ times. Since each execution of the procedure takes $\tilde{O}(n_\square \Phi(n))$ time and each point participates in $O(\log(n\Delta))$ cells, the total running time of the extended Hungarian search procedure on all cells of $\mathcal{H}$ would be $O(n^{4/5} \sum_{\square\in\mathcal{H}} n_\square \Phi(n_\square)) = O(n^{9/5} \Phi(n_\square) \log(n\Delta))$, as claimed.

For any execution of the search procedure on $\square$, since $\square$ has the minimum key in PQ, the value φ represents the net-cost of the minimum net-cost augmenting path inside $\square$. Recall that for any non-leaf cell $\square$, we categorized the selections of $\square$ by the search procedure as low-net-cost if the value of φ in that iteration is at most $\ell_\square n^{-1/5}$ and high-net-cost otherwise. To bound the high-net-cost selections, we show that as soon as the value φ exceeds $\ell_\square n^{-1/5}$, the number of free points inside $\square$ cannot be more than $O(n^{4/5})$, and therefore, the number of high-net-cost selections of $\square$ is $O(n^{4/5})$.

Lemma 3.3. *Given a feasible extended matching $M^{\mathcal{C}}, y(\cdot)$ and any cell $\square \in \mathcal{C}$, if the net-cost of the minimum net-cost augmenting path inside $\square$ is greater than $\ell_{\square} n^{-1/5}$, then the number of free points of $M^{\mathcal{C}}$ is $O(n^{4/5})$.*

Proof. Let M' be the matching inside $\square$ as constructed in Lemma 2.2, and let M denote the matching of the extended matching $M^{\mathcal{C}} = (M, B^{\mathcal{C}})$. Define $B^{\mathcal{C}}_{\square}$ as the set of free points of $B_{\square}$ with respect to $M^{\mathcal{C}}, y(\cdot)$. Let $M_{\square}$ denote the subset of matching edges of M that lie inside $\square$, and let $\mathcal{P}_{\text{aug}}$ (resp. $\mathcal{P}_{\text{alt}}$) denote the set of (standard) augmenting (resp. alternating) paths in the symmetric difference $M_{\square} \oplus M'$ with one endpoint in the set $B^{\mathcal{C}}_{\square}$. Note that each path in $\mathcal{P}_{\text{alt}}$ has one endpoint that is free in M' and therefore, $|\mathcal{P}_{\text{alt}}| \leq |F(M')| = O(n^{4/5})$; here, $F(M')$ denotes the set of free points of M'. Next, we show that $|\mathcal{P}_{\text{aug}}| = O(n^{4/5})$.

From the definition of the net-cost of an augmenting path,

$$\sum_{P \in \mathcal{P}_{\text{aug}}} \phi(P) = \sum_{P \in \mathcal{P}_{\text{aug}}} \left(\sum_{(a,b) \in P \cap M'} \mathrm{d}(a,b) - \sum_{(a,b) \in P \cap M_{\square}} \mathrm{d}(a,b) \right)$$
$$\leq \sum_{P \in \mathcal{P}_{\text{aug}}} \left(\sum_{(a,b) \in P \cap M'} \mathrm{d}(a,b) \right) \leq w(M'). \tag{26}$$

For each path $P \in \mathcal{P}_{\text{aug}}$, let $b_P \in B_{\square}$ and $a_P \in A_{\square}$ denote the two end-points of P. Define $B_{\text{aug}} := \{b_P : P \in \mathcal{P}_{\text{aug}}\}$ as the set of free endpoints of the paths in $\mathcal{P}_{\text{aug}}$. Using Lemma B.1 and Equation (26),

$$\sum_{b \in B_{\text{aug}}} y(b) = \sum_{P \in \mathcal{P}_{\text{aug}}} y(b_P) \leq \sum_{P \in \mathcal{P}_{\text{aug}}} \phi(P) \leq w(M'). \tag{27}$$

From invariant (I2), the dual weight of all free points of $B_{\square}$ equals $\varphi > \ell_{\square} n^{-1/5}$. Therefore,

$$|B_{\text{aug}}| = \frac{\sum_{b \in B_{\text{aug}}} y(b)}{\varphi} \leq \frac{w(M')}{\ell_{\square} n^{-1/5}} = O(n_{\square}^{3/5} n^{1/5}) = O(n^{4/5}).$$

Combining the two bounds, the total number of free points with respect to $M^{\mathcal{C}}, y(\cdot)$ is $|\mathcal{P}_{\text{alt}}| + |\mathcal{P}_{\text{aug}}| = O(n^{4/5})$. □

Define $\mathcal{C}_{\square}$ as the set of all cells of $\mathcal{H}$, including $\square$ itself, that are processed by the merge step of our algorithm while $\square \in \mathcal{C}$ and $\varphi \leq \ell_{\square} n^{-1/5}$. For any cell $\square' \in \mathcal{C}_{\square}$, let $\mathcal{B}_{\square'}$ denote the set of points of B that are matched to the divider $\Gamma_{\square'}$ at the beginning of the merge step at $\square'$. During the execution of our algorithm, before processing $\square'$ by the merge step, there are k free points across all cells is $\mathcal{C}$. After combining the two children of $\square'$ (and removing the divider of $\square'$), the number of free points across all cells in the partitioning is now increased to $k + |\mathcal{B}_{\square'}|$. Each iteration of the merge step either (i) finds an admissible augmenting path, which reduces the number of free points by one, or (ii) finds an admissible alternating path to a point $b \in B_{\square'}$ with $y(b) = \varphi$, which does not change the number of free points. Therefore, after the merge step at $\square'$, the number of free points across all cells in the partitioning is at most $k + |\mathcal{B}_{\square'}|$. Our algorithm then iteratively executes the search procedure to reduce the number of free points across all cells to k. Hence, for each cell $\square' \in \mathcal{C}_{\square}$, the merge step at $\square'$ might lead to the execution of a low-net-cost extended Hungarian search procedure on $\square$ at most $|\mathcal{B}_{\square'}|$ times.

Lemma C.3. *For any cell $\square$, the number of low-net-cost executions of the extended Hungarian search procedure on $\square$ is at most $\sum_{\square' \in \mathcal{C}_{\square}} |\mathcal{B}_{\square'}|$.*

Recall that in a low-net-cost selection of $\square$, the value $\varphi \leq \ell_{\square} n^{-1/5}$. Using invariant (I2), the dual weight of all points of B is at most φ, and therefore, for any cell $\square' \in \mathcal{C}_{\square}$ and each point $b \in \mathcal{B}_{\square'}$, $y(b) \leq \ell_{\square} n^{-1/5}$, i.e., b is a boundary-matched point that is matched to the divider $\Gamma_{\square'}$ and has a dual weight at most $\ell_{\square} n^{-1/5}$. Therefore, $\mathrm{d}(b, \Gamma_{\square'}) = y(b) \leq \ell_{\square} n^{-1/5}$ and all points in $\mathcal{B}_{\square'}$ are at a distance at most $\ell_{\square} n^{-1/5}$ from the divider $\Gamma_{\square'}$. From Lemma 3.4, $\ell_{\square} \leq 3\ell_{\square'}$. Therefore, using Lemma 2.1, the number of points of $B_{\square'}$ at a distance at most $\lambda \ell_{\square'} \geq \ell_{\square} n^{-1/5}$ to the divider $\Gamma_{\square'}$ is $O(n_{\square'} n^{-1/5})$; hence, $|\mathcal{B}_{\square'}| = O(n_{\square'} n^{-1/5})$.

Lemma 3.4. *For any non-leaf cell $\square$ in $\mathcal{H}$ and any cell $\square' \in \mathcal{C}_{\square}$, $\frac{1}{3}\ell_{\square'} \leq \ell_{\square} \leq \frac{9}{4}\ell_{\square'}$.*

Proof. To prove this lemma, we first provide two useful relations between the sum of side-lengths $p_{\hat{\square}}$ and the largest side-length $\ell_{\hat{\square}}$ of any cell $\hat{\square}$ of $\mathcal{H}$. Let ℓ_x (resp. ℓ_y) denote the width (resp. height) of $\hat{\square}$. Since the aspect ratio of $\hat{\square}$ is at most 3, i.e., $\min\{\ell_x, \ell_y\} \geq \frac{1}{3}\ell_{\hat{\square}} = \frac{1}{3}\max\{\ell_x, \ell_y\}$,

$$\frac{4}{3}\ell_{\hat{\square}} \leq \ell_x + \ell_y = p_{\hat{\square}} = \ell_x + \ell_y \leq 2\ell_{\hat{\square}}.$$

Furthermore, for the smaller child $\hat{\square}_1$ of $\hat{\square}$, if the cell is divided on the x axis, then the width of $\hat{\square}_1$ is within $\frac{1}{3}\ell_{\hat{\square}}$ and $\frac{1}{2}\ell_{\hat{\square}}$; therefore,

$$\frac{4}{3}p_{\hat{\square}_1} \leq p_{\hat{\square}} \leq 2p_{\hat{\square}_1}.$$

For the cell $\square$ (resp. $\square'$), suppose $\square_{\min}$ (resp. $\square'_{\min}$) denote the smaller child of $\square$ (resp. $\square'$). Since our algorithm picked $\square_{\min}$ for being merged as the smallest cell before $\square'_{\min}$,

$$\frac{4}{3}\ell_{\square} \leq p_{\square} \leq 2p_{\square_{\min}} \leq 2p_{\square'_{\min}} \leq \frac{3}{2}p_{\square'} \leq 3\ell_{\square'}. \tag{28}$$

Similarly, since our algorithm picked $\square'_{\min}$ as the smallest cell to be processed rather than $\square$,

$$\frac{4}{3}\ell_{\square'} \leq p_{\square'} \leq 2p_{\square'_{\min}} \leq 2p_{\square} \leq 4\ell_{\square}. \tag{29}$$

Combining Equations (28) and (29),

$$\frac{1}{3}\ell_{\square'} \leq \ell_{\square} \leq \frac{9}{4}\ell_{\square'}.$$

□

We next show that $\sum_{\square' \in \mathcal{C}_{\square}} n_{\square'} = O(n)$ and conclude that $\sum_{\square' \in \mathcal{C}_{\square}} |\mathcal{B}_{\square'}| = O(n^{4/5})$, which bounds the number of low-net-cost executions of the search procedure on $\square$ by $O(n^{4/5})$, as desired. By Lemma 3.4, for any cell $\square' \in \mathcal{C}_{\square}$, $\ell_{\square'} \in [\frac{1}{3}\ell_{\square}, \frac{9}{4}\ell_{\square}]$. By the construction of $\mathcal{H}$, for any cell $\square'$ and its grandparent $\square'_g$, $\ell_{\square'} \leq \frac{2}{3}\ell_{\square'_g}$. Therefore, for $\square' \in \mathcal{C}_{\square}$, only the ancestor of $\square'$ up to $2\log_{3/2}\frac{27}{4}$ levels higher can also be in $\mathcal{C}_{\square}$; therefore, for any point $u \in A \cup B$, the point u lies inside at most $O(1)$ cells of $\mathcal{C}_{\square}$, and $\sum_{\square' \in \mathcal{C}_{\square}} n_{\square'} = O(n)$, as claimed.

We conclude that the total number of executions of the extended Hungarian search for each cell $\square$ of $\mathcal{H}$ is $O(n^{4/5})$. The next lemma follows since each execution takes $\tilde{O}(n_{\square}\Phi(n))$ time.

Lemma C.4. *For any cell $\square$ of $\mathcal{H}$, the total execution time of the extended Hungarian search procedure on $\square$ takes $\tilde{O}(n^{4/5} n_{\square}\Phi(n))$ time.*

C.5 Extension to higher dimensions

Given a set of requests σ in the d dimensions, for any $d \geq 2$, let $\mathcal{G}_\sigma$ on point sets A and B denote the graph constructed for the k-SP problem on σ under any ℓ_p norm for some $p \geq 1$. Let $\mathcal{H}_d$ denote the hierarchical partitioning constructed for the point set $A \cup B$ with a parameter $\lambda = 9n^{-\frac{1}{2d+1}}$. The hierarchical partitioning has a height $O(d \log(n\Delta))$. Using $\mathcal{H}_d$, we run our algorithm as described in Section 3.

We summarize the efficiency analysis of our algorithm for d-dimensional point sets next and show that our algorithm runs in $\tilde{O}(n^{2-\frac{1}{2d+1}}\Phi(n)\log\Delta)$ time. In particular, we show that for any cell $\square$, the number of iterations of the merge step on $\square$ is $O(n^{1-\frac{1}{2d+1}})$, where each iteration takes $\tilde{O}(n_\square \Phi(n))$ time. We also show that our algorithm runs the extended Hungarian search procedure in $O(n^{1-\frac{1}{2d+1}})$ time on $\square$, each in $\tilde{O}(n_\square\Phi(n))$ time. Adding these bounds for all cells of $\mathcal{H}_d$, the total running time of our algorithm is $\tilde{O}(n^{2-\frac{1}{2d+1}}\Phi(n)\log\Delta)$. We summarize the details below.

Given a feasible extended matching $M^{\mathcal{C}}, y(\cdot)$, for any cell $\square \in \mathcal{C}$, let $B^{\mathcal{C}}_\square$ denote the set of all points of $B_\square$ that are matched to the divider $\Gamma_\square$ in the matching before the execution of the merge step at $\square$. By Lemma C.2, the number of iterations of the merge step on $\square$ is $O(|B^{\mathcal{C}}_\square|)$. To bound the number of points in $B^{\mathcal{C}}_\square$, we first show that there exists a partial matching M' of high cardinality and low cost.

Lemma C.5. *For any cell $\square$ of $\mathcal{H}_d$, there exists a matching M' over $\mathcal{G}_\sigma$ inside $\square$ that matches all except $O(n_\square^{1-\frac{1}{2d+1}})$ points of $B_\square$ and has a cost $O(\ell_\square n_\square^{1-\frac{2}{2d+1}})$.*

Proof. Similar to our construction for Lemma 2.2, to construct the matching M' for Lemma C.5, we place a grid of cell-side-length $\ell_\square n_\square^{-\frac{2}{2d+1}}$ and compute the matching corresponding to the 1-partitioning of the requests inside each cell of this grid. It is easy to confirm that the matching M' achieves the bounds claimed in Lemma C.5. $\square$

For the feasible extended matching $M^{\mathcal{C}} = (M, B^{\mathcal{C}}), y(\cdot)$, let $M_\square$ denote the subset of matching edges of M that lie inside $\square$. Let $\mathcal{P}_{\text{aug}}$ (resp. $\mathcal{P}_{\text{alt}}$) denote the set of augmenting paths (resp. alternating paths) with an endpoint in $|B^{\mathcal{C}}_\square|$ in the symmetric difference $M_\square \oplus M'$. As discussed above, $|\mathcal{P}_{\text{alt}}| \leq |F(M')| = O(n^{1-\frac{1}{2d+1}})$, where $F(M')$ denotes the set of free points of $B_\square$ with respect to M'. We partition the free endpoints of $\mathcal{P}_{\text{aug}}$ into the set $B^{\text{close}}_{\text{aug}}$ (resp. $B^{\text{far}}_{\text{aug}}$) that are at a distance closer than (resp. further than) $\lambda'_\square = \ell_\square n^{-\frac{1}{2d+1}}$ to the divider $\Gamma_\square$ of $\square$. From Lemma 2.1, $|B^{\text{close}}_{\text{aug}}| = O(n^{1-\frac{1}{2d+1}})$. Finally, by Equation (18), $\sum_{b \in B^{\text{far}}_{\text{aug}}} y(b) \leq w(M')$. Since the dual weight of each free point in $B^{\text{far}}_{\text{aug}}$ is at least $\lambda'_\square$, we get a bound of $O(n^{1-\frac{1}{2d+1}})$ on the number of such points, and a total execution time of $\tilde{O}(n^{1-\frac{1}{2d+1}} n_\square \Phi(n))$ for the merge step on a cell $\square$ of $\mathcal{H}$.

Next, we bound the number of executions of the extended Hungarian search procedure for each cell $\square$. A selection of $\square$ by the extended Hungarian search procedure is low-net-cost if $\varphi \leq \ell_\square n^{-\frac{1}{2d+1}}$ and high-net-cost otherwise. For the high-net-cost selections, one can use the matching M' from Lemma C.5 and a similar argument as in Lemma 3.3 to show that when $\varphi > \ell_\square n^{-\frac{1}{2d+1}}$, the total number of free points inside $\square$ cannot exceed $O(n^{1-\frac{1}{2d+1}})$ and conclude an upper bound of $O(n^{1-\frac{1}{2d+1}})$ on the number of high-net-cost selections of $\square$. To bound the number of low-net-cost selections of $\square$, define $\mathcal{C}_\square$ as the set of all cells $\square' \in \mathcal{H}$ that are processed by the merge step while $\square$ is in $\mathcal{C}$ and $\varphi \leq \ell_\square n^{-\frac{1}{2d+1}}$. For any cell $\square' \in \mathcal{C}_\square$, let $\mathcal{B}_{\square'}$ denote the set of boundary-matched points of $B_{\square'}$ matched to the divider of $\square'$ before the execution of the merge step on $\square'$. By Lemma C.3, the number of low-net-cost selections of $\square$ is at most

$\sum_{\square' \in \mathcal{C}_\square} |\mathcal{B}_{\square'}|$. Since $\varphi \leq \ell_\square n^{-\frac{1}{2d+1}}$, for each cell $\square' \in \mathcal{C}_\square$, all points in $\mathcal{B}_{\square'}$ are at a distance at most $\ell_\square n^{-\frac{1}{2d+1}}$ to the divider of $\square'$. Therefore, from Lemma 2.1 and 3.4 and using an identical discussion as above, $\sum_{\square' \in \mathcal{C}_\square} \mathcal{B}_{\square'} = O(dn^{1-\frac{1}{2d+1}})$. Therefore, the total execution time of the extended Hungarian search step on any cell $\square$ of $\mathcal{H}$ is $\tilde{O}(dn^{1-\frac{1}{2d+1}} n_\square \Phi(n))$.

Combining the total execution times of the merge step and the extended Hungarian search step, the running time of our algorithm would be $\tilde{O}(\sum_{\square \in \mathcal{H}} dn^{1-\frac{1}{2d+1}} n_\square \Phi(n)) = \tilde{O}(dn^{2-\frac{1}{2d+1}} \Phi(n) \log \Delta)$, leading to the following theorem.

Theorem C.6. *Given any sequence σ (resp. $\sigma' = \eta\sigma$) of n requests (resp. n requests and k initial server locations) in d dimensions with a spread of Δ, and a value $1 \leq k \leq n$, there exists a deterministic algorithm that computes the optimal solution for the instance of k-SP (resp. k-SPI) problem under the ℓ_p norm in $\tilde{O}(\min\{nk, n^{2-\frac{1}{2d+1}} \log \Delta\} \cdot \Phi(n))$ time.*

D Missing Proofs of Section 4

Lemma 4.1. *For any $t > 0$ and any dual-optimal t-matching $M, y(\cdot)$ on a graph $\mathcal{G}_\sigma$, the matching M is a minimum-cost t-matching on $\mathcal{G}_\sigma$.*

Proof. Let A_F (resp. B_F) denote the set of free points of A (resp. B). Let $y_{\max} := \max_{b \in B} y(b)$. Using Conditions (10) and (11), we rewrite the cost of the matching M as

$$\begin{aligned} w(M) &= \sum_{(a,b) \in M} \mathrm{d}(a,b) = \sum_{(a,b) \in M} y(b) - y(a) \\ &= \left(\sum_{b \in B} y(b) - \sum_{a \in A} y(a) \right) - \sum_{b \in B_F} y(b) + \sum_{a \in A_F} y(a) \\ &= \left(\sum_{b \in B} y(b) - \sum_{a \in A} y(a) \right) - |B_F| \cdot y_{\max}. \end{aligned} \tag{30}$$

Let M^* denote any minimum-cost t-matching on $\mathcal{G}_\sigma$. Let A^*_F (resp. B^*_F) denote the set of points of A (resp. B) that are free in M^*. Since both M and M^* are t-matchings, $|B_F| = |B^*_F|$. Using Condition (9),

$$\begin{aligned} w(M^*) &= \sum_{(a,b) \in M^*} \mathrm{d}(a,b) \geq \sum_{(a,b) \in M^*} y(b) - y(a) \\ &= \left(\sum_{b \in B} y(b) - \sum_{a \in A} y(a) \right) - \sum_{b \in B^*_F} y(b) + \sum_{a \in A^*_F} y(a) \\ &\geq \left(\sum_{b \in B} y(b) - \sum_{a \in A} y(a) \right) - |B^*_F| \cdot y_{\max}, \end{aligned} \tag{31}$$

where the last inequality holds since $y(b) \leq y_{\max}$ for each point $b \in B$ and $y(a) \geq 0$ for each point $a \in A$. Combining Equations (30) and (31),

$$w(M) = \sum_{b \in B} y(b) - \sum_{a \in A} y(a) - |B_F| \cdot y_{\max} \leq w(M^*).$$

Since M^* is a minimum-cost t-matching, $w(M) = w(M^*)$, and M is also a minimum-cost t-matching. $\square$

Lemma 4.2. *For any $1 \le t \le k$, the matching $M, y(\cdot)$ maintained by our algorithm after $(t-1)$ iterations is a dual-optimal $(n-t)$-matching on $\mathcal{G}_\sigma$.*

Proof. For any $1 \le t \le k$, let $M_t, y_t(\cdot)$ denote the $(n-t)$-matching computed by our algorithm, i.e., the matching $M_t, y_t(\cdot)$ is computed from $M_{t-1}, y_{t-1}(\cdot)$ by executing the REVERSEHUNGARIANSEARCH procedure and reducing M_{t-1} along the path P returned by the REVERSEHUNGARIANSEARCH procedure. By our initial dual assignments, $M_1, y_1(\cdot)$ is a dual-optimal $(n-1)$-matching. Below, assuming that $M_{t-1}, y_{t-1}(\cdot)$ is dual-optimal, we show that $M_t, y_t(\cdot)$ is also dual-optimal. To do so, first, we show that $M_{t-1}, y_t(\cdot)$ is dual-optimal and the path P is admissible with respect to the updated dual weights $y_t(\cdot)$. We then conclude Lemma 4.2 by showing that the matching M_t obtained by reducing M_{t-1} along P remains dual-optimal along with $y_t(\cdot)$.

For any edge $(a,b) \in E$, let $s_{t-1}(a,b)$ denote the slack of (a,b) with respect to $y_{t-1}(\cdot)$. For any matching edge $(a,b) \in M_{t-1}$, $\kappa_b = \kappa_a$ since the only edge directed to a in the reversed residual graph is the zero-slack edge (a,b). Thus, Condition (10) holds since

$$y_t(b) - y_t(a) = (y_{t-1}(b) - \kappa + \kappa_b) - (y_{t-1}(a) - \kappa + \kappa_a) = y_{t-1}(b) - y_{t-1}(a) = \mathrm{d}(a,b).$$

Similarly, for any non-matching edge $(b,a) \in E$, $\kappa_b \le \kappa_a + s_{t-1}(b,a)$, and Condition (9) holds since

$$y_t(b) - y_t(a) = (y_{t-1}(b) - \kappa + \kappa_b) - (y_{t-1}(a) - \kappa + \kappa_a) \le y_{t-1}(b) - y_{t-1}(a) + s_{t-1}(a,b) = \mathrm{d}(a,b),$$

and the equality holds for the edges on Dijkstra's shortest path tree; consequently, the path P is admissible. Finally, we show that the free points of B have the highest dual weight among all points. Let $y_{\max} := \max_{b' \in B} y_{t-1}(b')$. From the fact that $M_{t-1}, y_{t-1}(\cdot)$ is dual-optimal, for any free point $b \in B$, $y_{t-1}(b) = y_{\max}$; therefore, $\kappa_b = 0$ since there is a zero-cost edge from source to b. Thus,

$$y_t(b) = y_{t-1}(b) - \kappa + \kappa_b = y_{\max} - \kappa. \tag{32}$$

As a result, all free points of B have the same dual weight of $y_{\max} - \kappa$ in $y_t(\cdot)$. For any matched point $b \in B$, if $y_{t-1}(b) \ge y_{\max} - \kappa$, then the edge from the source to b has a cost $y_{\max} - y_{t-1}(b) < \kappa$, and therefore, $\kappa_b \le y_{\max} - y_{t-1}(b) \le \kappa$; hence,

$$y_t(b) = y_{t-1}(b) - \kappa + \kappa_b \le y_{t-1}(b) - \kappa + y_{\max} - y_{t-1}(b) = y_{\max} - \kappa.$$

Finally, note that there are no paths in the reversed residual graph from the source to the free points $a \in A$ with respect to M_{t-1}, and therefore, $y_t(a) = y_{t-1}(a) = 0$. Additionally, for the path P returned by the procedure, if P ends at a point $a \in A$, then $\kappa = \kappa_a + y_{t-1}(a)$ and $y_t(a) = y_{t-1}(a) - \kappa + \kappa_a = 0$. Hence, $M_t, y_t(\cdot)$ is a dual-optimal matching. □

E Bipartite Matching on Randomly Colored Points

E.1 Missing Proofs and Details of Section 5

Constructing the Partial Matching. In this part, for any cell $\square$ of $\mathcal{H}$, we construct a matching M' of $A_\square \cup B_\square$ that, in expectation, matches all except $\tilde{O}(n_\square^{3/4})$ points of $B_\square$ and has a cost $\tilde{O}(\ell_\square n_\square^{-1/4})$, proving Lemma 5.3. We begin by introducing a set of notations. Let $\square$ be any cell of $\mathcal{H}$, and let G be a grid dividing $\square$ into smaller squares. For any square $\xi \in G$, let $A_\square^\xi$ (resp. $B_\square^\xi$) denote the subset of points of $A_\square$ (resp.

$B_\square$) that lie inside ξ. Define the *excess* of ξ as $\text{exc}(\xi) := \big||B_\square^\xi| - |A_\square^\xi|\big|$. Define $\text{exc}(G) := \sum_{\xi \in \mathcal{G}} \text{exc}(\xi)$ as the excess of G. We next show an important property of randomly colored point sets, which is critical in constructing the matching.

Lemma E.1. *For any square $\square$ of $\mathcal{H}$ and a grid G inside $\square$ with cell side length $O(\ell_\square n_\square^{-\alpha})$, $\mathbb{E}[\text{exc}(G)] = \tilde{O}(n_\square^{\alpha+\frac{1}{2}})$.*

Proof. If $\alpha \geq \frac{1}{2}$, the lemma statement holds trivially since $n_\square^{\alpha+\frac{1}{2}} \geq n_\square$. Therefore, we assume $\alpha \leq \frac{1}{2}$. Using the Hoeffding's inequality [10], for any hypercube ξ of G,

$$\Pr\left[\big||B_\square^\xi| - |A_\square^\xi|\big| \geq c_1\sqrt{n_\square logn}\right] \leq n^{-c_2}$$

for some constants $c_1, c_2 > 1$. Since $\text{exc}(\xi) \leq n_\square$,

$$\mathbb{E}[\text{exc}(\xi)] = O(\sqrt{n_\square \log n}).$$

Summing over all squares of G,

$$\mathbb{E}[\text{exc}(G)] = \sum_{\xi \in G} \mathbb{E}[\text{exc}(\xi)] = O\left(\sum_{\xi \in G} \sqrt{n_\square \log n}\right) = O\left(\sqrt{n \log n \times |G|}\right) = \tilde{O}(n^{\alpha+\frac{1}{2}}).$$

□

We next use Lemma E.1 to show that there exists a low-cost high-cardinality matching inside each sub-problem.

Lemma 5.3. *For any cell $\square$ of $\mathcal{H}$, there exists a matching M' that, in expectation, matches all except $O(n_\square^{3/4})$ points of $B_\square$ and has a cost $O(\ell_\square^2 n_\square^{1/4})$.*

Proof. Let $\langle G_1, \ldots, G_t\rangle$ denote a sequence of grids, where $t = \lceil \log\log n \rceil$ and each grid G_i has a side-length $O(\ell_\square n_\square^{-\alpha_i})$ for

$$\alpha_i := \frac{1}{2} - \frac{2^t}{2^{t+2}-3}\left(1 - \frac{1}{2^i}\right).$$

Using these grids, we construct a matching M' as follows. Let $A_\square^0 := A_\square$ and $B_\square^0 := B_\square$. Starting from $i = 1$, we compute a matching M_i from $B_\square^{i-1}$ to $A_\square^{i-1}$ that for each square of G_i, matches as many points as possible inside the square arbitrarily. Define $A_\square^i$ (resp. $B_\square^i$) as the set of free points of $A_\square^{i-1}$ (resp. $B_\square^{i-1}$) and process the next grid G_{i+1}. We continue this procedure until the last grid G_t is processed. Define $M' := \sum_{i=1}^t M_i$. This completes the construction of M'. In the following, we first show that the total number of free points with respect to M' is $O(n_\square^{3/4})$; we then show that $w(M') = O(\ell_\square^2 n_\square^{1/4})$ and conclude the lemma statement.

The matching M matches as many points as possible inside each square of the grid G_t. Therefore, the total number of unmatched points is equal to the excess of G_t, which by Lemma E.1 is

$$\mathbb{E}[\text{exc}(G_t)] = \tilde{O}(n_\square^{\alpha_t+\frac{1}{2}}) = O(n_\square^{\frac{3}{4}+\frac{1}{4(2^{t+2}-3)}}) = O(n_\square^{\frac{3}{4}+\frac{1}{16\log n-12}}) = O(n_\square^{\frac{3}{4}}). \tag{33}$$

We next analyze the expected cost of M'. By the linearity of expectation,

$$\mathbb{E}[w(M')] = \sum_{i=1}^{t} \mathbb{E}[w(M_i)]. \tag{34}$$

For $i = 1$, since all matching edges in M_1 have a squared Euclidean cost at most $O((\ell_\square n_\square^{-\alpha_1})^2)$, the cost of M_1 would be

$$w(M_1) = O(n_\square \times (\ell_\square n_\square^{-\alpha_1})^2) = O(\ell_\square^2 n_\square^{\frac{2^t}{2^{t+2}-3}}) = O(\ell_\square^2 n_\square^{\frac{1}{4}}). \tag{35}$$

Finally, for each $1 < i \le t$, the matching M_i consists of matching edges with squared Euclidean cost of at most $O((\ell_\square n_\square^{-\alpha_i})^2)$. By Lemma E.1, the expected number of matching edges in M_i is at most $\tilde{O}(n_\square^{\alpha_{i-1}+\frac{1}{2}})$; therefore,

$$\mathbb{E}[w(M_i)] = \tilde{O}(n_\square^{\alpha_{i-1}+\frac{1}{2}} \times (\ell_\square n_\square^{-\alpha_i})^2) = \tilde{O}(\ell_\square^2 n_\square^{\frac{2^t}{2^{t+2}-3}}) = \tilde{O}(\ell_\square^2 n_\square^{\frac{1}{4}}). \tag{36}$$

Combining Equations (34), (35), and (36), $\mathbb{E}[w(M)] = \tilde{O}(\ell_\square^2 n_\square^{\frac{1}{4}})$. □

Bounding the number of iterations. We next show that the number of iterations of the merge step and the extended Hungarian search procedure on any cell $\square$ of $\mathcal{H}$ is bounded by $O(n^{3/4})$.

Lemma 5.4. *For any cell $\square$ of $\mathcal{H}$, the total number of iterations of the merge step and the total number of executions of the search procedure on $\square$ is $O(|B_\square^{\mathcal{C}}|)$.*

Proof. From Lemma C.2, the total number of iterations of the merge step on $\square$ is $O(|B_\square^{\mathcal{C}}|)$. Next, note that the parameter k is set to 0; therefore, there are no free points inside $\square$ when the merge step is executed on $\square$. When erasing the divider of $\square$, our algorithm creates $|B_\square^{\mathcal{C}}|$ free points. Each iteration of the merge step either (i) finds an augmenting path, which reduces the number of free points by one, or (ii) finds an alternating path to a matched point $b \in B_\square$ with $y(b) = \varphi$, which does not change the number of free points inside $\square$. Therefore, after the merge step on $\square$, the number of free points remaining inside $\square$ is at most $|B_\square^{\mathcal{C}}|$, and our algorithm executes one extended Hungarian search procedure for each remaining free point. □

Lemma 5.5. *For any cell $\square$ of $\mathcal{H}$, $|B_\square^{\mathcal{C}}| = O(n^{3/4})$.*

Proof. Let M' denote the matching constructed in Lemma 5.3, and let M denote the matching of the extended matching $M^{\mathcal{C}}$ maintained by our algorithm. Let $M_\square$ denote the matching edges of M that lie inside $\square$. Note that by lemma B.3, no matching edges can cross the boundaries of $\mathcal{C}$. Let $\mathcal{P}_{\text{aug}}$ (resp. $\mathcal{P}_{\text{alt}}$) denote the set of augmenting paths (resp. alternating paths) with an endpoint in $B_\square^{\mathcal{C}}$ in the symmetric difference $M_\square \oplus M'$. Clearly, $|B_\square^{\mathcal{C}}| = |\mathcal{P}_{\text{alt}}| + |\mathcal{P}_{\text{aug}}|$. For the alternating paths, $|\mathcal{P}_{\text{alt}}| = O(n^{3/4})$ since each alternating path has one free endpoint in M'. Next, we show that $|\mathcal{P}_{\text{aug}}| = O(n^{3/4})$.

As shown in Equation (18),

$$\sum_{b \in B_{\text{aug}}} y(b) = \sum_{P \in \mathcal{P}_{\text{aug}}} y(b_P) \le \sum_{P \in \mathcal{P}_{\text{aug}}} \phi(P) \le w(M'). \tag{37}$$

Define $\alpha := \ell_\square n^{-1/4}$. Each free point $b \in B_{\text{aug}}$ is called a *close* (resp. *far*) point if the Euclidean distance of b to the divider of $\square$ is at most (resp. more than) α. Let $B_{\text{aug}}^{\text{close}}$ (resp. $B_{\text{aug}}^{\text{far}}$) denote the set of all close (resp. far) points of B_{aug}. By Lemma 2.1,

$$|B_{\text{aug}}^{\text{close}}| = O(n_\square n^{-1/4}) = O(n^{3/4}). \tag{38}$$

For each far point $b \in B_{\text{aug}}^{\text{far}}$, $y(b) = \mathrm{d}(b, \Gamma_\square) \geq \alpha^2$ since b is matched to the divider $\Gamma_\square$ (note that the distance function $\mathrm{d}(\cdot,\cdot)$ is the squared Euclidean distance). Therefore,

$$\sum_{b \in B_{\text{aug}}} y(b) \geq \sum_{b \in B_{\text{aug}}^{\text{far}}} y(b) \geq \alpha^2 \times |B_{\text{aug}}^{\text{far}}|. \tag{39}$$

Combining Equations (37) and (39),

$$|B_{\text{aug}}^{\text{far}}| \leq \frac{\sum_{b \in B_{\text{aug}}} y(b)}{\alpha^2} \leq \frac{w(M')}{\alpha^2} = O(n^{3/4}). \tag{40}$$

Combining with Equation (38),

$$|B_\square^{\mathcal{C}}| \leq |B_{\text{aug}}| + |B_{\text{alt}}| \leq |B_{\text{aug}}^{\text{close}}| + |B_{\text{aug}}^{\text{far}}| + |B_{\text{alt}}| = \tilde{O}(n^{3/4}).$$

□

E.2 Analysis for General d and q

For a point set U of $2n$ points in d dimensions and any $q \geq 1$, let A denote a random subset of n points of U and let $B = U \setminus A$. To compute a minimum-cost perfect matching between A and B under ℓ_2^q distances, we construct our hierarchical partitioning $\mathcal{H}$ with a parameter $\lambda = 9n^{-\frac{1}{d+2}}$ and execute our algorithm from Section 3 by setting $k = 0$. In the following, we extend our analysis from Section 5 to any dimension $d \geq 2$ and any $q \geq 1$ and show the following result.

Lemma E.2. *Suppose U is a set of $2n$ points inside the unit d-dimensional hypercube, $d \geq 2$, and A is a subset chosen uniformly at random from all subsets of size n. Let $B = U \setminus A$. Then, for any parameters $q \geq 1$, the expected running time of our algorithm for computing the minimum-cost matching on the complete bipartite graph on A and B under ℓ_2^q costs is*

$$\begin{cases} \tilde{O}(n^{2-\frac{q}{(q+1)d}}\Phi(n)\log\Delta), & q \leq \frac{d}{2}, \\ \tilde{O}(n^{2-\frac{1}{d+2}}\Phi(n)\log\Delta), & q > \frac{d}{2}. \end{cases}$$

Note that when $q \leq \frac{d}{2}$, the bound claimed in Lemma E.2 is identical to the runtime bound proven in [9, Theorem B.2] and their analysis directly applies to our algorithm; hence, in the following, we analyze the running time of our algorithm assuming $q > \frac{d}{2}$.

We begin by showing that there exists a low-cost high-cardinality partial matching.

Lemma E.3. *For any cell $\square$ of $\mathcal{H}$, there exists a matching M' that, in expectation, matches all except $O(n_\square^{1-\frac{1}{d+2}})$ points of $B_\square$ and has a cost $O((2\ell_\square)^q n_\square^{1-\frac{q+1}{d+2}})$.*

Proof. Define $r := \frac{2q}{d} > 1$ and $t := \lceil \log_r \log_2 n \rceil$. Let $\beta := q(d+2)r^t - d(q+1)$. Define a sequence of grids $\langle G_1, \ldots, G_t \rangle$, where each grid G_i has a side-length $O(\ell_\square n^{-\alpha_i})$ for

$$\alpha_i := \frac{1}{d} - \frac{r^{t+1}}{\beta}\left(1 - \frac{1}{r^i}\right).$$

We construct a matching M as follows. Let $A_\square^0 := A_\square$ and $B_\square^0 := B_\square$. Starting from $i = 1$, let M_i be a matching from $B_\square^{i-1}$ to $A_\square^{i-1}$ that matches as many points as possible inside each hypercube of G_i. Let

$A^i_\square$ and $B^i_\square$ denote the set of free points of $A^{i-1}_\square$ and $B^{i-1}_\square$, respectively. Set $i \leftarrow i+1$ and continue this procedure until the last grid G_t is processed. Define $M := \sum_{i=1}^{t} M_i$. In the following, we show that the total number of free points with respect to M is $\tilde{O}(n_\square^{1-\frac{1}{d+2}})$ and the cost of M is $\mathbb{E}[w(M)] = O((2\ell_\square)^q n_\square^{1-\frac{q+1}{d+2}})$.

The matching M matches as many points as possible inside each square of the grid G_t. Therefore, the total number of unmatched points is equal to the excess of G_t, which by Lemma E.4 below is

$$\mathbb{E}[\mathrm{exc}(G_t)] = \tilde{O}(n_\square^{\frac{d}{2}\alpha_t+\frac{1}{2}}) = \tilde{O}(n_\square^{1-\frac{1}{d+2}}), \tag{41}$$

where the second equality holds since

$$\begin{aligned}\frac{d}{2}\alpha_t + \frac{1}{2} &= \frac{d}{2}\left(\frac{1}{d} - \frac{r^{t+1}}{\beta}(1-\frac{1}{r^t})\right) + \frac{1}{2} = \left(\frac{1}{2} - \frac{qr^t - q}{q(d+2)r^t - d(q+1)}\right) + \frac{1}{2} \\ &= 1 - \frac{1}{d+2} + \frac{q - \frac{d(q+1)}{d+2}}{q(d+2)r^t - d(q+1)} \le 1 - \frac{1}{d+2} + \frac{1}{2\log n}.\end{aligned}$$

We next analyze the expected cost of M. For $i=1$, since all matching edges in M_1 has a cost of at most $O((\ell_\square n_\square^{-\alpha_1})^q)$, the cost of M_1 would be

$$w(M_1) = O(n_\square \times (\ell_\square n_\square^{-\alpha_1})^q) = O((2\ell_\square)^q n_\square^{1-\frac{q+1}{d+2}}), \tag{42}$$

where the second equality holds since

$$\begin{aligned}1 - q\alpha_1 &= 1 - \frac{q}{d} + \frac{q(r-1)r^t}{\beta} = 1 - \frac{q}{d} + \frac{\frac{2q^2-qd}{d}r^t}{q(d+2)r^t - d(q+1)} \\ &= 1 - \frac{q}{d} + \frac{2q-d}{d(d+2)} + \frac{\frac{(2q-d)(q+1)}{d+2}}{q(d+2)r^t - d(q+1)} \le 1 - \frac{q+1}{d+2} + \frac{q}{d\log n}.\end{aligned}$$

Finally, for each $1 < i \le t$, the matching M_i consists of matching edges with a cost of $O((\ell_\square n_\square^{-\alpha_i})^q)$. By Lemma E.4, the expected number of matching edges in M_i is at most $\tilde{O}(n_\square^{\frac{d}{2}\alpha_{i-1}+\frac{1}{2}})$; therefore,

$$\mathbb{E}[w(M_i)] = \tilde{O}(n_\square^{\frac{d}{2}\alpha_{i-1}+\frac{1}{2}} \times (\ell_\square n_\square^{-\alpha_i})^q) = \tilde{O}((2\ell_\square)^q n_\square^{1-\frac{q+1}{d+2}}), \tag{43}$$

where the second equality is resulted from as follows.

$$\begin{aligned}\frac{d}{2}\alpha_{i-1} + \frac{1}{2} - q\alpha_i &= \left(\frac{1}{2} - \frac{qr^t(1-\frac{1}{r^{i-1}})}{\beta}\right) + \frac{1}{2} - \left(\frac{q}{d} - \frac{qr^{t+1}(1-\frac{1}{r^i})}{\beta}\right) \\ &= 1 - \frac{q}{d} + \frac{q(r-1)r^t}{\beta} \le 1 - \frac{q+1}{d+2} + \frac{q}{d\log n}.\end{aligned}$$

Combining Equations (42) and (43), the expected cost of M is

$$\mathbb{E}[w(M)] = \sum_{i=1}^{t} \mathbb{E}[w(M_i)] = O((2\ell_\square)^q n_\square^{1-\frac{q+1}{d+2}}).$$

□

Lemma E.4. *For any random partitioning of a set U of $2n$ points inside the d-dimensional unit hypercube into two sets A and B of n points each, a hypercube $\square$ of $\mathcal{H}$, and a grid G inside $\square$ with cell side length $O(\ell_\square n_\square^{-\alpha})$, $\mathbb{E}[\mathrm{exc}(G)] = \tilde{O}(n_\square^{\frac{d}{2}\alpha+\frac{1}{2}})$.*

Proof. For $\alpha \geq \frac{1}{d}$, the lemma statement holds trivially since $n_\square^{\frac{d}{2}\alpha+\frac{1}{2}} \geq n_\square$. Therefore, we assume $\alpha \leq \frac{1}{d}$. Using the Hoeffding's inequality [10], for any hypercube ξ of G,

$$\Pr\left[|B_\square^\xi| - |A_\square^\xi| \geq c_1\sqrt{n_\square log n}\right] \leq n^{-c_2}$$

for some constants $c_1, c_2 > 1$. Since $\mathrm{exc}(\xi) \leq n_\square$,

$$\mathbb{E}[\mathrm{exc}(\xi)] = O(\sqrt{n_\square \log n}).$$

Summing over all squares of G,

$$\mathcal{B}[\mathrm{exc}(G)] = \sum_{\xi\in G}\mathbb{E}[\mathrm{exc}(\xi)] = O\left(\sum_{\xi\in G}\sqrt{n_\square \log n}\right) = O\left(\sqrt{n\log n \times |G|}\right) = \tilde{O}(n^{\frac{d}{2}\alpha+\frac{1}{2}}).$$

□

We use Lemma E.3 to show that the number of points in $B_\square^{\mathcal{C}}$ is $\tilde{O}(n^{1-\frac{1}{d+2}})$. Let M' denote the matching constructed in Lemma E.3, and let M denote the matching of the extended matching $M^{\mathcal{C}}$ maintained by our algorithm. Let $M_\square$ denote the matching edges of M that lie inside $\square$. Let $\mathcal{P}_{\mathrm{aug}}$ (resp. $\mathcal{P}_{\mathrm{alt}}$) denote the set of augmenting paths (resp. alternating paths) with an endpoint in $|B_\square^{\mathcal{C}}|$ in the symmetric difference $M_\square \oplus M'$. As discussed above, $|\mathcal{P}_{\mathrm{alt}}| \leq |F(M')| = O(n^{1-\frac{1}{d+2}})$. We partition the free endpoints of $\mathcal{P}_{\mathrm{aug}}$ into the set $B_{\mathrm{aug}}^{\mathrm{close}}$ that are at a Euclidean distance closer than $\lambda'_\square = 2\ell_\square n^{-\frac{1}{d+2}}$ to the divider $\Gamma_\square$ of $\square$ and the set $B_{\mathrm{aug}}^{\mathrm{far}}$ that are at a Euclidean distance further than $\lambda'_\square$ from $\Gamma_\square$. From Lemma 2.1, $|B_{\mathrm{aug}}^{\mathrm{close}}| = \tilde{O}(n^{1-\frac{1}{d+2}})$. Finally, by Equation (37), $\sum_{b\in B_{\mathrm{aug}}^{\mathrm{far}}} y(b) \leq w(M')$. Since the dual weight of each free point in $B_{\mathrm{aug}}^{\mathrm{far}}$ is at least $(\lambda'_\square)^q$, we get a bound of $\tilde{O}(n^{1-\frac{1}{d+2}})$ on the number of such points.

Therefore, the total number of iterations of the merge and extended Hungarian search processes on each cell $\square$ is $\tilde{O}(n^{1-\frac{1}{d+2}})$. Each iteration of the merge step and the search procedure takes $\tilde{O}(n_\square\Phi(n))$ time; as a result, the total execution time of our algorithm on $\square$ would be $\tilde{O}(n^{1-\frac{1}{d+2}}n_\square\Phi(n))$. Since each point participates in $O(\log(n\Delta))$ cells in $\mathcal{H}$, the total execution time of our algorithm across all cells would be $\tilde{O}(n^{2-\frac{1}{d+2}}\Phi(n)\log\Delta)$, proving Lemma E.2.

References

[1] Pankaj K Agarwal, Alon Efrat, and Micha Sharir. Vertical decomposition of shallow levels in 3-dimensional arrangements and its applications. In *Proc. 11th Annual Sympos. on Computational Geometry*, pages 39–50, 1995.

[2] Sébastien Bubeck, Michael B Cohen, Yin Tat Lee, James R Lee, and Aleksander Mądry. K-server via multiscale entropic regularization. In *Proc. 50th Annual ACM SIGACT Symposium on Theory of Computing*, pages 3–16, 2018.

[3] Li Chen, Rasmus Kyng, Yang P Liu, Richard Peng, Maximilian Probst Gutenberg, and Sushant Sachdeva. Maximum flow and minimum-cost flow in almost-linear time. In *Proc. 63rd IEEE Annual Sympos. Foundations of Computer Science*, pages 612–623. IEEE, 2022.

[4] M. Chrobak and L. L. Larmore. An optimal on-line algorithm for k servers on trees. *SIAM Journal of Computing*, 20(1):144–148, 1991.

[5] Marek Chrobak, H Karloof, Tom Payne, and Sundar Vishwnathan. New results on server problems. *SIAM Journal on Discrete Mathematics*, 4(2):172–181, 1991.

[6] David Eppstein. Dynamic Euclidean minimum spanning trees and extrema of binary functions. *Discrete & Computational Geometry*, 13:111–122, 1995.

[7] Harold N. Gabow and Robert E. Tarjan. Faster scaling algorithms for network problems. *SIAM Journal of Computing*, 18(5):1013–1036, October 1989. ISSN 0097-5397.

[8] Harold N. Gabow and Robert E. Tarjan. Faster scaling algorithms for general graph-matching problems. *Journal ACM*, 38(4):815–853, 1991.

[9] Akshaykumar Gattani, Sharath Raghvendra, and Pouyan Shirzadian. A robust exact algorithm for the Euclidean bipartite matching problem. *Advances in Neural Information Processing Systems*, 36, 2024.

[10] Wassily Hoeffding. Probability inequalities for sums of bounded random variables. *The Collected Works of Wassily Hoeffding*, pages 409–426, 1994.

[11] Haim Kaplan, Wolfgang Mulzer, Liam Roditty, Paul Seiferth, and Micha Sharir. Dynamic planar Voronoi diagrams for general distance functions and their algorithmic applications. *Discrete & Computational Geometry*, 64:838–904, 2020.

[12] E. Koutsoupias. The k-server problem. *Computer Science Review*, 3(2):105–118, 2009.

[13] Elias Koutsoupias and Christos H. Papadimitriou. On the k-server conjecture. *Journal of the ACM*, 42 (5):971–983, 1995.

[14] Harold W Kuhn. The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1-2):83–97, 1955.

[15] James R. Lee. Fusible HSTs and the randomized k-server conjecture. In Mikkel Thorup, editor, *Proc. 59th IEEE Annual Symposium on Foundations of Computer Science*, pages 438–449, 2018.

[16] Richard J Lipton and Robert Endre Tarjan. Applications of a planar separator theorem. *SIAM Journal on Computing*, 9(3):615–627, 1980.

[17] Mark S. Manasse, Lyle A. McGeoch, and Daniel D. Sleator. Competitive algorithms for server problems. *Journal of Algorithms*, 11(2):208–230, 1990.

[18] Krati Nayyar and Sharath Raghvendra. An input sensitive online algorithm for the metric bipartite matching problem. In *Proc. 58th IEEE Annual Symposium on Foundations of Computer Science*, pages 505–515, 2017.

[19] Abhijeet Phatak, Sharath Raghvendra, Chittaranjan Tripathy, and Kaiyi Zhang. Computing all optimal partial transports. In *Proc. 11th Internat. Conference on Learning Representations*, 2022.

[20] Sharath Raghvendra. A robust and optimal online algorithm for minimum metric bipartite matching. In *Approximation, Randomization, and Combinatorial Optimization*, 2016.

[21] Sharath Raghvendra. Optimal analysis of an online algorithm for the bipartite matching problem on a line. In *34th International Symposium on Computational Geometry*, pages 67:1–67:14, 2018.

[22] Sharath Raghvendra and Rachita Sowle. A scalable work function algorithm for the k-server problem. In *18th Scandinavian Symposium and Workshops on Algorithm Theory*, volume 227, 2022.

[23] Sharath Raghvendra, Pouyan Shirzadian, and Kaiyi Zhang. A new robust partial p-Wasserstein-based metric for comparing distributions. In *41st Internat. Conference on Machine Learning*, 2024.

[24] Tomislav Rudec and Robert Manger. A new approach to solve the k-server problem based on network flows and flow cost reduction. *Computers & operations research*, 40(4):1004–1013, 2013.

[25] Tomislav Rudec, Alfonzo Baumgartner, and Robert Manger. A fast implementation of the optimal off-line algorithm for solving the k-server problem. *Mathematical Communications*, 14(1):119–134, 2009.

[26] Tomislav Rudec, Alfonzo Baumgartner, and Robert Manger. A fast work function algorithm for solving the k-server problem. *Central European Journal of Operations Research*, 21:187–205, 2013.

[27] R. Sharathkumar. A sub-quadratic algorithm for bipartite matching of planar points with bounded integer coordinates. In *Proc. 29th Annual Symposium on Computational Geometry*, pages 9–16, 2013.

[28] R Sharathkumar and Pankaj K Agarwal. Algorithms for the transportation problem in geometric settings. In *Proc. 23rd Annual ACM-SIAM Sympos. on Discrete Algorithms*, pages 306–317, 2012.

[29] Pravin M Vaidya. Geometry helps in matching. *SIAM Journal of Computing*, 18(6):1201–1225, 1989.